

Playbooks

Execution Model and Builder Design

Decision-complete snapshot through the deferred human-interaction question

A formal specification of the decisions made during the August 28-29, 2026 design session

Status	Design specification. No engine implementation is authorized by this document.
Captured through	The corrected atomic-session example and the decision to enumerate examples before answering human interaction question 38.
Repository baseline	89678c6ca42266816c49f18413b50ff949e36dd6 on main, inspected 2026-08-29.
Planning artifacts	DEL-603, "Build file-based Playbooks and the Playbook Builder," and child DEL-470.
Source of truth	Direct user decisions in the design session take precedence over earlier issue text, meeting summaries, implementation behavior, and literature analogies.

Contents

1	Executive specification	7
1.1	Product intent	7
1.2	The five evidence classes used here	7
2	Scope, provenance, and authority	8
2.1	Scope of this document	8
2.2	Evidence precedence	8
3	Normative terminology	9
3.1	Core terms	9
3.2	Statements that must remain literally true	10
3.3	Terminology explicitly removed from the model	10
4	Formal execution model	10
4.1	State and artifact valuation	10
4.2	Immutable history DAG	11
4.3	Step definitions, instances, and sessions	11
4.4	Trigger-expression grammar	12
4.5	Bindings, consumption, and freshness	12
4.6	Applicability and always-live reevaluation	13
4.7	Atomic success, failure, and sealing	13
4.7.1	Success	13
4.7.2	Failure or semantic rejection	13
4.7.3	Post-completion activity	14
4.8	Concurrent siblings	14
4.9	Agentic merge	14
5	Process semantics without an authored control graph	15
5.1	No hidden control state	15
5.2	Branching and repetition	15
5.3	Persistent attachment and quiescence	15
6	Artifacts and decision handoffs	15
6.1	Exact names only	15

6.2	Append-only history, mutable current view	16
6.3	Outputs and semantic validation	16
7	Git-backed persistence direction	16
7.1	Locked direction	16
7.2	History publication	16
7.3	Unresolved storage boundary	16
8	Static definition graph and concrete execution history	17
8.1	Derived Playbook dependency graph	17
8.2	Concrete state-history DAG	17
9	Worked examples	17
9.1	Example A: linear RPI	18
9.2	Example B: review and revision	18
9.3	Example C: conditional documentation	18
9.4	Example D: parallel specialists and synthesis	19
9.5	Example E: corrected revision before synthesis	19
9.6	Example F: mechanically triggered, semantically rejected	19
9.7	Example G: human approval inside a step	19
9.8	Example H: manual work after quiescence	20
10	Current Alinery baseline and replacement delta	20
10.1	Current verified behavior	20
10.2	Replacement delta	20
10.3	Current issue scope that should remain unless explicitly revised	21
11	Conceptual data contract, not serialized schema	21
12	Runtime invariants and acceptance properties	22
13	Exhaustive decision ledger	23
14	Rejected and deferred models	26
14.1	Explicitly rejected	26
14.2	Explicitly deferred	27
15	Open decision frontier	27

15.1 Human interaction	27
15.2 Merge candidate and parent semantics	27
15.3 Boolean binding semantics	28
15.4 Failure and retry	28
15.5 State frontier and manual creation	28
15.6 Storage, recovery, and Playbook evolution	28
15.7 Serialization and display	28
16 Implementation gates after design completion	29
17 Literature alignment, without importing its control model	29
18 Design session continuation: explicit lineage and state publication	31
18.1 Independent states, not an active frontier	31
18.2 Human-created continuations are ordinary branches	31
18.3 Cross-state merge and exact parent selection	31
18.4 The state DAG is explicit persisted data	32
18.5 One immutable manifest per successful state	32
18.6 The engine owns the manifest	33
18.7 Derived consumption and freshness	34
18.8 Append-only decision ledger for this continuation	34
19 Motivating exemplar Playbooks	36
19.1 Exemplar 1: customizable RPI-style delivery	36
19.1.1 High-level idea	36
19.1.2 Current Alinery baseline	36
19.1.3 High-level Playbook realization	37
19.1.4 Nuances and low-level questions exposed	38
19.2 Exemplar 2: evidence-backed review	38
19.2.1 High-level idea	38
19.2.2 Current Alinery baseline	39
19.2.3 High-level Playbook realization	39
19.2.4 Nuances and low-level questions exposed	40
19.3 Exemplar 3: one-shot implementation	40
19.3.1 High-level idea	40
19.3.2 Current Alinery baseline	40

19.3.3 High-level Playbook realization	41
19.3.4 Nuances and low-level questions exposed	41
19.4 Cross-example acceptance requirements	41
19.5 Append-only exemplar ledger	42
20 Sources and traceability	42
20.1 Product and repository sources	42
Design checkpoint	44
21 Addendum: history-informed Playbook Builder	45
21.1 High-level idea	45
21.2 Possible first linear realization	45
21.3 Questions this exemplar preserves	45
21.4 Append-only meta-exemplar ledger	46
22 Addendum: four research Playbook graph variations	47
22.1 Recommended handoff cardinality	50
22.2 Who creates a state?	50
22.3 Late grounding under unknown cardinality	51
22.4 V1 synthesis readiness: complete coverage	51
22.5 Atomic Step execution across sequential Sessions	52
23 Authoring-time definitions and runtime materialization	53
23.1 Four representations with distinct owners	53
23.2 Author-written Playbook source	53
23.3 Engine-owned execution and state records	54
23.4 Engine-generated Session context	55
23.5 State to execution to state	55
23.6 Details intentionally left open	56
24 Worked execution: three-pass serial research loop	57
24.1 Four representations instantiated by the serial loop	57
24.2 Author-written Playbook source	57
24.3 Engine-owned execution and state records	60
24.3.1 Identifier legend	60
24.3.2 One record pair expanded	60

24.3.3 All twelve Step execution records	61
24.3.4 Every successor is a full state	62
24.4 Engine-generated Session context	62
24.4.1 Pass 1 contexts	62
24.4.2 Pass 2 contexts	63
24.4.3 Pass 3 contexts	64
24.5 State to execution to state	64
24.5.1 Concrete forward-only lineage	64
24.5.2 Literal handoff contents for the three passes	65
24.5.3 Eligibility after every publication	66
24.5.4 Why the third pass stops	67
24.6 Details intentionally left open	67
25 Worked execution: three-pass parallel appraisal and merge	68
25.1 Four representations instantiated by parallel appraisal	68
25.2 Author-written Playbook source	68
25.3 Engine-owned execution and state records	74
25.3.1 Identifier and artifact legend	74
25.3.2 Discover success freezes one finite expansion	75
25.3.3 One question-Synthesis merge record expanded	75
25.3.4 The final Design merge record expanded	76
25.3.5 All twenty-seven Step execution records	76
25.3.6 Every successor is a full state	77
25.4 Engine-generated Session context	77
25.4.1 Pass 1 contexts	78
25.4.2 Pass 2 contexts	79
25.4.3 Pass 3 contexts	80
25.5 State to execution to state	81
25.5.1 Concrete forward-only DAG	81
25.5.2 Literal handoff contents for all three passes	82
25.5.3 Eligibility and coverage after every publication	84
25.5.4 How the outer loop advances and stops	85
25.6 Details intentionally left open	85
25.7 Collection-driven continuation and a smaller merge contract	86
25.7.1 Full question-Synthesis definition	87

26 Semantic reference algorithm for the Playbook execution engine	89
26.1 Inputs and runtime records	89
26.2 Exact collection grounding and complete-set resolution	92
26.3 Atomic Step execution across sequential Sessions	93
26.4 Validated completion and engine-owned state publication	94
26.5 Collection-driven continuation, quiescence, and invariants	94
26.6 Mechanisms intentionally abstract	95
A Executable Playbook specimens	96
A.1 Linear research to design	96

1 Executive specification

The model in one paragraph

A **Playbook** is a persistent, file-defined process made of configured agentic **step definitions**. For every newly sealed immutable task state, Alinery evaluates each step’s Boolean trigger expression over exact named artifacts and provenance-derived **NewFor** facts. Every applicable step may be attempted. One step instance is executed by exactly one PTY harness session against an isolated input. Success atomically publishes one new immutable state; failure publishes none. Concurrent successes from the same input produce sibling states. An explicit agentic merge step may select multiple parents and publish a multi-parent successor. There is no hidden control cursor, no explicit routing graph, no first-class loop, no goal expression, and no runtime planning search. The Playbook remains live after quiescence and reevaluates when later manual or agentic work creates another state.

1.1 Product intent

Status	Decision
LOCKED	“Playbook” replaces “workflow” as the user-facing product term.
LOCKED	A Playbook represents a preconfigured team of agent sessions cooperating in a known process or configuration.
LOCKED	The process should be powerful and customizable while preserving a clear human mental model and inspectable execution history.
LOCKED	Playbooks replace and generalize the existing workflow and auto-advance behavior rather than creating an unrelated runtime.
REJECTED	A Playbook is not a planner, goal solver, POMDP, arbitrary code program, or BPMN-style routing model.
DEFERRED	Goal-directed planning may be designed later as a distinct capability.

The design is intentionally *process based*. A Playbook describes work that a group of agents and humans generally wants performed. It does not declare a goal state, search for a plan, or prove that a task has succeeded. A branch with no work currently applicable is merely **quiescent**. A human may inspect it, change it, or add work later.

1.2 The five evidence classes used here

LOCKED	Explicitly chosen, affirmed, or forcefully corrected by the user.
DERIVED	A necessary or near-necessary consequence of locked decisions. Derived statements are kept separate so they can be challenged without rewriting history.
CURRENT	Verified behavior or structure in the current repository or live Linear issue. It is not automatically a future Playbook requirement.
OPEN	A semantic or product decision that remains genuinely unresolved. Implementations must not guess.
DEFERRED / REJECTED	Deliberately postponed or explicitly declined. Deferred does not mean secretly required in V1.

2 Scope, provenance, and authority

2.1 Scope of this document

This document captures the execution semantics, artifact model, state history, completion model, concurrency, merge behavior, persistent lifetime, visual derivation, current implementation delta, worked examples, and open decision frontier discussed so far. It also records the corrected and rejected models so they are not accidentally reintroduced later.

It does **not** implement the runtime, finalize the physical Playbook file syntax, change Linear, define the full human interaction experience, or authorize application launch. It is a design checkpoint.

2.2 Evidence precedence

When sources disagree, use this order:

1. Direct decisions from the current design conversation.
2. Later explicit corrections over earlier tentative statements.
3. The live DEL-603 and DEL-470 issue records.
4. Current repository behavior at the inspected commit.
5. The August 28 AI-generated meeting notes.
6. Planning and systems literature used only to sharpen terminology.

The meeting notes are useful evidence that the team chose the name Playbook, a file-based architecture, a builder, and pre-release timing. They are not a transcript and cannot support verbatim speaker attribution or timestamps.

Live planning-record conflict

The live DEL-603 epic still lists “a general graph or loop execution engine” as a first-local-milestone non-goal. The current design session has since selected state-triggered repetition, parallelism, and agentic multi-parent merges as core Playbook behavior. Before implementation, DEL-603 and DEL-470 must be reconciled with this newer design. This document records the conflict but does not mutate Linear.

3 Normative terminology

3.1 Core terms

Term	Status	Normative meaning
Playbook	LOCKED	A static, declarative, persistently attached process definition containing configured step definitions. It has no runtime goal search or hidden control position.
Step definition	LOCKED	The configured transition template: stable identifier, prompt or instructions, harness, model, harness/model settings, Boolean trigger expression, exact named outputs, and enough input-binding declaration to derive consumption.
Step instance	DERIVED	One attempted application of one exact step-definition version to an exact immutable input context and binding.
Session	LOCKED	The PTY harness execution of exactly one step instance. A session is not a state. There is one session per step instance and one step instance per session.
Task state	LOCKED	A complete material version of the task: code, filesystem, Git position and relevant changes, exact current artifact values, ticket inputs, and other durable task facts included by the final state schema. It is immutable once sealed.
Artifact	LOCKED	A durable, exact named file or handoff value visible in a task state. V1 predicates do not infer artifact types or match prefixes, wildcards, or semantic categories.
Artifact version	DERIVED	The exact occurrence/value of a named artifact in an immutable state. The canonical identity scheme remains open.
Trigger expression	LOCKED	Arbitrary Boolean data composed only from Exists , NewFor , AND , OR , NOT , and grouping. Truth means a step should be attempted, not that it must succeed.
Input binding	LOCKED	The complete exact set or tuple of input versions consumed together by a step instance. A changed member produces a distinct binding.
Provenance	LOCKED	Immutable transition evidence identifying the step definition/version, session, exact parents, exact input binding, outputs, result state, and outcome facts needed to reconstruct lineage and derive consumption.
ConsumedBy	LOCKED	A relation derived from immutable successful-transition provenance. It is never an independently mutable freshness flag.
NewFor	LOCKED	A per-consumer, provenance-derived fact that an exact complete input binding has not previously been successfully consumed by that consumer under the eventually chosen provenance scope.
Merge step	LOCKED	An ordinary agentic step whose successful transition has multiple exact parent states selected by the merge agent. It may fail instead of publishing a successor.

Term	Status	Normative meaning
Quiescence	DERIVED	No eligible unattempted work is currently running or triggerable under the attached Playbook. It is not goal achievement or task success.
Loop	LOCKED	A human description of repeated step applicability along an ever-growing immutable history. It is not a runtime object, counter, or cyclic state-history edge.

3.2 Statements that must remain literally true

- A session is not a state.
- A step is a transition specification, not a state.
- Runtime states are inherently independent objects and do not inherit meaning from a hidden phase cursor.
- Concrete states are unnamed and unbounded in number.
- A successful atomic step instance produces at most one successor state.
- A completed session cannot later change its sealed output state.

3.3 Terminology explicitly removed from the model

The following are not V1 execution concepts: destination step, routing rule, first matching edge, split, gateway, exclusive-choice node, routing token, named checkpoint, loop iteration counter, goal state, or plan search. A visual edge may be derived for explanation, but it is not an authored order constraint and not the engine's source of truth.

4 Formal execution model

4.1 State and artifact valuation

Let $\mathcal{N}$ be the set of exact artifact names, $\mathcal{V}$ the set of immutable artifact-version identities, and $\mathcal{W}$ the set of complete material workspace snapshots. A sealed state is modeled as

$$s = (\text{id}_s, w_s, A_s, m_s), \quad (1)$$

where $w_s \in \mathcal{W}$, $A_s : \mathcal{N} \rightarrow \mathcal{V}$ maps each visible exact name to its current version, and m_s is durable state metadata with no hidden Playbook control position. A future optional title may live in m_s , but is empty and semantically inert in V1.

For ordinary single-state evaluation:

$$\text{Exists}(n, s) \iff n \in \text{dom}(A_s). \quad (2)$$

OPEN

The exact state fields, artifact occurrence identity, byte-identical rewrite behavior, rename/deletion semantics, and capture of non-Git metadata remain unresolved. The semantic requirement is a complete immutable version, not a particular storage encoding.

4.2 Immutable history DAG

For a task, let

$$H = (S, R, \Pi) \quad (3)$$

be the state history, where S is the set of sealed states, $R \subseteq S \times S$ is the parent relation, and Π is immutable transition provenance.

For an ordinary successful step instance i with parent p and result s_i :

$$\text{Parents}(s_i) = \{p\}. \quad (4)$$

For a successful merge instance with exact selected parent set Q :

$$\text{Parents}(s_i) = Q, \quad |Q| \geq 2. \quad (5)$$

New states refer only to already sealed parents, so history remains acyclic. What humans call a loop is repeated transition labeling along a longer DAG path:

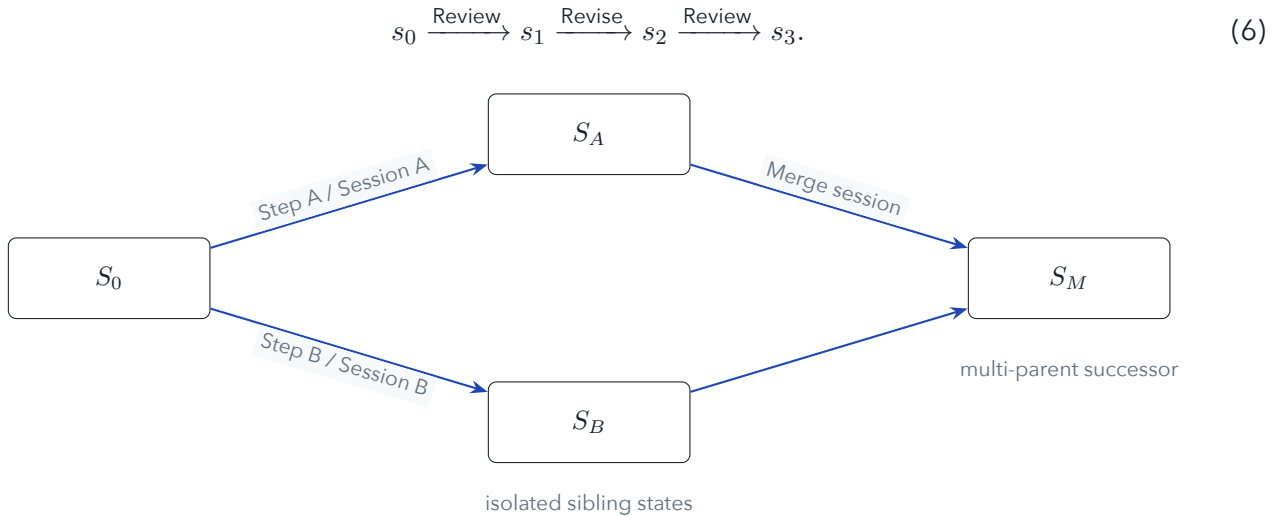


Figure 1: Concrete execution history. Parallel success creates siblings; combination is an explicit agentic transition.

4.3 Step definitions, instances, and sessions

A step definition is conceptually

$$d = (\text{id}, \text{revision}, \text{instruction}, \text{harness}, \text{model}, \text{settings}, \text{trigger}, \text{outputs}, \text{bindingSpec}). \quad (7)$$

The exact serialized fields are not yet locked. The tuple states the semantic information the design already requires. A step instance i binds one exact definition revision to an execution context and one session q_i .

$$\text{SessionOf}(i) = q_i, \quad \text{InstanceOf}(q_i) = i. \quad (8)$$

A running session can make many edits, commits, tool calls, and human-visible messages. Those operations remain inside the isolated attempt. They do not each create Playbook states.

4.4 Trigger-expression grammar

The selected language is arbitrary Boolean logic, not Horn clauses:

$$\begin{aligned} \phi ::= & \text{Exists}(n) \\ & | \text{NewFor}(\text{this_step}, \beta) \\ & | \neg \phi \\ & | (\phi \wedge \phi) \\ & | (\phi \vee \phi), \end{aligned} \quad (9)$$

where n is one exact artifact name and β resolves a complete exact input binding. No arbitrary code, content query, wildcard, type inference, user-defined predicate, rule chaining, or fixed-point inference is implied.

```
NewFor(this_step, ["implementation.md"])
AND NOT Exists("skip-docs.md")

Exists("approved.md")
OR (
  Exists("needs-revision.md")
  AND NewFor(this_step, ["implementation.md", "needs-revision.md"])
)
```

Truth means only: *this configured agentic step should be attempted*. An agent may inspect the full state, determine that it is not semantically adequate, explicitly fail, and publish no successor.

4.5 Bindings, consumption, and freshness

For consumer step definition d , a complete input binding b contains the exact artifact occurrences consumed together. Consumption is not per filename in isolation when the step depends on several inputs.

For successful-transition provenance scope Γ , define:

$$\text{ConsumedBy}(d, b, \Gamma) \iff \exists \rho \in \Gamma : \rho.\text{step} = d \wedge \rho.\text{binding} = b \wedge \rho.\text{outcome} = \text{success}. \quad (10)$$

Then:

$$\text{NewFor}(d, b, \Gamma) \iff \text{ValidBinding}(d, b) \wedge \neg \text{ConsumedBy}(d, b, \Gamma). \quad (11)$$

This makes freshness local to the consumer and the complete binding. Suppose Merge previously consumed $\{\text{tests@2}, \text{docs@1}\}$. The later binding $\{\text{tests@3}, \text{docs@1}\}$ is distinct even though the documentation occurrence did not change.

Single source of truth. ConsumedBy and NewFor are derived from the immutable provenance already required for the history DAG. Alinery does not maintain a second mutable freshness bit, global consumed flag, cursor, deletion convention, or attempt counter.

OPEN The relevant provenance scope Γ is not selected. Ancestry-local scope allows the same binding to be independently consumed on sibling branches; task-global scope lets consumption on one branch suppress another. Exact binding identity and definition-version identity are also open.

4.6 Applicability and always-live reevaluation

For a fixed Playbook revision P , state or candidate context c , and resolved binding b :

$$\begin{aligned} \text{Applicable}_P(d, c, b) \iff & d \in P \wedge \text{BindingValid}(d, c, b) \\ & \wedge \llbracket d.\text{trigger} \rrbracket_{c,d,b} = \text{true} \\ & \wedge \text{ActivationEligible}(d, c, b). \end{aligned} \tag{12}$$

The final definition of ActivationEligible is open for failed attempts and Boolean expressions without a positive NewFor witness. What is locked is that a new full-state identity alone does not rearm every step.

Whenever a new immutable state is sealed, the Playbook reevaluates every step definition. It does not move a stored program counter or traverse an authored outgoing edge. If several applications are eligible, their sessions may run concurrently.

4.7 Atomic success, failure, and sealing

4.7.1 Success

A successful session atomically adds exactly one sealed state and one successful provenance record:

$$(H, \Pi) \xrightarrow{\text{success}(i)} (H \cup \{s_i, \text{Parents}(s_i) \rightarrow s_i\}, \Pi \cup \{\rho_i\}). \tag{13}$$

All promoted workspace and artifact effects become visible together at this semantic boundary. The agent must explicitly signal completion. Idle, process exit, or mere artifact appearance is insufficient. The completion signal is accepted only after the mechanical output contract is satisfied.

4.7.2 Failure or semantic rejection

A failed attempt publishes no state and creates no successful-consumption provenance:

$$(H, \Pi) \xrightarrow{\text{failure}(i)} (H, \Pi). \tag{14}$$

Failure evidence may still be recorded durably outside the successful state DAG; partial workspace changes remain unpromoted and cannot alter the sealed input state.

4.7.3 Post-completion activity

After success, the step instance is over and its result is sealed. Even if its PTY process remains alive, subsequent output cannot mutate the sealed state or silently publish a second successor. New state-producing work requires a distinct step instance and session.

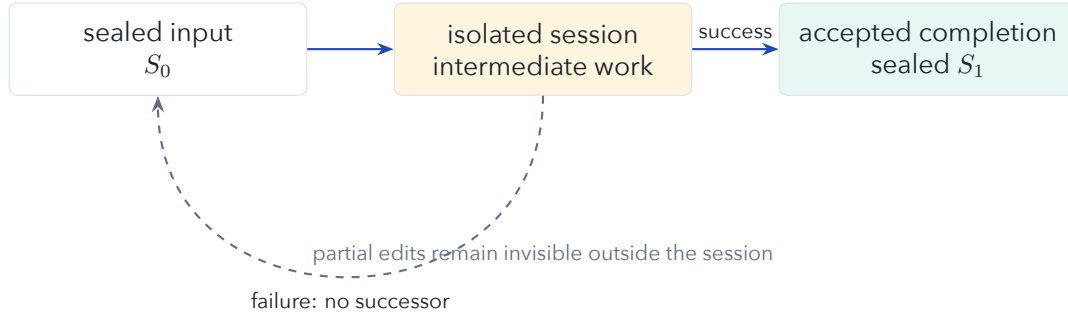


Figure 2: Atomicity is the Playbook visibility boundary. Implementation isolation and cleanup mechanics are still open.

4.8 Concurrent siblings

If two applicable step instances start from the same sealed state s , each receives an isolated view of s . If both succeed, wall-clock completion order does not make one result a descendant of the other:

$$s \xrightarrow{A} s_A, \quad s \xrightarrow{B} s_B, \quad \text{Parents}(s_A) = \text{Parents}(s_B) = \{s\}. \quad (15)$$

The results are full, coexisting sibling states. They do not silently merge. This is the chosen answer to parallel work: difficult combination becomes an explicit agent session with human guidance available through its instructions.

4.9 Agentic merge

The trigger evaluator decides whether a merge step should start. Once started, the merge agent decides which available states are semantically appropriate to combine, or fails. A successful result records exactly the chosen states as parents.

$$\text{Select}_m(C_m, K_m) = Q \subseteq C_m \quad \text{or} \quad \text{Select}_m(C_m, K_m) = \text{fail}, \quad (16)$$

where C_m is the candidate set exposed to the merge session and K_m is the context it can inspect. Neither is yet defined.

The central unresolved merge tension

Ordinary atomic steps begin with fixed exact input. The current merge decision says the agent selects exact parents after the merge session starts. The design must still specify the session's initial isolated workspace, whether candidates are frozen at launch, what artifacts witness the trigger, and which binding is consumed if selected parents differ from trigger witnesses.

5 Process semantics without an authored control graph

5.1 No hidden control state

Given a Playbook and the current immutable material context, Alinery can determine which configured steps should be attempted. The answer must not depend on a stored “current phase,” token marking, routing cursor, first-matching edge, or previously traversed graph edge.

Ordering emerges because successful work changes material state and provenance. For example, Design does not follow Research because an edge says so. It follows because its trigger references an exact Research artifact occurrence that Research can produce.

5.2 Branching and repetition

If several trigger expressions are true, several steps may run. Mutually exclusive behavior arises only if the relevant predicates are mutually exclusive. A decision agent can write one ordinary named marker:

```
approved.md
```

or:

```
needs-revision.md
```

The engine does not enforce that only one exists. If both exist and both downstream expressions are true, both downstream steps may be attempted. The producing agent’s instructions own that semantic discipline in V1.

Repetition arises when a successor contains a distinct binding that makes an earlier step applicable again. No loop object, prefix syntax, version counter, or attempt-count guard is required.

5.3 Persistent attachment and quiescence

A Playbook does not close when it runs out of immediate work. It remains attached to the task. Every new immutable state, including a future supported manual state, wakes reevaluation. Quiescence therefore means only “nothing configured is currently eligible or running.” It does not certify correctness, completion, or goal achievement.

Immutable history makes future rewind and branching possible without deleting later work. The rewind operation itself is deferred.

6 Artifacts and decision handoffs

6.1 Exact names only

V1 trigger predicates refer to exact names such as `design.md`. They do not mean “a Research document,” “the latest file beginning with research-,” or “any artifact of type approval.” This removes implicit selection and keeps the visual and runtime dependency models inspectable.

A logical artifact can retain one stable name across revisions. A successor may overwrite or append to it; immutable ancestor states preserve its earlier values. There is no need to encode the iteration in the filename.

6.2 Append-only history, mutable current view

The state history is append-only even when a successor's current view overwrites or removes a named artifact. The old value remains reachable through its ancestor. Therefore preserving inspectability does not require keeping every historical filename simultaneously visible in the current workspace.

6.3 Outputs and semantic validation

Successful completion normally requires explicit agent completion plus one or more declared exact named outputs or handoffs. Engine validation should remain mechanical: output identity, presence, regular-file constraints, nonempty requirements where declared, session ownership, and state sealing.

Rich quality checks remain agentic. A Structure step may trigger on a fresh `design.md`, read it, identify unresolved questions, and fail. This is not a defect in the trigger. The trigger correctly answered whether Structure should inspect the state.

7 Git-backed persistence direction

7.1 Locked direction

Git is the preferred substrate for complete snapshots, lineage, branching, worktrees, and multi-parent history. Alinery should avoid inventing independent snapshot, diff, restore, merge, and garbage-collection machinery where Git already provides the required mechanics.

Likely mapping:

- each task begins on a task branch and usually a worktree;
- concurrent attempts use isolated worktrees and sub-branches or equivalent private refs;
- successful state publication creates or retains an immutable Git-reachable checkpoint;
- a successful agentic merge records multiple selected parents;
- non-Git task metadata is included through a versioned manifest or other checkpoint record if required.

Only the direction is locked. The exact object/ref strategy is not.

7.2 History publication

Detailed checkpoint or commit history is the default. This matches present Alinery behavior, where agent work commonly creates several commits. Squashing remains an optional setting, especially for large Playbooks or final publication. Squashing is not required by the execution model.

Secret-history risk was explicitly placed outside the Playbook engine's V1 scope. Ordinary PR review and remediation remain responsible, as with current agent-authored Git history.

7.3 Unresolved storage boundary

OPEN The following must be selected before implementation:

- whether every semantic state is exactly one Git commit;

- whether internal commits made during a session are semantic states or hidden implementation detail;
- how uncommitted filesystem state is sealed;
- how artifacts and non-Git metadata are manifested;
- which internal refs keep private objects reachable and garbage-collection safe;
- how detailed history is later projected or squashed into ordinary user history.

8 Static definition graph and concrete execution history

There are two distinct visualizations.

8.1 Derived Playbook dependency graph

For step d , let $\text{Outputs}(d)$ be its declared exact names and $\text{References}(d)$ the exact names mentioned in **Exists** or **NewFor** bindings. A display dependency can be derived:

$$d_u \rightsquigarrow_n d_v \iff n \in \text{Outputs}(d_u) \cap \text{References}(d_v). \quad (17)$$

This edge is explanatory, not executable order. Because expressions include **AND**, **OR**, and **NOT**, a plain edge list is incomplete. The display must eventually preserve Boolean grouping, negative references, multiple producers, and cycles.

8.2 Concrete state-history DAG

The execution view shows actual sealed states, step instances, sessions, exact artifacts, success/failure attempts, and parent lineage. It can be large and is task-specific. Runtime states are never assigned predefined semantic names by the Playbook.

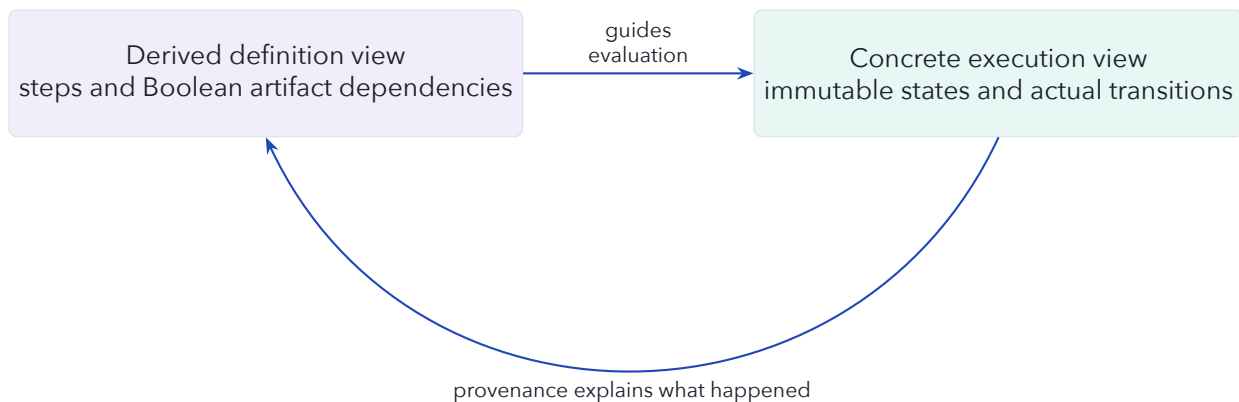


Figure 3: The visual Playbook graph and one task's state DAG are different projections of the same process.

9 Worked examples

These examples are semantic test cases, not a settled file syntax.

9.1 Example A: linear RPI

Step	Illustrative trigger	Declared output
Research	<code>NewFor(this, [ticket.md])</code>	<code>research.md</code>
Design	<code>NewFor(this, [research.md])</code>	<code>design.md</code>
Structure	<code>NewFor(this, [design.md])</code>	<code>plan.md</code>
Implement	<code>NewFor(this, [plan.md])</code>	<code>code changes and implementation.md</code>
Validate	<code>NewFor(this, [implementation.md])</code>	<code>validation.md</code>

Successful trace:

$$S_0 \xrightarrow{\text{Research}} S_1 \xrightarrow{\text{Design}} S_2 \xrightarrow{\text{Structure}} S_3 \xrightarrow{\text{Implement}} S_4 \xrightarrow{\text{Validate}} S_5. \quad (18)$$

The Playbook does not store “ResearchReady” or “DesignReady” states. Each label belongs to the transition that created the next unnamed material state.

9.2 Example B: review and revision

Review examines a fresh implementation and produces exactly one intended marker under its instructions:

```
Review trigger:
  NewFor(Review, ["implementation.md"])

Revise trigger:
  Exists("needs-revision.md")
  AND NewFor(Revise, ["implementation.md", "needs-revision.md"])

PrepareForPR trigger:
  Exists("approved.md")
  AND NewFor(PrepareForPR, ["implementation.md", "approved.md"])
```

If Review requests revision, a distinct Revise session creates a new state and new implementation binding. Review can then become applicable again. The history stays acyclic even though the step label repeats.

9.3 Example C: conditional documentation

```
NewFor(Docs, ["implementation.md"])
AND NOT Exists("skip-docs.md")
```

This exercises arbitrary Boolean composition. The predicate is evaluated against the immutable input context. A later concurrent state cannot retroactively change why the Docs session started.

OPEN An Exists-only or negative-only expression has no obvious positive artifact binding to consume. Its durable activation identity remains unresolved.

9.4 Example D: parallel specialists and synthesis

Security Research and UX Research are both applicable at S_0 . They receive isolated views and complete independently:

$$S_0 \xrightarrow{\text{SecurityResearch}} S_A, \quad (19)$$

$$S_0 \xrightarrow{\text{UXResearch}} S_B. \quad (20)$$

Neither state contains the other's partial work. Synthesis is an agentic merge. It inspects candidate lineage, selects appropriate parents, reconciles their work, and either publishes

$$\text{Parents}(S_M) = \{S_A, S_B\} \quad (21)$$

or fails with no successor.

This example exposes the unresolved evaluation scope: no individual sibling necessarily contains both specialist artifacts, so multi-state *Exists/NewFor* semantics and candidate construction must be defined.

9.5 Example E: corrected revision before synthesis

The following is **not** allowed:

$$S_0 \xrightarrow{A/\text{session } A_1} S_{A1} \quad \text{and later the same completed session silently emits } S_{A2}. \quad (22)$$

Atomic completion sealed S_{A1} . A later revision requires a distinct transition:

$$S_{A1} \xrightarrow{\text{ReviseA/new session}} S_{A2}. \quad (23)$$

Only an explicit new step instance, manual state-producing operation, or separately valid activation can create S_{A2} . If several candidate versions remain live before synthesis, the merge agent's visible candidate set and selection protocol are still open.

9.6 Example F: mechanically triggered, semantically rejected

Structure triggers because a fresh `design.md` exists. Its agent reads the document and discovers unresolved design questions. It explicitly fails. The attempt is recorded, no successor state is published, and no successful consumption provenance is created.

This behavior is intentional. The trigger is a dispatch rule, not a semantic theorem prover.

9.7 Example G: human approval inside a step

The Design agent is instructed to consult a human and withhold completion until approval. The running session, dirty isolated workspace, and conversation remain inside one atomic step. After approval and accepted completion, one successor is sealed.

A standalone human-only step is deferred. A human comment made after the session is sealed cannot change the completed state through that old session; the interaction mechanism is the paused open question.

9.8 Example H: manual work after quiescence

The Playbook becomes quiescent. Later, a supported manual operation creates a new immutable state with a changed relevant artifact. The Playbook remains attached, wakes, and reevaluates all triggers. Only newly eligible bindings may run; a new state identifier alone does not refresh old inputs.

OPEN The exact operation that captures and seals manual filesystem edits is not yet designed.

10 Current Alinery baseline and replacement delta

This section is descriptive evidence from repository commit `89678c6ca42266816c49f18413b50ff949e36dd6`. It is not a claim that the current implementation already satisfies the proposed model.

10.1 Current verified behavior

- The current schema types are `WorkflowFile`, `Workflow`, `WorkflowStep`, and `AutoAdvanceEdge`. They are defined in `alinery-core/src/types.rs`.
- Repository-owned `.alinery/workflows.toml` is parsed and cached by modification time, with bundled defaults.
- Current auto-advance is represented by named `from/to` edges. The task selects edge keys.
- `find_enabled_auto_edge` returns the first selected edge whose `from` equals the completed phase, so the current runtime creates at most one next phase for a source.
- Completion requires an explicit accepted `alinery_phase_complete` event and validation of the expected nonempty regular artifact. Idle, end of turn, and process exit are not completion.
- Auto-advance uses a deterministic target session identifier and exclusive metadata creation to prevent duplicate target sessions across daemons.
- Current task execution uses a task worktree and phase/session metadata, not immutable full-state siblings and multi-parent transitions.
- Bundled definitions include RPI, One-shot, Free-form, Review, and Bug Hunting.

10.2 Replacement delta

Concern	Current workflow behavior	Selected Playbook model
Terminology	Workflow, phase, auto-advance edge	Playbook, step definition/instance, trigger expression, state transition
Ordering	Authored <code>from/to</code> edges	Derived from state predicates, exact outputs, and provenance
Branching	One selected outgoing edge for a completed phase	Every independently applicable step may run
State	One task worktree plus session/phase metadata	Immutable complete task states with explicit lineage
Concurrency	Not a full sibling-state semantic model	Isolated sessions from the same parent yield siblings

Concern	Current workflow behavior	Selected Playbook model
Merge	No general multi-parent agent step	Explicit agentic step selects parents or fails
Repetition	Target phase deduplicated as a one-time session	Repeated definition instances over distinct bindings; no loop object
Freshness	Phase/session identity and target existence	Per-consumer complete binding derived from provenance
Completion	Explicit phase-complete plus expected artifact	Preserve explicit completion; generalize mechanical output contract
Graph	Authored edge graph	Derived dependency view plus concrete history DAG
Lifetime	Primary workflow progression	Persistently attached reactive process

10.3 Current issue scope that should remain unless explicitly revised

The live DEL-603 epic additionally requires file loading and validation shared across app, daemon, and MCP; actionable parse errors rather than silent fallback; Settings to manage Playbooks; raw and form editing; registered-harness authorization; Generic sessions without a Playbook; per-step harness/model guidance; custom board projection; and an agentic Playbook Builder that writes a candidate file for explicit review and installation.

The physical choice between one portable Markdown/YAML-style file and the existing registry plus external Markdown prompts remains open. The epic explicitly says not to add a second parser until the chosen form requires it.

11 Conceptual data contract, not serialized schema

The following records are the minimum conceptual information implied by locked semantics. Field names and serialization are deliberately provisional.

```

PlaybookRevision {
  playbook_id
  revision_id
  metadata
  step_definitions[]
}

StepDefinitionRevision {
  step_id
  revision_id
  prompt_or_instruction
  harness
  model
  harness_settings
  trigger_expression
  binding_specification
  declared_outputs[]
  completion_contract
}

```

```

TaskState {
  state_id
  parent_state_ids[]
  snapshot_reference
  artifact_manifest
  creating_transition_id
  optional_title = ""
}

StepAttempt {
  attempt_id
  step_definition_revision_id
  session_id
  evaluation_context
  proposed_or_resolved_binding
  status
  failure_reason?
}

SuccessfulTransitionProvenance {
  transition_id
  step_definition_revision_id
  session_id
  exact_parent_state_ids[]
  exact_input_binding
  exact_outputs
  result_state_id
  accepted_completion
}

```

Do not freeze these fields by implementation accident

The conceptual records expose information the semantics require. They do not decide the Playbook file syntax, state storage layout, Git mapping, failure-suppression key, merge parent protocol, or client-server wire representation.

12 Runtime invariants and acceptance properties

Any implementation proposal should be rejected if it cannot demonstrate these locked or directly derived properties.

ID	Status	Invariant
I-01	LOCKED	Every sealed task state is immutable and independently inspectable.
I-02	LOCKED	Every successful ordinary step instance has exactly one parent and at most one successor.
I-03	LOCKED	Every successful merge records every and only the exact parent states chosen by its merge agent.
I-04	LOCKED	One step instance corresponds to exactly one session; one session cannot represent two step instances.

ID	Status	Invariant
I-05	LOCKED	Intermediate session mutations are not Playbook states and are withheld from trigger evaluation.
I-06	LOCKED	Failure or semantic rejection creates no successor state.
I-07	DERIVED	Failed partial filesystem changes cannot leak into or rewrite the sealed input state.
I-08	LOCKED	Idle, process exit, or end of turn alone cannot complete a step.
I-09	LOCKED	Successful completion requires an explicit agent completion signal accepted by Alinery.
I-10	LOCKED	Trigger truth means "attempt," not "guaranteed success."
I-11	LOCKED	Concurrent successful instances from the same parent produce sibling states, regardless of wall-clock finish order.
I-12	LOCKED	Concurrent results never combine without an explicit successful merge transition.
I-13	LOCKED	A completed session cannot mutate its sealed result or publish another successor.
I-14	LOCKED	The evaluator uses no hidden control cursor, authored ordering edge, gateway, or first-match rule.
I-15	LOCKED	Every applicable step may run; concurrency follows from independent trigger truth.
I-16	LOCKED	Trigger predicates reference only exact named artifacts and provenance-derived binding freshness.
I-17	LOCKED	Consumption is per consumer and complete input binding.
I-18	LOCKED	ConsumedBy and NewFor have immutable provenance as their sole source of truth.
I-19	LOCKED	Unrelated state identity changes do not automatically refresh an unchanged binding.
I-20	LOCKED	The Playbook remains attached and reevaluates after every newly sealed state.
I-21	LOCKED	Quiescence is not represented as task success or goal satisfaction.
I-22	LOCKED	Runtime states have no predefined semantic names or Playbook control identity.
I-23	LOCKED	Repetition creates new immutable states and step instances; no state-history cycle or loop counter is required.
I-24	LOCKED	Decision-marker exclusivity is not engine-enforced in V1.
I-25	DERIVED	The concrete state history is a DAG even when the derived Playbook dependency view contains cycles.

13 Exhaustive decision ledger

This ledger condenses the chronological elicitation into implementation-reviewable decisions. It intentionally includes rejections so future design work does not reopen them accidentally.

ID	Status	Decision or boundary
D-001	LOCKED	Use "Playbook" as the product term replacing "workflow."
D-002	LOCKED	A Playbook is a preconfigured team/process of agent sessions.
D-003	LOCKED	Optimize for powerful customization and a simple, inspectable human mental model.
D-004	REJECTED	No runtime planning search, goal solving, POMDP model, or arbitrary code control language in V1.
D-005	LOCKED	A state is the complete material task state, not a session or lifecycle label.
D-006	LOCKED	States are immutable complete versions with lineage and remain independently addressable.
D-007	LOCKED	States have no hidden control position or predefined semantic name.
D-008	DEFERRED	Optional state title/description may come later with no V1 semantics.
D-009	LOCKED	A step is a transition specification between states.
D-010	LOCKED	A step definition includes prompt/instructions, harness, model, settings, trigger, and exact outputs.
D-011	LOCKED	One step instance is executed by one PTY harness session, one-to-one.
D-012	LOCKED	A step is atomic at the Playbook visibility boundary.
D-013	LOCKED	Running-session edits remain withheld until the complete contract succeeds.
D-014	LOCKED	Failure produces no successor and preserves the sealed input.
D-015	OPEN	Failed workspace cleanup and durable retry suppression are not specified.
D-016	LOCKED	A completed session cannot later change its result or emit a second result state.
D-017	LOCKED	Several independently applicable steps may run concurrently.
D-018	LOCKED	Concurrent instances receive isolated views of the same input and produce full sibling states.
D-019	REJECTED	Concurrent sessions do not directly mutate one shared output state.
D-020	LOCKED	Combination requires an explicit agentic merge step.
D-021	LOCKED	A successful merge result records all chosen parents, producing a traceable DAG.
D-022	OPEN	The current active frontier and merge candidate-set construction remain undefined.
D-023	LOCKED	Git is the preferred lineage/snapshot substrate; avoid custom snapshot/diff/merge/GC machinery where possible.
D-024	OPEN	Exact Git checkpoint, private-ref, manifest, and uncommitted-state mapping remain unresolved.
D-025	LOCKED	Detailed commit/checkpoint history is default; squashing is optional.
D-026	REJECTED	Secret-history policy is not added to the Playbook engine's V1 responsibilities.
D-027	LOCKED	Idle or process exit never completes a step.
D-028	LOCKED	The agent explicitly requests completion; mechanical declared outputs are validated.
D-029	REJECTED	No rich engine-enforced semantic postconditions in V1.

ID	Status	Decision or boundary
D-030	LOCKED	Agents may reject mechanically eligible states and fail without successors.
D-031	LOCKED	Human approval may occur inside an agent step before atomic completion.
D-032	REJECTED	Standalone human-only steps are excluded from V1.
D-033	LOCKED	Manual and Generic work outside the configured Playbook remains possible.
D-034	OPEN	Human interaction with completed sessions is paused for later examples.
D-035	REJECTED	No explicit authored ordering edges or routing objects in the selected model.
D-036	LOCKED	Given the Playbook and immutable state/context, trigger expressions determine applicable work.
D-037	LOCKED	If several triggers are true, several steps may run; no split/gateway construct is required.
D-038	LOCKED	The displayed graph is derived from predicates and declared outputs, not an execution source of truth.
D-039	LOCKED	A loop is repeated state-based applicability, not a first-class object or counter.
D-040	LOCKED	V1 predicates reference exact named artifacts only.
D-041	REJECTED	No semantic artifact types, prefixes, wildcards, or implicit latest-version lookup.
D-042	LOCKED	Stable artifact names may be overwritten/appended in successors; immutable history preserves prior values.
D-043	REJECTED	Do not delete an artifact merely to mark it consumed.
D-044	REJECTED	No global freshness, global consumed bit, mutable cursor, or separate consumption source of truth.
D-045	LOCKED	Consumption is per consumer and complete exact binding.
D-046	LOCKED	ConsumedBy and NewFor are derived from immutable transition provenance.
D-047	OPEN	Exact artifact occurrence, binding identity, and provenance scope remain undefined.
D-048	LOCKED	Decision agents communicate through ordinary exact named marker artifacts.
D-049	LOCKED	Agent instructions, not the V1 engine, own marker exclusivity.
D-050	DERIVED	If conflicting markers coexist and both triggers are true, both steps may be attempted.
D-051	REJECTED	A step trigger is not limited to one flat conjunction or to Horn clauses.
D-052	LOCKED	Trigger expressions allow arbitrary AND, OR, and NOT over Exists and NewFor.
D-053	LOCKED	Trigger truth means "should attempt," never "will succeed."
D-054	OPEN	Boolean witness/binding semantics for OR, negative predicates, and Exists-only triggers remain unresolved.
D-055	LOCKED	The Playbook stays live and attached after quiescence.

ID	Status	Decision or boundary
D-056	LOCKED	Every newly sealed state wakes reevaluation of all step definitions.
D-057	REJECTED	A new full-state identity alone does not rearm all true steps.
D-058	LOCKED	Quiescence is a process condition, not a goal or success proof.
D-059	DEFERRED	Rewind should be enabled by history later but is not in the current design scope.
D-060	LOCKED	The merge trigger decides when to start; the merge agent decides exact parents or fails.
D-061	OPEN	Merge session context, trigger witnesses, launch snapshot, candidate freezing, and parent declaration protocol remain undefined.
D-062	LOCKED	Same-session post-completion activity cannot explain a second state; a separate transition is required.
D-063	DEFERRED	The specific UX for creating that new manual transition is not decided.
D-064	LOCKED	Playbooks are file-based and should be authorable by an ordinary agent session.
D-065	OPEN	Single portable file versus registry plus prompt files, and the final Boolean AST syntax, remain design work.
D-066	CURRENT	DEL-603 requires shared validation, actionable errors, builder/management UI, registered harnesses, and Generic sessions.
D-067	CURRENT	Current auto-advance uses selected from/to edges and one next phase, not the selected state model.
D-068	OPEN	Playbook revision identity and its effect on old consumption and always-live reevaluation remain undefined.
D-069	OPEN	External side effects outside the isolated filesystem are not covered by atomic state rollback.
D-070	OPEN	Crash recovery, exactly-once attempt claiming, resource limits, and scheduling fairness require implementation design after semantics settle.

14 Rejected and deferred models

14.1 Explicitly rejected

- Session as state; step as state; named phase as concrete task state.
- Required named checkpoints or semantic state nodes.
- Hidden control cursor, stored routing state, authored ordering edges, first-match routing, gateways, splits, or routing tokens.
- First-class loops, loop identity, loop counters, or attempt-count predicates as core control.
- Runtime goal expressions, plan synthesis, search, or partially observable decision models.
- Arbitrary executable trigger code, semantic artifact queries, prefixes, wildcard matching, or implicit artifact types.
- Global freshness, global consumption, deletion-as-consumption, separate mutable consumption records, or counters.

- Trigger truth as semantic validity or success proof.
- Rich engine-enforced semantic postconditions.
- Silent combination of concurrent changes.
- Same completed session producing several successor states.
- Human-only steps in V1.
- Required squashing as default history behavior.

14.2 Explicitly deferred

- First-class human-only steps, regulatory approvals, signatures, and role authorization.
- Human interaction with completed sessions and automatic versus explicit follow-up transitions.
- Rewind UI and operations.
- Optional state titles/descriptions.
- Goal-directed planning and dynamic Playbooks.
- Engine-enforced marker exclusivity.
- Hosted community marketplace, ratings, moderation, and publishing after the local format stabilizes.

15 Open decision frontier

The following are real semantic questions. They must remain visibly open rather than being answered by an implementation shortcut.

15.1 Human interaction

1. If a human messages a completed session, is it read-only until an explicit new session is created, or does the UI create a new manual step instance?
2. What event atomically captures manual filesystem edits as a new state?
3. How is post-completion approval reversal or correction represented while human-only steps remain deferred?

15.2 Merge candidate and parent semantics

1. Which states, artifacts, and lineage are exposed to a triggered merge session: every live branch head, only trigger witnesses, or another derived candidate set?
2. Is the candidate set frozen when the session starts? May the agent select a state produced while it is running?
3. Which isolated workspace does a merge session initially receive before choosing parents?
4. How does the agent declare exact parents and how does Alinery validate them?
5. Does a failed merge reject one candidate binding or the whole visible candidate set?
6. What happens to unchosen live alternatives after success?

15.3 Boolean binding semantics

1. Consider `NewFor(A) OR NewFor(B)`. When both are new, is there one binding, two bindings, or an expression-witness binding?
2. What durable activation identity suppresses repeated `Exists(A)` or `NOT Exists(A)` attempts when unrelated states appear?
3. What does `NOT NewFor(A)` mean when absence and prior consumption can both make `NewFor` false?
4. Is `ConsumedBy` evaluated over ancestry, a branch, the active frontier, or all task provenance?
5. Does changing a Playbook or step-definition revision make an old material binding new for the revised consumer?

15.4 Failure and retry

1. How is a failed but still mechanically applicable binding prevented from auto-retrying forever?
2. How does explicit retry override that suppression without falsely recording successful consumption?
3. Are crash, cancellation, semantic rejection, and infrastructure failure distinct attempt outcomes for retry policy?

15.5 State frontier and manual creation

1. What exact set of immutable states is currently live when several branches coexist?
2. How do branch heads or refs move after ordinary successors and merges?
3. Can ancestor and descendant states both remain live candidates?
4. Which manual operations create states, and can a manual state have multiple parents?

15.6 Storage, recovery, and Playbook evolution

1. Must every semantic state be a Git commit? How are uncommitted and non-Git data sealed?
2. What keeps internal Git objects reachable and garbage-collection safe?
3. What exact Playbook/step revision is captured by a running step instance?
4. Do Playbook edits immediately reevaluate existing live states or wait for the next state event?
5. How are state publication, provenance persistence, and attempt claiming recovered exactly once across crashes?
6. What scheduler capacity and fairness policy limits simultaneous applicable steps without changing their logical applicability?

15.7 Serialization and display

1. What is the portable file format and Boolean-expression AST?
2. How are prompts embedded or referenced while preserving one-file portability where desired?

3. How does the builder explain **AND/OR/NOT**, negative dependencies, and merge candidate requirements?
4. How are the derived definition graph and concrete state DAG presented without conflating them?

16 Implementation gates after design completion

These are not implementation instructions yet. They are evidence the eventual design must make testable.

1. A validated Playbook revision produces the same trigger AST in app, daemon, and MCP consumers.
2. Trigger evaluation is deterministic for a fixed Playbook revision, state/candidate context, binding, and provenance scope.
3. At most one live attempt may claim the same activation key, including across daemon crashes or competitors.
4. No partial session edit changes a sealed state or becomes visible to another isolated sibling.
5. Accepted success persists result state, parent edges, and provenance atomically or recovers to one consistent outcome.
6. Failure persists enough attempt evidence to explain and suppress uncontrolled retries while adding no successor state.
7. A merge cannot claim a parent it was not authorized to inspect and did not declare.
8. Graph rendering is derived from the same parsed trigger/output model used by execution.
9. Invalid Playbooks cannot start sessions and never silently fall back to a different definition.
10. Built-in RPI, Review/revision, Boolean conditional, parallel sibling, merge, failure, manual wake, and post-completion cases are executable conformance traces.

17 Literature alignment, without importing its control model

The selected design uses familiar concepts but is not a direct implementation of any one formalism.

- Classical planning gives precise vocabulary for state, action applicability, and successor transitions, but this Playbook does not search for a plan or declare a goal [1].
- The concrete material state is unbounded because repositories and artifacts are unbounded. “Labelled transition system” or “extended state machine” is more technically accurate than a literal finite-state machine. The product can still visualize finite definitions and finite execution traces [2].
- Long-running harness sessions reinforce the distinction between the duration of execution and the atomic before/after boundary used for Playbook semantics [3].
- PDDL derived predicates use restricted rule semantics and fixed-point evaluation. The selected arbitrary Boolean **Exists/NewFor** expression language is simpler direct evaluation and should not be called Horn clauses or a derived-predicate engine [4, 5].

- Git natively represents immutable trees, commits with parents, branches/refs, and merge commits. The exact mapping remains a storage decision, but Git is the preferred substrate rather than a model Alinery should reimplement [6, 7].

These citations clarify terminology. They do not override product decisions, add control tokens, or imply runtime planning.

18 Design session continuation: explicit lineage and state publication

Status of this append

This section records the design decisions made after the original checkpoint, through question 47. It is intentionally additive: earlier sections have not been reorganized or reconciled. Where an earlier open question conflicts with a locked statement here, this later section is authoritative. Questions explicitly marked open below remain unresolved.

18.1 Independent states, not an active frontier

Every sealed task state remains an immutable, independently valid object. Creating a descendant does not supersede, deactivate, consume, or make its ancestors stale. The V1 model has no active branch, stale branch, stale artifact, leaf-state restriction, or stored frontier. A “branch” is only a path that can be derived from the recorded state DAG.

Ordinary step applicability remains state-local. A session started from S_A receives S_A ; it does not implicitly see artifacts that exist only in sibling S_B . If a Playbook trigger is applicable to several independent states, each state may support an independent step instance. Existing states remain available for later inspection and future transitions.

No supersession rule. Lineage explains how states were produced. It does not select one current truth and does not prune older states from the execution model.

18.2 Human-created continuations are ordinary branches

Questions and conversation may occur at any time without creating a state. If a human asks an agent to perform material work from a sealed state, that work requires a distinct step instance and session rooted at the selected state. Intermediate edits remain isolated. Accepted success seals one new child; failure seals none.

The original state and the human-created child then coexist. The UI should expose this continuation as another visible branch of the history. The Playbook evaluates the new state normally while every other independent state remains valid and independently eligible. No branch is activated or deactivated as a side effect of this interaction.

18.3 Cross-state merge and exact parent selection

Merge is the cross-state exception to ordinary state-local evaluation. Its precondition can search the recorded immutable states for the exact named outputs it requires. Candidate states are the states that can supply those qualifying outputs, wherever they occur in history. Requiring candidates to be leaves would add an unrelated control concept and is rejected.

One configured Synthesis step may succeed from one parent when one state contains all required material, or from several parents when the required material is distributed. “Merge” describes the multi-parent execution case, not a separate authored split, gateway, or routing node.

The candidate set is context, not lineage. The merge session inspects the available candidates and, on completion, declares the exact subset it actually selected. Only that validated subset becomes the parent set of its successor:

$$Q_m \subseteq C_m, \quad \text{Parents}(S_m) = Q_m. \quad (24)$$

Alinery cannot infer Q_m from every state the agent opened, read, or considered, because inspection does not imply selection. If semantic combination is unclear, the merge agent may ask the human for clarification inside the same still-running atomic session. Playbook instructions should tell it when and how to do so. That conversation and any intermediate work publish no state. The session eventually declares a parent subset and completes, or fails.

The exact mechanism for delivering and freezing merge candidates remains implementation work. The settled requirement is that candidate discovery is not restricted to an active or leaf frontier and exact selected parents are declared at accepted completion.

18.4 The state DAG is explicit persisted data

Playbooks must persist their concrete state DAG directly. Parentage is not reconstructed later from Git history, artifact coincidence, session ordering, or which files an agent happened to inspect.

- A genesis state records `parent_state_ids`: `[]`.
- For an ordinary execution started from S_0 , Alinery already knows the execution context and records `parent_state_ids`: `[S0]`.
- For a human-created continuation, Alinery records the sealed state selected when that continuation started.
- For a merge, the agent declares its chosen subset at completion; Alinery validates and records exactly those state identifiers.

Current implementation gap

Current Alinery workflows do not persist this model. They have task metadata, session metadata, a task worktree, mutable artifacts, and a small accepted phase-completion checkpoint. They do not have immutable task-state identifiers, explicit parent arrays, artifact-version identities, exact successful input bindings, or a persisted state DAG. The present `alinery_phase_complete` path advances workflow/session metadata; it does not publish an immutable task-state manifest.

18.5 One immutable manifest per successful state

The minimum durable representation is one engine-generated immutable manifest for every successfully sealed state. The manifest is published when Alinery accepts an explicit step-completion signal, not when a harness exits, becomes idle, stops producing output, or merely writes an artifact. In Playbook terminology, `step_complete` generalizes the current phase-completion event.

The sealing sequence is:

1. Alinery starts an isolated step instance and retains its engine-known context: session identifier, exact step-definition revision, and ordinary parent state. A merge additionally receives candidate-state context.
2. The agent and any participating human perform work inside the session. Code, artifacts, commits, dirty filesystem changes, and conversation remain intermediate and create no Playbook state.

3. The agent emits the explicit completion signal. A merge supplies its selected parent-state identifiers with that structured signal.
4. Alinery prevents a second publication for the same attempt, validates the mechanical output contract, and validates any merge-parent selection against the session's authorized candidates.
5. Alinery creates the internal Git checkpoint that captures committed and dirty work and snapshots or hashes the visible artifact versions. The exact Git object and ref layout remains a storage decision.
6. Alinery generates the state identifier and authoritative manifest fields, then writes the complete manifest atomically.
7. Publication of that manifest is the state-creation commit point. Only afterward does Alinery record accepted success and reevaluate the attached Playbook against the new state.

If validation, checkpointing, artifact capture, or manifest publication fails, no successor state exists and no successful provenance is recorded. A completed session cannot later mutate its sealed result. Further material work requires another step instance and, on success, another manifest.

18.6 The engine owns the manifest

The agent must not write or edit the authoritative manifest. Prompts may explain the completion contract and provide candidate state identifiers, but prefilled bookkeeping is not authoritative provenance. Otherwise lineage correctness would depend on an agent copying engine facts without error.

Alinery mechanically supplies:

- the new state identifier;
- exact parent-state identifiers, using the launch parent for ordinary work and the validated selected subset for merge;
- session and step-definition revision identifiers;
- the Git checkpoint or equivalent snapshot reference;
- exact artifact-version identities and declared outputs;
- the resolved exact input binding;
- the accepted completion timestamp and transition outcome.

The agent supplies only facts that require its semantic judgment: the actual code and artifact contents, the explicit completion signal, and, for a merge, `selected_parent_state_ids`. For V1, every successful state has exactly one creating transition, so the transition provenance can live in the same immutable manifest. A separate successful-transition database is not required by the selected semantics.

The following is a conceptual example, not a locked serialized schema:

```
TaskStateManifest {
  state_id: "S18"
  parent_state_ids: ["S15", "S17"]

  snapshot: {
    git_checkpoint: "3f4c..."
  }

  artifacts: {
    "security.md": "artifact-version-91"
    "ux.md":       "artifact-version-104"
  }

  created_by: {
    session_id: "session-42"
    step_definition_revision_id: "synthesis@7"
    resolved_input_binding: {
      "security.md": "artifact-version-91"
      "ux.md":       "artifact-version-104"
    }
    declared_outputs: ["implementation-plan.md"]
    accepted_completion_at: "..."/>

```

18.7 Derived consumption and freshness

The manifest makes successful transition provenance concrete. It does not add mutable consumed or fresh fields. Given step-definition revision d and exact resolved binding b :

$$\begin{aligned} \text{ConsumedBy}(d, b) &\iff \text{an immutable successful state manifest records} \\ &\quad \text{creating step revision } d \text{ with exact binding } b, \\ \text{NewFor}(d, b) &\iff \text{ValidBinding}(d, b) \wedge \neg \text{ConsumedBy}(d, b). \end{aligned} \tag{25}$$

Thus the state DAG and its embedded successful provenance are explicit persisted facts; **ConsumedBy** and **NewFor** are queries derived from those facts. Exposing a candidate to a merge, inspecting a state, or starting a failed attempt does not itself write successful-consumption provenance.

Still open: binding and **NewFor** surface

The design has not selected where an ordinary step's exact input binding comes from. It may be a separately declared input field, the artifact references inside **NewFor**, or another mechanically resolved specification. The design also has not selected whether authors place **NewFor** explicitly in trigger expressions or whether Alinery automatically enforces one successful execution per exact binding while exposing **NewFor/ConsumedBy** only as derived engine and UI concepts. Boolean witness semantics, provenance scope, and step-revision effects remain open as recorded earlier. No file schema should silently answer these questions.

18.8 Append-only decision ledger for this continuation

ID	Status	Decision or boundary
D-071	LOCKED	Every immutable state remains independently valid; there is no active, stale, leaf-only, or stored frontier semantics.
D-072	LOCKED	Material human-requested follow-up runs as a distinct session from a selected sealed state and may produce an ordinary coexisting child.
D-073	LOCKED	Merge candidate discovery may search all recorded states for required exact named outputs; it is not leaf-restricted.
D-074	LOCKED	One step definition may complete with one parent or several, depending on where its selected inputs reside.
D-075	LOCKED	A merge agent declares the exact chosen parent subset at completion; inspection and candidate exposure do not define parentage.
D-076	LOCKED	A merge agent may seek human clarification inside its still-running atomic session, as guided by its Playbook prompt.
D-077	LOCKED	Every successful state records its parent identifiers explicitly; the task-state history DAG is persisted, not reconstructed.
D-078	LOCKED	Accepted explicit completion publishes exactly one immutable engine-generated state manifest; failure publishes none.
D-079	LOCKED	Alinery owns state identifiers, parent validation, checkpoint and artifact capture, provenance, and atomic manifest publication.
D-080	LOCKED	An ordinary step's parent is mechanically known from its launch state; only merge parent selection requires agent-supplied lineage judgment.
D-081	LOCKED	The merge agent supplies selected parent-state identifiers through structured completion, not by editing the manifest.
D-082	DERIVED	V1 may embed the one successful creating-transition record in its result-state manifest instead of introducing a separate transition database.
D-083	LOCKED	ConsumedBy and NewFor remain derived from immutable successful provenance; they are not mutable flags.
D-084	OPEN	The authoritative source and exact identity of an ordinary step's resolved input binding remain undecided.
D-085	OPEN	Whether NewFor is authored trigger syntax or an automatic once-per-binding activation rule remains undecided.

19 Motivating exemplar Playbooks

Why these examples come before more trigger design

The remaining execution semantics will be evaluated against three Playbooks that are already central to real Alinery use: a customizable RPI-style delivery process, an evidence-backed code review process, and a one-shot implementation process. A proposed trigger, binding, retry, merge, or storage rule is inadequate if it cannot explain these examples naturally. This section therefore records the user-facing process first and deliberately leaves the still-open trigger and binding syntax unspecified.

All three examples are structurally linear in their common path, but “linear” does not mean “automatically advance through every adjacent step.” The current workflows contain deliberate manual boundaries. Preserving those boundaries is part of the motivating behavior, not an incidental limitation of the present implementation.

Each exemplar is described at three levels:

1. the high-level human purpose;
2. the high-level realization in the selected immutable-state Playbook model; and
3. the nuances that the remaining low-level design must eventually explain.

The current baselines below were verified against `alinery-app/src-tauri/workflows.default.toml`, the corresponding bundled prompt files under `alinery-app/src-tauri/workflows/`, and this repository’s materialized `.alinery/workflows.toml` on 2026-08-30. The examples summarize those sources; they do not freeze their current filenames or prose as the final Playbook schema.

19.1 Exemplar 1: customizable RPI-style delivery

19.1.1 High-level idea

This Playbook is for substantial repository work where the team wants to separate understanding, design, planning, verification strategy, implementation, and delivery preparation. A human begins with a task. Configured agent sessions progressively clarify the request, research the current system, develop a design with the human, turn that design into an executable plan and test strategy, implement and verify the change, and prepare an accurate PR handoff.

The essential promise is not merely a list of seven labels. Each step has a distinct responsibility and leaves durable evidence that later work can inspect. A custom version may change prompts, harnesses, models, settings, or selected stages while retaining this staged, inspectable style of collaboration.

19.1.2 Current Alinery baseline

The current bundled workflow declares the following ordered steps and output artifacts:

Step	Responsibility	Current output
Research Questions	Derive the concrete unknowns, decisions, and investigation needed for the task.	01-research-questions.md
Research	Answer those questions with read-only, source-grounded evidence.	02-research.md
Design	Record current behavior, desired end state, architecture, non-goals, patterns, and human-owned design questions.	03-design.md
Structure	Convert a sufficiently resolved design into a phased, verifiable implementation plan.	04-structure.md
TDD	Define the verification strategy and, when appropriate, create failing tests before implementation.	05-tdd.md
Implementation	Execute the plan, change the repository, run checks, and maintain an evidence-backed work report.	06-implementation.md
PR	Inspect the completed result and prepare truthful PR title, body, changed-file, verification, risk, and follow-up material.	07-pr.md

Its default progression is:

```
Research Questions -> Research -> Design
                        [manual boundary]
                        Structure -> TDD -> Implementation -> PR
```

Five default auto-advance relationships are enabled: Questions to Research, Research to Design, Structure to TDD, TDD to Implementation, and Implementation to PR. Design to Structure is deliberately absent. The current Structure instructions also refuse to publish a plan when design questions remain unresolved. Thus the present workflow combines automatic runs with a human-controlled design boundary.

Current completion is explicit. A session must produce its expected nonempty regular-file artifact and emit accepted phase completion; idle status and process exit do not advance the workflow.

19.1.3 High-level Playbook realization

The seven current phases become seven configured step definitions. The common successful trace is a chain of immutable full task states:

```
S0 --ResearchQuestions--> S1 --Research--> S2 --Design--> S3
S3 --Structure-----> S4 --TDD-----> S5
S5 --Implementation-----> S6 --PreparePR-> S7
```

Each arrow is one step instance executed by one isolated agent session. The arrow label names the

transition; the resulting state is an unnamed material snapshot. Accepted completion atomically publishes one child manifest containing the full resulting workspace, exact artifact versions, its ordinary parent, the executing session and step-definition revision, and whatever exact input provenance the final binding design requires.

Ordering must arise from material applicability rather than a phase cursor or traversed from/to edge. Later sessions receive exact artifact versions from their selected parent state instead of scanning a mutable task directory for the “latest/highest-numbered” matching file.

The design boundary remains visible without yet choosing its serialized mechanism. The system must support the human and Design agent resolving questions before Structure legitimately proceeds. That may occur inside a still-running agentic step or through a later material continuation, subject to the general human-interaction rules already selected.

TDD and Implementation demonstrate why a state is more than a handoff document. TDD may add tests, and Implementation changes product code, commits, dirty filesystem contents, and earlier planning material. Successful completion seals the entire resulting task state.

19.1.4 Nuances and low-level questions exposed

1. **Human design approval.** What exact material event makes Structure eligible after a design discussion, while preserving the deliberate human boundary?
2. **Revised upstream work.** If the human revises the design after Structure or revises the plan after TDD, which downstream steps should run again?
3. **Multi-input work.** TDD consults the plan, research, and ticket. Implementation consults the plan, tests, design, research, ticket, and optional handoffs. The final model must identify which exact versions constitute the relevant input together.
4. **Semantic rejection.** Structure may start, discover unresolved questions, and fail without publishing a successor. The same unchanged input must not cause an uncontrolled retry storm.
5. **Long-running collaboration.** Implementation may wait for human confirmation of manual checks. Conversation and dirty work remain inside the atomic session until completion is accepted.
6. **Custom stage selection.** A custom RPI-style Playbook may omit TDD, insert validation, or use different agents without requiring a new execution engine or hidden routing model.
7. **Delivery versus publication.** The current PR step prepares evidence and text; it does not inherently prove that a remote PR was opened. A customized publishing step would introduce an external side effect outside local snapshot rollback.

19.2 Exemplar 2: evidence-backed review

19.2.1 High-level idea

Given a specific PR, branch, or diff, this Playbook establishes the exact review target, runs and records appropriate checks, inspects the change for substantive problems, and produces a final review decision. When explicitly authorized, the final session may submit or prepare the external review response.

The split prevents orientation, mechanical evidence, findings, and publication from collapsing into one opaque turn. A request-changes conclusion is not a failed Playbook: accurately finding severe defects and recording them is a successful review result.

19.2.2 Current Alinery baseline

Step	Responsibility	Current output
Review Context	Identify the PR, branch, or diff; record repository state, prerequisites, and unresolved context gaps.	01-review-context.md
Review Checks	Run relevant tests, lint, typechecks, and other checks; preserve exact commands, failures, and unavailable coverage.	02-review-checks.md
Review Findings	Inspect correctness, security, regressions, and maintainability; produce severity-rated findings with evidence.	03-review-findings.md
Review Response	Produce an approve, request-changes, or comment-only response and record external submission evidence when a response was actually submitted.	04-review-response.md

The default progression is:

```
Review Context -> Review Checks -> Review Findings
                                [manual boundary]
                                Review Response
```

Only Context to Checks and Checks to Findings auto-advance by default. Findings to Response is deliberately absent, preserving a decision and publication boundary before any GitHub approval, request-changes review, or comment is submitted.

19.2.3 High-level Playbook realization

```
S0 --ReviewContext--> S1 --ReviewChecks--> S2
S2 --ReviewFindings-> S3 --ReviewResponse-> S4
```

Each accepted step publishes one ordinary single-parent successor. The initial state must identify the review request and the exact code or review-target snapshot. Later sessions receive exact context, check, and finding artifacts from their parent lineage rather than choosing a task-global latest file.

The Findings result can truthfully contain Critical defects, failed tests, or a request-changes recommendation while still being a successful state transition. Step failure is reserved for inability to complete the review work reliably, not for an unfavorable judgment about the code.

The boundary before Review Response remains explicit at this level. The example does not yet choose whether the final session begins through a manual start, an authorization artifact, or another activation mechanism. It does require that sealing findings alone never silently publishes a remote review.

19.2.4 Nuances and low-level questions exposed

1. **Exact review-target identity.** Context must bind the run to an exact base/head revision or equivalent immutable diff. A later PR push must not mutate the input underneath running sessions.
2. **Re-review after a push.** A changed target should permit the appropriate review work to run again. This scenario should help define activation identity without presupposing a `NewFor` syntax.
3. **Successful negative outcomes.** Failed checks and blocking findings are valid outputs, distinct from failure of the Review Checks or Review Findings session itself.
4. **Publication authorization.** Findings completion must not automatically authorize Review Response or a GitHub mutation.
5. **External-effect recovery.** If GitHub accepts a review but local state publication fails, retry must not post the same review twice. Local state atomicity alone cannot solve this.
6. **Cross-task handoff.** Current prompts accept review handoff files. A Playbook must eventually represent that input with immutable provenance rather than as an untracked file read.
7. **Qualified evidence.** Mechanical validation can require the named output, while the agent decides whether unavailable checks, incomplete reproduction, or weak evidence require failure or a qualified conclusion.

19.3 Exemplar 3: one-shot implementation

19.3.1 High-level idea

This Playbook is for a clear, bounded task that does not warrant separate research, design, and planning sessions. One agent reads the task and permitted context, implements the requested change directly, and verifies it. A second focused session may then turn that exact implementation into truthful PR-ready material.

“One-shot” therefore means one direct implementation pass, not necessarily one session for the entire lifecycle.

19.3.2 Current Alinery baseline

Step	Responsibility	Current output
Implementation	Read the ticket, existing artifacts, optional review handoff, and extra instructions; change the worktree; run targeted checks; and record the result.	01-implementation.md
PR	Inspect the exact implementation and code changes, rerun named verification when feasible, and prepare truthful PR material.	02-pr.md

The current workflow declares the order Implementation then PR, but it declares no default auto-advance relationship at all:

```
Implementation
  [manual boundary]
PR
```

Successful Implementation completion therefore does not itself create or launch a PR session in the current system. The user starts that phase separately when it is wanted.

19.3.3 High-level Playbook realization

```
S0 --Implementation--> S1
S1 --PreparePR-----> S2      (when explicitly activated)
```

Implementation begins from the initial state containing the task request and starting source snapshot. One isolated session changes the repository and writes its implementation record. Accepted completion atomically publishes the complete resulting code and artifact state as S_1 .

Prepare PR, when invoked, receives that exact implementation state and publishes a second child containing the PR-preparation record. The example remains linear because each ordinary step has one selected parent; it does not require an authored ordering edge or stored workflow position.

This description deliberately does not decide whether Prepare PR eventually has an automatic material trigger or remains explicitly user-started. Reproducing both configurations is itself a Playbook requirement.

19.3.4 Nuances and low-level questions exposed

1. **Initial activation.** The task ticket normally exists forever. The system needs a simple reason to start Implementation once without launching it again on every unrelated successor state.
2. **Manual optional follow-up.** A user must be able to request Prepare PR after Implementation without introducing a hidden phase cursor.
3. **Retries.** A failed implementation publishes no successor, but the user still needs an explicit, non-looping way to retry the unchanged task state.
4. **Late instructions.** New human direction or a review handoff after success may justify another Implementation attempt. The model must explain which new material makes that attempt distinct.
5. **Exact predecessor.** If several implementation branches coexist, Prepare PR must use an explicitly selected state and artifact version rather than the task-global latest implementation file.
6. **Completion contract.** Implementation changes both source files and a summary artifact. The manifest must seal the full result while mechanical validation remains narrow and predictable.

19.4 Cross-example acceptance requirements

Taken together, the three exemplars require the eventual Playbook design to support all of the following without special-purpose workflow engines:

1. step-specific prompts, harnesses, models, settings, and declared outputs;
2. ordinary linear traces built from immutable full-state successors;

- 3. explicit completion and atomic state-manifest publication;
- 4. automatic stretches and deliberate human or manual boundaries in the same Playbook;
- 5. exact state and artifact-version selection instead of mutable-directory "latest" discovery;
- 6. safe reruns when relevant upstream material changes and suppression when it does not;
- 7. safe explicit retry after failure without an automatic retry storm;
- 8. semantic outcomes such as request-changes that remain successful process results;
- 9. optional external effects with authorization, evidence, recovery, and duplicate prevention; and
- 10. later customization without adding gateways, loop objects, or hidden control state.

Questions intentionally left open

These examples do not yet select authored trigger syntax, exact input-binding syntax, the role of NewFor, retry keys, Playbook-revision scope, manual activation representation, or external-side-effect recovery. They establish the behaviors those decisions must explain. The next design rounds should walk concrete state histories for these examples before selecting a file schema.

19.5 Append-only exemplar ledger

ID	Status	Decision or boundary
D-086	LOCKED	Customizable RPI-style delivery, evidence-backed Review, and One-shot implementation are motivating Playbooks that the V1 semantics must express naturally.
D-087	CURRENT	Current RPI is a seven-step linear workflow with a deliberate manual boundary between Design and Structure.
D-088	CURRENT	Current Review is a four-step linear workflow with a deliberate manual boundary between Review Findings and Review Response.
D-089	CURRENT	Current One-shot is Implementation followed by optional PR preparation, with no default auto-advance relationship.
D-090	LOCKED	Structural linearity does not imply automatic advancement through every adjacent step.
D-091	LOCKED	These exemplars are requirements and diagnostic cases, not permission to answer open trigger, binding, retry, or storage questions implicitly.
D-092	OPEN	The common activation and manual-boundary semantics that reproduce all three examples remain to be derived from concrete traces.
D-093	OPEN	Exact review-target and external-side-effect identity remain unresolved requirements exposed by Review.

20 Sources and traceability

20.1 Product and repository sources

- Direct Playbook design conversation, August 28-29, 2026, through the deferred human-interaction question 38.

- Linear DEL-603, [Build file-based Playbooks and the Playbook Builder](#), read back 2026-08-29.
- Linear DEL-470, [Design the Playbook format, builder, and server-client contract](#), read back 2026-08-29.
- AI-generated August 28 meeting notes supplied by the user, explicitly not a transcript.
- Repository commit 89678c6ca42266816c49f18413b50ff949e36dd6, especially:
 - `alinery-app/src-tauri/alinery-core/src/types.rs`
 - `alinery-app/src-tauri/alinery-core/src/shared.rs`
 - `alinery-app/src-tauri/alineryd/src/main.rs`
 - `alinery-app/src-tauri/workflows.default.toml`
 - `docs/architecture/session-daemon.md`

References

- [1] R. E. Fikes and N. J. Nilsson. "STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving." *Artificial Intelligence*, 2(3-4), 1971. [https://doi.org/10.1016/0004-3702\(71\)90010-5](https://doi.org/10.1016/0004-3702(71)90010-5)
- [2] D. Harel. "Statecharts: A Visual Formalism for Complex Systems." *Science of Computer Programming*, 8(3), 1987. [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)
- [3] M. Fox and D. Long. "PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains." *Journal of Artificial Intelligence Research*, 20, 2003. <https://doi.org/10.1613/jair.1129>
- [4] S. Edelkamp and J. Hoffmann. "PDDL2.2: The Language for the Classical Part of the 4th International Planning Competition." Technical Report 195, University of Freiburg, 2004. https://planning.wiki/_citedpapers/pddl222004.pdf
- [5] S. Thiebaux, J. Hoffmann, and B. Nebel. "In Defense of PDDL Axioms." *Proceedings of IJCAI*, 2003. <https://www.ijcai.org/Proceedings/03/Papers/139.pdf>
- [6] Git project. "git-commit-tree: Create a new commit object." <https://git-scm.com/docs/git-commit-tree>
- [7] Git project. "Git glossary." <https://git-scm.com/docs/gitglossary>

Design checkpoint

What is safe to carry forward

The semantic center is settled: immutable complete task states, configured agentic transitions, one PTY session per step instance, explicit completion, atomic publish, failure without successor, isolated concurrent siblings, explicit agentic multi-parent merge, exact named artifacts, provenance-derived per-consumer bindings, arbitrary Boolean triggers over `Exists/NewFor`, no hidden control graph, no first-class loops, no goal search, and an always-live attached process. The next design work should begin with worked merge and human-interaction traces. It must resolve candidate context, Boolean binding witnesses, failed-attempt suppression, active-frontier semantics, manual state creation, and post-completion session interaction before a file schema or runtime is treated as final.

End of design snapshot. This document intentionally preserves open questions rather than converting them into implementation assumptions.

21 Addendum: history-informed Playbook Builder

A Playbook for discovering Playbooks

A future meta-Playbook should be able to inspect an authorized scope of the immutable Alinery history the user already owns, recognize work the user repeatedly performs, and propose either a new Playbook or a revision to an existing one. Its output is an evidence-linked proposal for human review, never an automatic mutation of the active Playbook set.

21.1 High-level idea

Users often discover a useful process by performing it several times before they can name or formalize it. Alinery's task states, sessions, artifacts, handoffs, retries, and human-created continuations provide a record of that real work. A Playbook Builder could examine those histories for recurring sequences, repeated interventions, and stages the user repeatedly adds by hand, then explain the reusable process it believes is present.

The proposal might recommend a new Playbook, such as a recurring research and design process, or a change to an existing Playbook, such as adding a review step that users consistently create manually. Every recommendation should cite the concrete histories that motivated it so the user can distinguish an intentional practice from coincidence or task-specific variation.

This is meta at the product level but need not require a special execution engine. The Builder itself may be an ordinary linear Playbook whose subject matter and output happen to be other Playbooks.

21.2 Possible first linear realization

```
Select History -> Analyze Repeated Work -> Draft Playbook Proposal  
[manual review boundary]
```

Select History. Establish the user-authorized tasks, states, sessions, and artifacts that the run may inspect.

Analyze Repeated Work. Identify a candidate recurring process, distinguish stable structure from incidental variation, and preserve links to the histories supporting the inference.

Draft Playbook Proposal. Express the candidate as either a new Playbook or a revision, including the proposed steps, expected artifacts, evidence, examples, and unresolved assumptions.

The first version ends at a reviewable proposal. Human discussion, refinement, and eventual adoption remain separate from the analysis that generated it. This addendum deliberately does not choose a pattern-mining algorithm, recurrence threshold, trigger schedule, or automatic installation mechanism.

21.3 Questions this exemplar preserves

1. **Cross-task input scope.** How does a run receive an explicit, authorized view over histories from several tasks without weakening ordinary state isolation?
2. **Evidence and traceability.** What exact state, session, artifact, and human-action references must support a proposal?

- 3. **Intent versus repetition.** How should the Builder distinguish a genuine reusable practice from repeated recovery work, accidental habits, or superficially similar tasks?
- 4. **Proposal versus adoption.** What review boundary allows a user to refine or reject the proposal before any active Playbook changes?

21.4 Append-only meta-exemplar ledger

ID	Status	Decision or boundary
D-094	LOCKED	A history-informed Playbook Builder is a future motivating meta-Playbook: it examines authorized immutable Alinery history and proposes new Playbooks or revisions based on recurring observed work.
D-095	LOCKED	Builder output is a traceable proposal for human review and never automatically mutates the active Playbook set.
D-096	OPEN	The authorized cross-task history scope, evidence threshold, recurrence test, and adoption interaction remain undecided.

22 Addendum: four research Playbook graph variations

How to read these graphs

These figures compare four conceptual material-dependency graphs, not authored routing graphs and not concrete task-state histories. Unbound nodes denote configured step definitions; nodes labeled with explicit question or source bindings illustrate bound instances of one definition. An arrow means that material produced by one step can make another step applicable. A back-edge means that a later immutable state may make an earlier step applicable again; it never points a concrete execution back to an old state. Every actual run still unfolds forward as an acyclic DAG of immutable full task states.

The four views form an incremental semantic ladder: a one-pass chain, one serial refinement cycle, dynamic parallel appraisal with explicit synthesis, and independently progressing question cycles followed by an agentic design merge. Iteration limits, approval, and token budgets remain separate stopping policies and are intentionally absent from the step graphs.



Figure 4: Variation 1: one linear research pass with no recurrence or parallel work. Open questions may be recorded, but they do not activate another pass in this version.

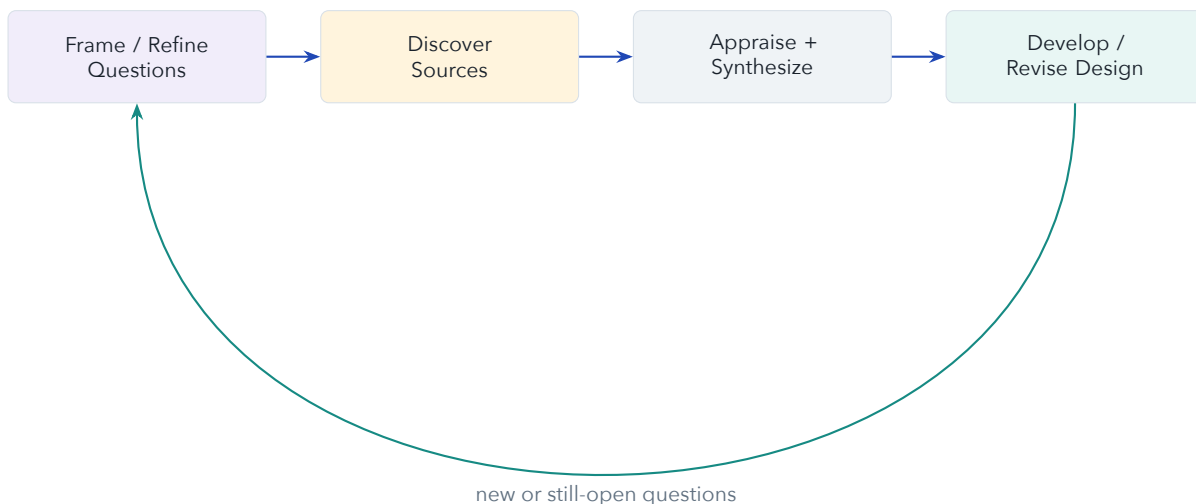


Figure 5: Variation 2: one serial epistemic refinement cycle. The back-edge denotes later instances over new descendant states, not a cycle in concrete state lineage.

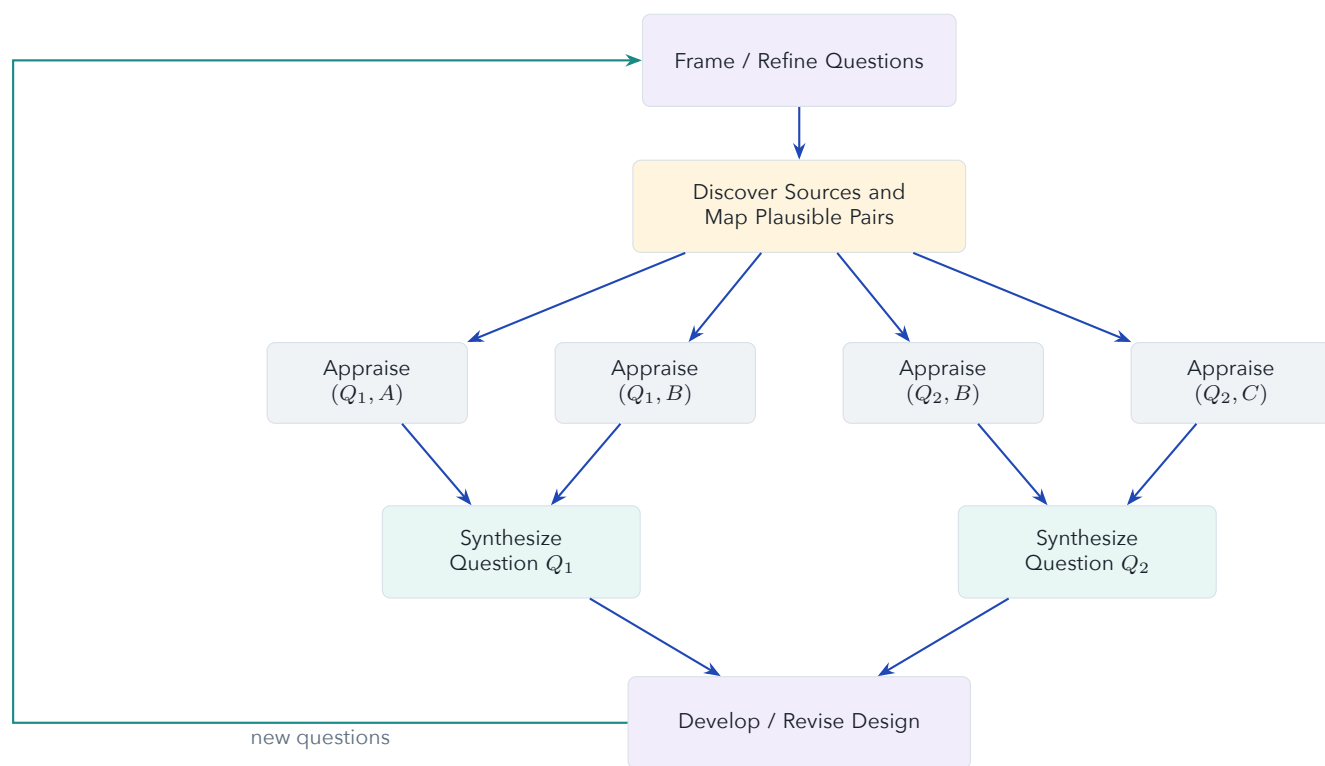


Figure 6: Variation 3: one outer refinement cycle with parallel appraisal instances bound only to plausible question-source pairs. Each question synthesis is an explicit merge; Design merges the selected question syntheses.

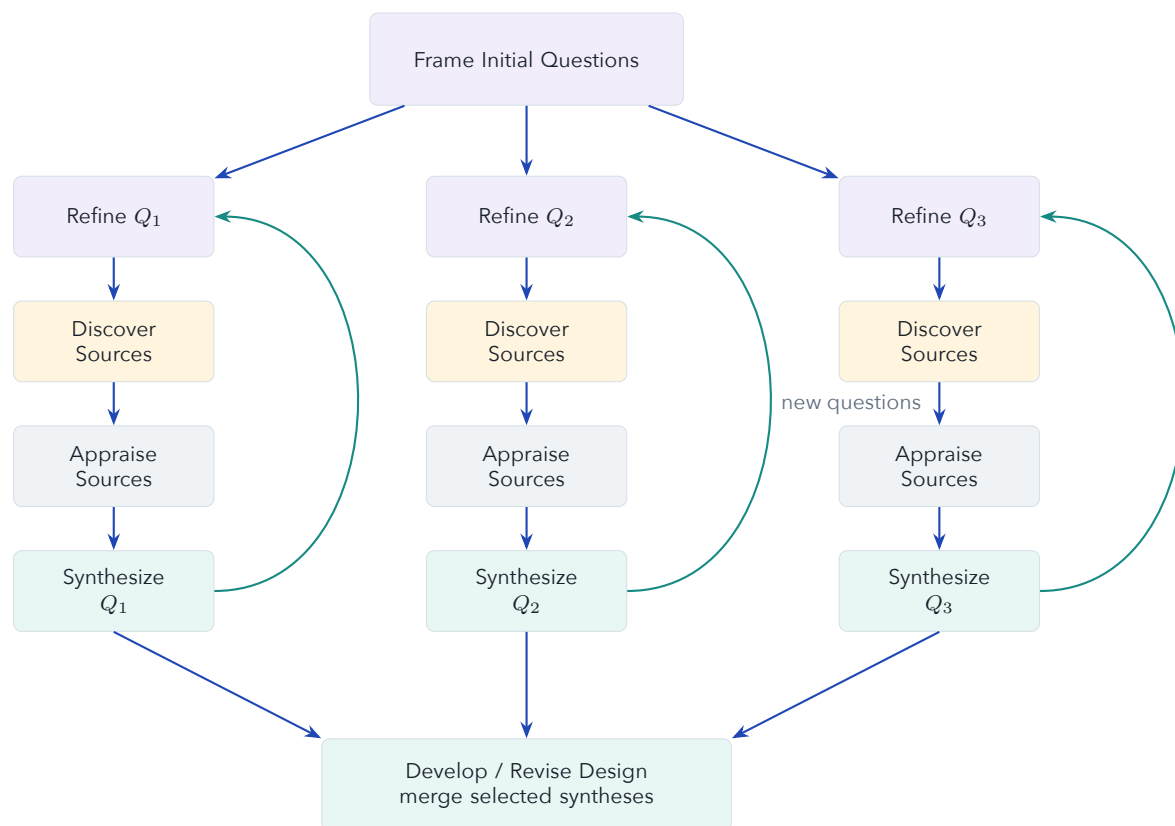


Figure 7: Variation 4: question-bound research cycles progress independently and may complete different numbers of passes. Develop / Revise Design is an explicit agentic merge over the selected question-synthesis results.

22.1 Recommended handoff cardinality

Working handoff convention. Prefer one primary input handoff and one primary output handoff per ordinary step instance. A session may inspect its full input state. Expansion steps may emit a finite collection of independent handoffs, each grounding one downstream step instance. Merge executions select multiple exact handoffs and parent states and normally produce one synthesized handoff.

This is Playbook-authoring guidance, not a universal engine constraint. “Primary” identifies the handoff that focuses the session and may contribute to its complete exact input binding; it is not a visibility boundary over the rest of the immutable input state.

The engine mechanics remain open. Runtime expansion requires a finite collection-enumeration rule beyond the current exact-name grammar. For merge, exact parent-state selection is locked, while the relationship among trigger-witness handoffs, selected parents, and the consumed input binding is not yet settled.

22.2 Who creates a state?

Engine-owned state creation

The engine does, exactly once as part of accepting completion:

1. The agent edits its isolated workspace and writes its handoff files.
2. Those edits remain intermediate.
3. The agent calls `step_complete`.
4. Alinery validates the declared output contract.
5. Alinery checkpoints the filesystem and records exact artifact versions.
6. Alinery writes the immutable state manifest with exact parent-state identifiers, session, step definition, input binding, and outputs.
7. Only then does the state become visible and trigger evaluation resume.

The agent never writes the authoritative state manifest and never explicitly creates child states. It produces work; Alinery seals that work into one state.

22.3 Late grounding under unknown cardinality

The deeper reason is unknown cardinality: Discover determines the useful decomposition. Parallelism is then a natural consequence because each emitted handoff is independently processable.

This gives us a clean separation:

- The Playbook author defines what Appraise does.
- Discover decides how many appraisal units the evidence requires.
- Each handoff grounds one exact Appraise instance.
- The engine controls actual concurrency and capacity.
- The resulting state manifest records the decomposition that actually occurred.

That is essentially **late grounding**: definitions are fixed, but instances are materialized only when the state contains concrete work for them.

Open representation detail

Late grounding is the desired runtime behavior. The portable collection-enumeration syntax and durable instance-claim identity that realize it remain open design questions.

22.4 V1 synthesis readiness: complete coverage

Locked V1 decision. Synthesis becomes eligible only when every handoff in the finite appraisal collection frozen by one exact Discover successor manifest has a successfully completed Appraise result grounded from that handoff. If Discover also freezes explicit question groups, this remains one conservative global readiness barrier; after it is satisfied, each question-specific Synthesis instance binds and selects only the exact covered results in its declared group.

Concretely:

- Discover's completion freezes the finite appraisal set in its successor manifest.
- Each handoff grounds one Appraise input binding; every attempt for that binding is a distinct step instance.
- Any handoff without a successful Appraise result keeps Synthesis ineligible, whether it is unclaimed, has a running attempt, or has only failed attempts.
- Once every collection member is covered, each grounded Synthesis instance receives the exact successful result handoffs and states in its explicitly frozen coverage group as merge inputs.
- Accepted Synthesis completion must select parent states and result handoffs that cover that instance's entire frozen group; otherwise no successor state is published.
- No partial, timeout, quorum, or early-synthesis policy exists in V1.

This rule is more prone to blocking, but not to producing an ambiguous partial synthesis. That is the safer and simpler failure mode. V1 should not add a configurable completion-policy abstraction; one can be introduced later if partial synthesis becomes necessary.

22.5 Atomic Step execution across sequential Sessions

Locked decision. An atomic Step execution may require one or more sequential harness Sessions. Session boundaries are not state boundaries. Every Session shares the Step's exact input binding and isolated workspace. A Session ending without `step_complete` leaves the Step incomplete and continuable. Only accepted `step_complete` publishes the Step's single successor state.

Authoring guidance

Prefer one Session per Step. Authors should design each Step to finish in one harness Session. If work predictably needs several model contexts and has a meaningful intermediate handoff, split it into smaller Steps. Multi-Session continuation is an uncommon escape hatch for atomic work that cannot finish within one Session, not the default decomposition strategy.

The Step remains one atomic transition. Multiple harness invocations are an internal execution detail, not substeps and not intermediate state transitions:

$$S_0 \xrightarrow[\text{Sessions}(e)=\langle q_1, q_2, \dots, q_n \rangle]{\text{one atomic Step execution } e} S_1. \quad (26)$$

The concrete Step execution e is the entity called a step instance earlier in this report. It binds one Step-definition revision to one exact input binding and owns one isolated workspace. Each Session q_j is one bounded PTY harness invocation within that same execution. Every Session participating in a Step execution belongs to exactly one such execution, while one Step execution may contain one or more Sessions in order. Auxiliary Sessions outside Playbooks need not belong to a Step execution.

Continuation works as follows:

- The current Session or the user may request continuation. Accepted `step_continue` yields the current invocation and starts another Session inside the same Step execution.
- The continuation Session receives the original Step instruction plus a continuation instruction and resumes in the same unpublished isolated workspace. It does not reevaluate the Step's trigger, create a new input binding, or begin a retry.
- Sessions within one Step execution are sequential; at most one may be active and write-capable at a time. A Session ending without `step_complete` leaves the execution incomplete and continuable.
- `step_complete` is a blocking atomic operation for its caller and is serialized against continuation. On invocation, Alinery suspends writes from every participating Session and pauses or interrupts any other still-live invocation, then validates, checkpoints, and publishes. Acceptance seals one successor and closes the Step execution. Rejection publishes no state and leaves it continuable.
- An explicit Step failure ends the execution without a successor; retrying afterward is a new Step execution.

Provenance and supersession. Successful provenance records the Step execution and its ordered Session identifiers; Session boundaries are not state-DAG nodes. This supersedes invariants I-04 and D-011 and every earlier one-Session-per-step statement. Each participating Session belongs to exactly one Step execution, while an execution may span several sequential Sessions.

23 Authoring-time definitions and runtime materialization

Working architectural direction

A Playbook author writes reusable logical Step definitions. Alinery combines one of those definitions with exact runtime state to create a unique Step execution, materializes its isolated workspace, and generates the state-specific context given to its harness Sessions. Accepted completion seals a new immutable state and causes Playbook evaluation to run again. Runtime identifiers and Git checkpoints are produced by the engine rather than written into `playbook.md`.

This section begins after the engine has found a potentially eligible Step. It does not yet settle the exact eligibility, automatic-claim, crash-recovery, or retry semantics; it describes how a selected logical definition becomes an execution and, on success, a new state.

23.1 Four representations with distinct owners

Representation	Owner	Purpose
Playbook source	Author	Static, reusable logical configuration: Step identifiers, harnesses, models, settings, state conditions, logical input and output artifact names, and Markdown instruction references.
Step execution record	Engine	One unique attempted application of a configured Step to exact parent state. It records the execution identifier, definition revision, parent state identifiers, participating Sessions, start reason, lifecycle, and eventual result.
Sealed state manifest	Engine	One immutable successful result. It records exact parent states, the publishing Step execution, Git checkpoint, artifact occurrences, declared outputs, and other provenance needed to reconstruct lineage.
Generated Session context	Engine	The ephemeral runtime preamble supplied to a harness Session: its execution, input state and checkpoint, workspace and artifact paths, required outputs, completion operation, and causal predecessor information.

23.2 Author-written Playbook source

The author uses logical names only. The exact field spelling remains provisional, but the separation of responsibilities is the important part:

```
- id: discover-sources
  title: Discover Sources
  harness: codex
  model: gpt-5.6-sol
  settings:
    reasoning_effort: high
  when: 'Exists("research-questions.md")'
  inputs: [research-questions.md]
  outputs: [source-map.md]
  instructions: "#discover-sources"
```

The Playbook source contains neither a concrete state identifier nor an execution identifier. Its input and output names are resolved only when the engine creates a Step execution from a particular sealed state.

23.3 Engine-owned execution and state records

The following is conceptual record notation, not a selected YAML, JSON, or filesystem schema:

```
StepExecution
  id           = exec-18
  step_definition = discover-sources
  parent_states = [state-9]
  input_checkpoint = abc123
  sessions     = [session-31]
  started_by   = automatic
  result_state  = state-10

StateManifest state-10
  parents       = [state-9]
  produced_by   = exec-18
  git_checkpoint = def456
  input_artifacts = [research-questions.md@exec-17]
  output_artifacts = [source-map.md@exec-18]
```

Here @exec-18 denotes a manifest-level artifact occurrence, not a physical filename. An agent continues to read and write source-map.md. When an accepted Step execution publishes that declared artifact, Alinery records a new occurrence associated with the publishing execution even if the logical name already existed or its bytes did not change. The content hash may therefore remain equal while the publication occurrence and provenance are new.

For an ordinary execution, logical input names resolve to the artifact occurrences contained in its one exact parent state. The author and agent do not select versions manually. A merge execution generalizes the same rule to its exact selected parent-state set.

23.4 Engine-generated Session context

Before launching a harness Session, the engine combines runtime facts with the Step's authored Markdown instructions. A generated preamble may have the following shape:

```
You are executing Discover Sources.

Execution ID: exec-18
Input state: state-9
Input Git checkpoint: abc123
Workspace: /exact/isolated/workspace
Artifacts: /exact/artifact/directory

Primary inputs:
- research-questions.md

Required outputs:
- source-map.md

The input state was produced by:
- Frame / Refine Questions, execution exec-17

Call step_complete when the required output is ready.
```

The engine then appends the `discover-sources` instruction section from the Playbook. The exact paths, identifiers, checkpoint, and predecessor information are therefore accurate for this execution without making the Playbook definition task-specific. A continuation Session receives the same execution and input-state context, the original authored instructions, and an additional continuation instruction.

23.5 State to execution to state

```
Playbook definition + sealed parent state(s)
  |
  v
  evaluate eligible Steps
  |
  v
  create unique Step execution
  |
  v
  materialize workspace from exact checkpoint
  |
  v
  inject runtime context + authored instructions
  |
  v
  agent works and calls step_complete
  |
  v
  publish output occurrences and seal state
  |
  v
  evaluate Playbook against new state
```

The runtime contract is directionally:

1. Evaluation reads only sealed states and immutable provenance.
2. The scheduler atomically claims an automatic execution so it cannot accidentally launch a duplicate for the same Step and parent context.
3. A human may explicitly rerun the same Step from the same parent state. Each requested rerun receives a new execution identifier and isolated workspace and may publish a sibling successor state.
4. The engine materializes the exact parent checkpoint and supplies normal workspace and artifact paths to the Session.
5. Edits made during execution may be inspected by the human and participating Sessions, but they cannot satisfy another Playbook Step until accepted completion seals them into a successor state.
6. On accepted `step_complete`, every artifact published by that execution receives a new occurrence associated with its execution identifier, whether it was created, appended to, overwritten, or reaffirmed without a byte change.
7. The engine checkpoints the full resulting workspace, writes the immutable state manifest, and only then exposes the successor to Playbook evaluation.
8. Independent eligible executions may begin from the same sealed state and run concurrently. An execution that needs an output of another cannot see that output occurrence until its producer succeeds.

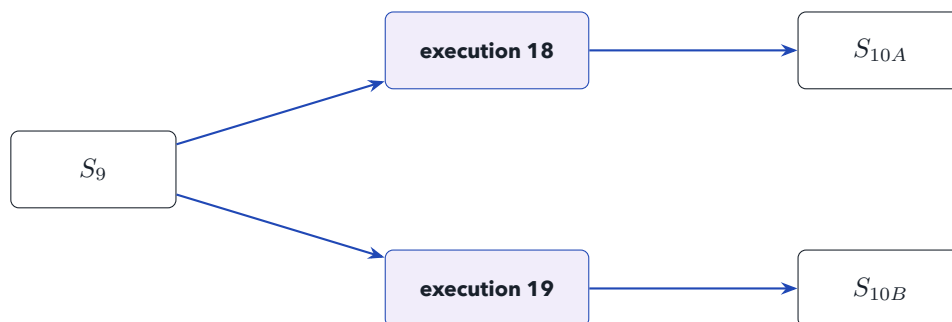


Figure 8: An explicit rerun from the same parent is an intentional fork: each unique Step execution may publish a different sibling successor.

23.6 Details intentionally left open

This direction does not yet select the on-disk serialization or path of execution records and state manifests, the exact Git checkpoint mechanism, the projection of artifact occurrences into a Session workspace, the representation of direct human-produced states, the exact durable automatic-claim key, crash-and-retry behavior, optional-output publication, merge workspace materialization, the explicit-rerun interface, or the final generated-prompt template. Those are implementation choices to refine against the exemplar Playbooks. The separation among authored definitions, engine-owned runtime materialization, and immutable state publication is the working architectural commitment.

24 Worked execution: three-pass serial research loop

Variation 2 made concrete

This section instantiates the serial refinement graph from Figure 5. The example completes three passes through Frame, Discover, Appraise, and Design. The first two Design executions publish a new `research-agenda.md` occurrence, which makes Frame applicable again. The third publishes `research-complete.md` but no new agenda occurrence. The absence of a new agenda leaves Frame ineligible, so the Playbook becomes quiescent. The completion marker documents the agent's decision; it is not an engine stop signal. The three pass labels are explanatory only: the engine stores no loop object, loop identifier, iteration counter, or backward state edge.

The example uses a single primary handoff per ordinary Step. The initial state contains a seed `research-agenda.md`. A continuing Design execution replaces that logical file with a newly published occurrence in its successor state. A final Design execution leaves the last agenda occurrence inherited but does not republish it.

24.1 Four representations instantiated by the serial loop

Representation	Owner	Concrete form in this example
Playbook source	Author	Four reusable Step definitions and their Markdown instructions in one file. No pass number, state identifier, execution identifier, or Git checkpoint appears in the source.
Step execution records	Engine	Twelve records, <code>exec-01</code> through <code>exec-12</code> , each binding one Step revision to one exact primary artifact occurrence and one parent state.
Sealed state manifests	Engine	Thirteen full states, <code>state-0</code> through <code>state-12</code> . Every successful execution publishes one successor with an explicit parent, checkpoint, full current artifact map, and creating provenance.
Generated Session contexts	Engine	Twelve runtime preambles containing the exact execution, state, checkpoint, workspace, primary handoff occurrence, predecessor, output contract, and completion operation for that Session.

24.2 Author-written Playbook source

The following is the complete literal source used by this worked trace. The `sketch-0` label is deliberate. The `inputs`, `required`, `possible`, and authored `NewFor` spellings make this example executable on paper without claiming that the final file schema has been selected.

```

1 ---
2 schema: alinery.playbook/sketch-0
3
4 playbook:
5   id: serial-research-refinement
6   title: Serial Research Refinement
7   revision: 1
8   description: >-
9     Repeatedly frame research questions, discover sources, appraise evidence,
10    and revise a design until no material research question remains.
11
12 steps:
13   - id: frame-questions
14     title: Frame / Refine Questions
15     revision: 1

```

```

16   harness: codex
17   model: gpt-5.6-sol
18   settings:
19     reasoning_effort: high
20   when: 'NewFor(this, ["research-agenda.md"])'
21   inputs: [research-agenda.md]
22   outputs:
23     required: [research-questions.md]
24   instructions: "#frame-questions"

26 - id: discover-sources
27   title: Discover Sources
28   revision: 1
29   harness: codex
30   model: gpt-5.6-sol
31   settings:
32     reasoning_effort: high
33   when: 'NewFor(this, ["research-questions.md"])'
34   inputs: [research-questions.md]
35   outputs:
36     required: [source-map.md]
37   instructions: "#discover-sources"

39 - id: appraise-and-synthesize
40   title: Appraise + Synthesize Evidence
41   revision: 1
42   harness: codex
43   model: gpt-5.6-sol
44   settings:
45     reasoning_effort: high
46   when: 'NewFor(this, ["source-map.md"])'
47   inputs: [source-map.md]
48   outputs:
49     required: [evidence-synthesis.md]
50   instructions: "#appraise-and-synthesize"

52 - id: develop-design
53   title: Develop / Revise Design
54   revision: 1
55   harness: codex
56   model: gpt-5.6-sol
57   settings:
58     reasoning_effort: high
59   when: 'NewFor(this, ["evidence-synthesis.md"])'
60   inputs: [evidence-synthesis.md]
61   outputs:
62     required: [design.md]
63     possible:
64       - research-agenda.md
65       - research-complete.md
66   instructions: "#develop-design"
67 ---

69 # Serial Research Refinement

71 Begin with `research-agenda.md` in the task state. It may contain an initial
72 open-ended problem or the unresolved questions emitted by an earlier design
73 pass. Each successful Step publishes a new immutable task state.

75 The `inputs`, `required`, `possible`, and authored `NewFor` spellings in this
76 specimen are provisional schema. They make the worked execution unambiguous;
77 they do not freeze the final Playbook file format.

79 ## Process

81 ```text
82 research-agenda.md
83   |
84   v
85 Frame / Refine Questions -> research-questions.md
86   |
87   v
88 Discover Sources -> source-map.md
89   |
90   v
91 Appraise + Synthesize Evidence -> evidence-synthesis.md
92   |
93   v
94 Develop / Revise Design -> design.md
95   |
96   +-- research-agenda.md --> another pass
97   |
98   +-- research-complete.md; no new agenda --> quiescent
99   ```

101 The apparent loop is repeated application of the same Step definitions to new
102 artifact occurrences in descendant states. Concrete state lineage never points
103 backward. `research-complete.md` records the agent's decision; the engine
104 becomes quiescent because that execution publishes no new
105 `research-agenda.md` occurrence.

107 <a id="frame-questions"></a>
108 ## Frame / Refine Questions

110 Read `research-agenda.md` as the primary handoff. Inspect the current design,
111 prior evidence, and the rest of the input state when useful. Turn the agenda

```

```

112 into a focused set of questions for this pass. Do not search for sources or
113 answer the questions in this Step.

115 Write `research-questions.md` with:

117 1. The design decision currently being informed.
118 2. Stable question identifiers such as `Q1`, `Q2`, and `Q3`.
119 3. For each question:
120     - the exact question;
121     - why answering it matters now;
122     - what an adequate answer must establish;
123     - important scope boundaries;
124     - assumptions that require evidence;
125     - its relationship to questions from earlier passes.
126 4. Questions retired, split, combined, or carried forward, with a reason.
127 5. A concise research scope for the current pass.

129 Call `step_complete` only when `research-questions.md` is precise enough for a
130 different agent to discover sources without reconstructing the design problem.

132 <a id="discover-sources"></a>
133 ## Discover Sources

135 Read `research-questions.md` as the primary handoff. Find plausible sources for
136 the current questions and map each source to the question or questions it may
137 inform. Use earlier source maps to avoid redundant searching, but write a new
138 current `source-map.md` occurrence for this pass.

140 Write `source-map.md` with:

142 1. The current question identifiers.
143 2. Search terminology, scope, and exclusions.
144 3. Sources retained from earlier passes because they remain relevant.
145 4. Newly discovered candidate sources.
146 5. For every candidate:
147     - title and author or issuing organization;
148     - publication venue and date when available;
149     - DOI, canonical URL, or other stable locator;
150     - source type;
151     - questions it may inform;
152     - why it appears plausibly relevant;
153     - access or retrieval limitations.
154 6. Questions for which no plausible source was found.

156 This Step discovers candidates. It does not claim that a source is credible,
157 sufficient, or that it answers a question.

159 Call `step_complete` only when `source-map.md` gives the next agent a
160 retrievable and question-indexed evidence set.

162 <a id="appraise-and-synthesize"></a>
163 ## Appraise + Synthesize Evidence

165 Read `source-map.md` as the primary handoff. Retrieve and appraise the mapped
166 sources, then synthesize the current evidence question by question. Inspect
167 `research-questions.md`, prior evidence, and the rest of the input state for
168 context. Distinguish findings from this pass from conclusions carried forward
169 from earlier passes.

171 Write `evidence-synthesis.md` with:

173 1. The appraisal method and important limitations.
174 2. A section for every current question identifier.
175 3. For each question:
176     - status: `answered`, `partially answered`, or `unanswered`;
177     - the best-supported current answer;
178     - evidence for each material claim;
179     - source relevance, credibility, and limitations;
180     - agreements, contradictions, and meaningful absences;
181     - uncertainty and unresolved assumptions;
182     - remaining or newly exposed research questions.
183 4. Changes from the previous synthesis.
184 5. Findings sufficiently supported to inform the next design revision.

186 Do not force a clean answer when the evidence remains incomplete or
187 contradictory.

189 Call `step_complete` only when `evidence-synthesis.md` makes both the supported
190 conclusions and remaining uncertainty explicit.

192 <a id="develop-design"></a>
193 ## Develop / Revise Design

195 Read `evidence-synthesis.md` as the primary handoff. Apply the supported
196 findings to the current design. If `design.md` already exists, revise it rather
197 than discarding useful earlier decisions.

199 Write `design.md` with:

201 1. The problem and current design objectives.
202 2. The current proposed design.
203 3. Material decisions and the evidence or explicit assumption behind each.
204 4. Changes made during this pass and why.
205 5. Alternatives considered.
206 6. Risks, limitations, and consequences of being wrong.
207 7. Remaining design questions.

```

```

208 8. Recommended next actions.
210 Then make exactly one research-continuation decision:
212 - If material research questions remain, publish a new `research-agenda.md`
213   occurrence. Include only questions whose answers could materially change or
214   validate the design, explain why each remains open, and state what evidence
215   would resolve it.
216 - If no material research questions remain for this design scope, publish
217   `research-complete.md`. Summarize the evidentiary basis, retained uncertainty,
218   and why another research pass is not presently warranted.
220 On a continuing pass, complete with:
222 ```text
223 step_complete(
224   publish = ["design.md", "research-agenda.md"]
225 )
226 ```
228 On the final pass, complete with:
230 ```text
231 step_complete(
232   publish = ["design.md", "research-complete.md"]
233 )
234 ```

```

Listing 1: Complete serial research-refinement Playbook specimen.

24.3 Engine-owned execution and state records

24.3.1 Identifier legend

```

states      = state-0 ... state-12
executions  = exec-01 ... exec-12
sessions    = session-01 ... session-12
checkpoints = checkpoint-000 ... checkpoint-012

```

All twelve executions use Step-definition revision 1, start automatically, use one Session, complete successfully, and publish exactly one successor. This one-Session choice follows the authoring recommendation; the runtime still permits continuation Sessions inside any one atomic execution.

24.3.2 One record pair expanded

The fourth execution and its successor illustrate the complete pairing. The agent writes normal file-names and calls `step_complete`; the engine creates every identifier and provenance field shown here.

```

StepExecution exec-04
  step_definition = develop-design@1
  parent_states  = [state-3]
  input_checkpoint = checkpoint-003
  input_binding   = [evidence-synthesis.md@exec-03]
  sessions        = [session-04]
  started_by      = automatic
  lifecycle        = completed
  started_at       = 2026-08-29T12:03:00Z
  accepted_at      = 2026-08-29T12:04:00Z
  outcome          = success
  result_state     = state-4
  published        = [design.md@exec-04,
                      research-agenda.md@exec-04]

StateManifest state-4
  parents         = [state-3]

```

```

git_checkpoint      = checkpoint-004
artifacts           = [research-agenda.md@exec-04,
                        research-questions.md@exec-01,
                        source-map.md@exec-02,
                        evidence-synthesis.md@exec-03,
                        design.md@exec-04]

created_by          = exec-04
session_ids         = [session-04]
input_binding       = [evidence-synthesis.md@exec-03]
output_contract     = {required: [design.md],
                       possible: [research-agenda.md,
                                   research-complete.md]}

published_outputs   = [design.md@exec-04,
                       research-agenda.md@exec-04]

accepted_at         = 2026-08-29T12:04:00Z

```

24.3.3 All twelve Step execution records

Each row below is one complete successful transition. The occurrence suffix is engine provenance, not part of the physical filename seen by the agent.

The compact occurrence notation used in both ledgers is:

```

G = research-agenda.md      Q = research-questions.md
M = source-map.md          E = evidence-synthesis.md
D = design.md               C = research-complete.md

```

Pass	Execution / Session	Step and states	Exact input binding	Newly published occurrences
1	exec-01 / session-01	Frame; state-0 → state-1	G@seed-01	Q@01
1	exec-02 / session-02	Discover; state-1 → state-2	Q@01	M@02
1	exec-03 / session-03	Appraise; state-2 → state-3	M@02	E@03
1	exec-04 / session-04	Design; state-3 → state-4	E@03	D@04, G@04
2	exec-05 / session-05	Frame; state-4 → state-5	G@04	Q@05
2	exec-06 / session-06	Discover; state-5 → state-6	Q@05	M@06
2	exec-07 / session-07	Appraise; state-6 → state-7	M@06	E@07
2	exec-08 / session-08	Design; state-7 → state-8	E@07	D@08, G@08
3	exec-09 / session-09	Frame; state-8 → state-9	G@08	Q@09
3	exec-10 / session-10	Discover; state-9 → state-10	Q@09	M@10
3	exec-11 / session-11	Appraise; state-10 → state-11	M@10	E@11
3	exec-12 / session-12	Design; state-11 → state-12	E@11	D@12, C@12

24.3.4 Every successor is a full state

Using the same legend, an occurrence such as *G@04* means *research-agenda.md@exec-04*. The table is the full current artifact map for every state, not an output delta. It abbreviates *checkpoint-000* through *checkpoint-012* as *000* through *012*.

State	Parent	Checkpoint / creator	Full current artifact map
state-0	none	000 / seed	<i>G@seed-01</i>
state-1	state-0	001 / exec-01	<i>G@seed-01, Q@01</i>
state-2	state-1	002 / exec-02	<i>G@seed-01, Q@01, M@02</i>
state-3	state-2	003 / exec-03	<i>G@seed-01, Q@01, M@02, E@03</i>
state-4	state-3	004 / exec-04	<i>G@04, Q@01, M@02, E@03, D@04</i>
state-5	state-4	005 / exec-05	<i>G@04, Q@05, M@02, E@03, D@04</i>
state-6	state-5	006 / exec-06	<i>G@04, Q@05, M@06, E@03, D@04</i>
state-7	state-6	007 / exec-07	<i>G@04, Q@05, M@06, E@07, D@04</i>
state-8	state-7	008 / exec-08	<i>G@08, Q@05, M@06, E@07, D@08</i>
state-9	state-8	009 / exec-09	<i>G@08, Q@09, M@06, E@07, D@08</i>
state-10	state-9	010 / exec-10	<i>G@08, Q@09, M@10, E@07, D@08</i>
state-11	state-10	011 / exec-11	<i>G@08, Q@09, M@10, E@11, D@08</i>
state-12	state-11	012 / exec-12	<i>G@08, Q@09, M@10, E@11, D@12, C@12</i>

24.4 Engine-generated Session context

For this trace, the illustrative runtime root is */runtime/serial-research*. The path is exact within the example but does not select a production directory layout. Each packet below is the engine-generated preamble for one Session; the engine appends the matching authored instruction section from Listing 1. The primary input uses a normal filename, while *resolved_occurrence* exposes the immutable provenance that selected that file version. Each preamble states only the completion contract known at launch. The accepted completion calls shown after each pass are later outcome records, not part of the Session context.

24.4.1 Pass 1 contexts

```
[exec-01 / session-01]
step=Frame / Refine Questions (frame-questions@1)
input_state=state-0
input_checkpoint=checkpoint-000
workspace=/runtime/serial-research/exec-01/workspace
artifacts=/runtime/serial-research/exec-01/workspace/artifacts
primary_input=research-agenda.md
resolved_occurrence=research-agenda.md@seed-01
predecessor=human seed seed-01
required_outputs=[research-questions.md]
completion_contract=step_complete(publish=["research-questions.md"])

[exec-02 / session-02]
step=Discover Sources (discover-sources@1)
input_state=state-1
input_checkpoint=checkpoint-001
workspace=/runtime/serial-research/exec-02/workspace
artifacts=/runtime/serial-research/exec-02/workspace/artifacts
primary_input=research-questions.md
resolved_occurrence=research-questions.md@exec-01
predecessor=Frame / Refine Questions, exec-01
required_outputs=[source-map.md]
completion_contract=step_complete(publish=["source-map.md"])

[exec-03 / session-03]
step=Appraise + Synthesize Evidence (appraise-and-synthesize@1)
input_state=state-2
input_checkpoint=checkpoint-002
workspace=/runtime/serial-research/exec-03/workspace
artifacts=/runtime/serial-research/exec-03/workspace/artifacts
primary_input=source-map.md
resolved_occurrence=source-map.md@exec-02
```

```

predecessor=Discover Sources, exec-02
required_outputs=[evidence-synthesis.md]
completion_contract=step_complete(publish=["evidence-synthesis.md"])

[exec-04 / session-04]
step=Develop / Revise Design (develop-design@1)
input_state=state-3
input_checkpoint=checkpoint-003
workspace=/runtime/serial-research/exec-04/workspace
artifacts=/runtime/serial-research/exec-04/workspace/artifacts
primary_input=evidence-synthesis.md
resolved_occurrence=evidence-synthesis.md@exec-03
predecessor=Appraise + Synthesize Evidence, exec-03
required_outputs=[design.md]
possible_outputs=[research-agenda.md,research-complete.md]
instruction=publish exactly one decision handoff
completion_contract=publish design.md plus exactly one possible output

```

Accepted outcomes after pass 1 (not Session context).

```

exec-01 accepted step_complete(publish=["research-questions.md"])
exec-02 accepted step_complete(publish=["source-map.md"])
exec-03 accepted step_complete(publish=["evidence-synthesis.md"])
exec-04 accepted step_complete(publish=["design.md","research-agenda.md"])

```

24.4.2 Pass 2 contexts

```

[exec-05 / session-05]
step=Frame / Refine Questions (frame-questions@1)
input_state=state-4
input_checkpoint=checkpoint-004
workspace=/runtime/serial-research/exec-05/workspace
artifacts=/runtime/serial-research/exec-05/workspace/artifacts
primary_input=research-agenda.md
resolved_occurrence=research-agenda.md@exec-04
predecessor=Develop / Revise Design, exec-04
required_outputs=[research-questions.md]
completion_contract=step_complete(publish=["research-questions.md"])

[exec-06 / session-06]
step=Discover Sources (discover-sources@1)
input_state=state-5
input_checkpoint=checkpoint-005
workspace=/runtime/serial-research/exec-06/workspace
artifacts=/runtime/serial-research/exec-06/workspace/artifacts
primary_input=research-questions.md
resolved_occurrence=research-questions.md@exec-05
predecessor=Frame / Refine Questions, exec-05
required_outputs=[source-map.md]
completion_contract=step_complete(publish=["source-map.md"])

[exec-07 / session-07]
step=Appraise + Synthesize Evidence (appraise-and-synthesize@1)
input_state=state-6
input_checkpoint=checkpoint-006
workspace=/runtime/serial-research/exec-07/workspace
artifacts=/runtime/serial-research/exec-07/workspace/artifacts
primary_input=source-map.md
resolved_occurrence=source-map.md@exec-06
predecessor=Discover Sources, exec-06
required_outputs=[evidence-synthesis.md]
completion_contract=step_complete(publish=["evidence-synthesis.md"])

[exec-08 / session-08]
step=Develop / Revise Design (develop-design@1)
input_state=state-7
input_checkpoint=checkpoint-007
workspace=/runtime/serial-research/exec-08/workspace
artifacts=/runtime/serial-research/exec-08/workspace/artifacts
primary_input=evidence-synthesis.md
resolved_occurrence=evidence-synthesis.md@exec-07
predecessor=Appraise + Synthesize Evidence, exec-07
required_outputs=[design.md]
possible_outputs=[research-agenda.md,research-complete.md]
instruction=publish exactly one decision handoff
completion_contract=publish design.md plus exactly one possible output

```

Accepted outcomes after pass 2 (not Session context).

```

exec-05 accepted step_complete(publish=["research-questions.md"])
exec-06 accepted step_complete(publish=["source-map.md"])
exec-07 accepted step_complete(publish=["evidence-synthesis.md"])
exec-08 accepted step_complete(publish=["design.md","research-agenda.md"])

```

24.4.3 Pass 3 contexts

```
[exec-09 / session-09]
step=Frame / Refine Questions (frame-questions@1)
input_state=state-8
input_checkpoint=checkpoint-008
workspace=/runtime/serial-research/exec-09/workspace
artifacts=/runtime/serial-research/exec-09/workspace/artifacts
primary_input=research-agenda.md
resolved_occurrence=research-agenda.md@exec-08
predecessor=Develop / Revise Design, exec-08
required_outputs=[research-questions.md]
completion_contract=step_complete(publish=["research-questions.md"])

[exec-10 / session-10]
step=Discover Sources (discover-sources@1)
input_state=state-9
input_checkpoint=checkpoint-009
workspace=/runtime/serial-research/exec-10/workspace
artifacts=/runtime/serial-research/exec-10/workspace/artifacts
primary_input=research-questions.md
resolved_occurrence=research-questions.md@exec-09
predecessor=Frame / Refine Questions, exec-09
required_outputs=[source-map.md]
completion_contract=step_complete(publish=["source-map.md"])

[exec-11 / session-11]
step=Appraise + Synthesize Evidence (appraise-and-synthesize@1)
input_state=state-10
input_checkpoint=checkpoint-010
workspace=/runtime/serial-research/exec-11/workspace
artifacts=/runtime/serial-research/exec-11/workspace/artifacts
primary_input=source-map.md
resolved_occurrence=source-map.md@exec-10
predecessor=Discover Sources, exec-10
required_outputs=[evidence-synthesis.md]
completion_contract=step_complete(publish=["evidence-synthesis.md"])

[exec-12 / session-12]
step=Develop / Revise Design (develop-design@1)
input_state=state-11
input_checkpoint=checkpoint-011
workspace=/runtime/serial-research/exec-12/workspace
artifacts=/runtime/serial-research/exec-12/workspace/artifacts
primary_input=evidence-synthesis.md
resolved_occurrence=evidence-synthesis.md@exec-11
predecessor=Appraise + Synthesize Evidence, exec-11
required_outputs=[design.md]
possible_outputs=[research-agenda.md,research-complete.md]
instruction=publish exactly one decision handoff
completion_contract=publish design.md plus exactly one possible output
```

Accepted outcomes after pass 3 (not Session context).

```
exec-09 accepted step_complete(publish=["research-questions.md"])
exec-10 accepted step_complete(publish=["source-map.md"])
exec-11 accepted step_complete(publish=["evidence-synthesis.md"])
exec-12 accepted step_complete(publish=["design.md","research-complete.md"])
```

These are twelve different execution contexts even though the Playbook has only four Step definitions. In particular, exec-01, exec-05, and exec-09 all use the same Frame definition but receive different parent states and different exact research-agenda.md occurrences.

24.5 State to execution to state

24.5.1 Concrete forward-only lineage

The logical graph contains a back-edge, but the concrete history is this ordinary acyclic chain:

```
Pass 1
state-0 --exec-01 Frame-----> state-1
state-1 --exec-02 Discover--> state-2
state-2 --exec-03 Appraise--> state-3
state-3 --exec-04 Design-----> state-4

Pass 2
```

```

state-4 --exec-05 Frame-----> state-5
state-5 --exec-06 Discover--> state-6
state-6 --exec-07 Appraise--> state-7
state-7 --exec-08 Design-----> state-8

Pass 3
state-8 --exec-09 Frame-----> state-9
state-9 --exec-10 Discover--> state-10
state-10 --exec-11 Appraise--> state-11
state-11 --exec-12 Design-----> state-12

```

The displayed pass boundaries do not exist in runtime state. They merely group twelve forward transitions so a human can see three applications of the same four-definition process.

24.5.2 Literal handoff contents for the three passes

The following intentionally compact files are the complete handoff payloads for this worked trace. They demonstrate occurrence and control semantics rather than claiming to be an adequate literature review.

Seed and pass 1.

```

[research-agenda.md@seed-01]
Determine how a state-based Playbook can repeat research work
without introducing a first-class loop construct.

[research-questions.md@exec-01]
Q1. What runtime identity distinguishes new work from a
    persistent artifact that has already been handled?
Q2. How can Design request another research pass without
    requiring the engine to interpret prose?

[source-map.md@exec-02]
Q1 <- planning operators and grounded-action identity
Q1 <- event provenance and idempotent consumption
Q2 <- named completion and decision artifacts

[evidence-synthesis.md@exec-03]
Q1: partially answered. Exact artifact occurrences are a
    promising unit of work, but consumer provenance is open.
Q2: partially answered. A named marker can request another
    pass, but publication semantics need definition.

[design.md@exec-04]
Draft 1: identify every published artifact occurrence by
logical name and producing execution.

[research-agenda.md@exec-04]
R1. Define when an occurrence is new for one Step definition.
R2. Distinguish duplicate suppression from explicit reruns.

```

Pass 2.

```

[research-questions.md@exec-05]
Q3. What provenance makes NewFor consumer-specific?
Q4. How can Design publish a continuation or completion marker
    without inherited files being mistaken for new outputs?

[source-map.md@exec-06]
Q3 <- event-consumer offsets and idempotency-key mechanisms
Q4 <- transaction validation and finite outcome contracts

```

```
[evidence-synthesis.md@exec-07]
Q3: answered for this design pass. Successful provenance can
    record consumption of one exact input occurrence.
Q4: partially answered. Validation must concern occurrences
    published now, not inherited file existence.

[design.md@exec-08]
Draft 2: derive NewFor from immutable successful-execution
provenance and keep explicit human reruns distinct.

[research-agenda.md@exec-08]
R3. Define the publication boundary between inherited files
    and outputs of the current execution.
R4. Confirm how the final pass becomes quiescent.
```

Pass 3.

```
[research-questions.md@exec-09]
Q5. How does explicit publication distinguish an old agenda
    from a newly emitted agenda occurrence?
Q6. In what order do validation, checkpointing, manifest
    writing, and state visibility occur?

[source-map.md@exec-10]
Q5 <- explicit output-set and immutable provenance mechanisms
Q6 <- transactional publication and immutable-log mechanisms

[evidence-synthesis.md@exec-11]
Q5: answered for this trace. Only names in the accepted
    publication set receive new occurrences.
Q6: answered. Validate and checkpoint before atomically
    exposing the successor manifest to evaluation.

[design.md@exec-12]
Draft 3: complete occurrence-specific consumption and atomic
state publication, with no loop object or mutable cursor.

[research-complete.md@exec-12]
No material research question remains open for this scope.
Retained implementation details are documented separately.
```

24.5.3 Eligibility after every publication

Every newly sealed state wakes evaluation of all four definitions. In this trace, exactly one new binding is eligible until the final state:

State	Newly eligible work	Exact reason
state-0	Frame exec-01	$G@seed-01$ is a valid Frame binding and has not been successfully consumed by Frame.
state-1	Discover exec-02	$Q@01$ is new for Discover; Frame has consumed the seed agenda occurrence.
state-2	Appraise exec-03	$M@02$ is new for Appraise; earlier bindings are consumed.
state-3	Design exec-04	$E@03$ is new for Design.
state-4	Frame exec-05	Design published $G@04$, a new Frame binding distinct from the consumed seed occurrence.
state-5	Discover exec-06	$Q@05$ is new for Discover; $G@04$ is now consumed by Frame.
state-6	Appraise exec-07	$M@06$ is new for Appraise.
state-7	Design exec-08	$E@07$ is new for Design.
state-8	Frame exec-09	Design published $G@08$, a new Frame binding distinct from $G@04$.
state-9	Discover exec-10	$Q@09$ is new for Discover; $G@08$ is now consumed by Frame.
state-10	Appraise exec-11	$M@10$ is new for Appraise.

State	Newly eligible work	Exact reason
state-11	Design exec-12	<i>E</i> @11 is new for Design.
state-12	none; quiescent	Design published <i>C</i> @12, not a new agenda. The current <i>G</i> @08 occurrence was already consumed by Frame in exec-09; every other current primary binding was likewise consumed by its downstream Step.

Requirement exposed by the loop. State sealing must distinguish files inherited from the parent from artifacts intentionally published by the current execution. Otherwise the final Design workspace still contains `research-agenda.md`, and the engine could incorrectly assign it a new occurrence and start a fourth pass. An explicit `step_complete(publish = [...])` set is one simple representation of the requirement; the exact API remains open.

24.5.4 Why the third pass stops

state-12 still contains `research-agenda.md@exec-08` as its current agenda file because states are complete snapshots and the final Design execution did not delete it. That occurrence is not stale and is not globally consumed. The precise fact is narrower: Frame revision 1 successfully consumed that exact binding in exec-09. Since exec-12 did not publish a new agenda occurrence, `NewFor` is false for Frame and no fourth automatic execution is created.

All earlier automatic activations exposed by state-0 through state-11 have already been claimed and completed in this trace. Their durable execution and successful-consumption records prevent reevaluation of those preserved states from silently launching duplicates. The exact durable claim key remains open, as Section 23 records; the quiescence statement here is scoped to the twelve automatic activations enumerated above.

The Playbook is quiescent rather than permanently terminated. A later human or agentic execution may publish another `research-agenda.md` occurrence from any selected state, after which ordinary evaluation may make Frame applicable again on that branch.

24.6 Details intentionally left open

This worked trace deliberately chooses one provisional surface so every value can be written down. It does not settle whether `NewFor` appears in authored syntax or is an implicit once-per-binding activation rule; whether `inputs` is the authoritative binding declaration; how possible outputs are serialized; the provenance scope used across sibling branches; failure, retry, claim, and crash recovery; or the physical checkpoint, manifest, and runtime-path layout.

Marker exclusivity remains an agent-instruction responsibility in V1. The engine records the exact output set submitted at accepted completion, but it does not semantically decide whether the agent should have continued or finished. The durable requirement is only that inherited `research-agenda.md` is not silently republished when the final execution publishes `research-complete.md`.

25 Worked execution: three-pass parallel appraisal and merge

Variation 3 made concrete

This section instantiates the parallel-appraisal graph from Figure 6. Three research passes discover four, three, and five question-source pairs respectively. The same five Step definitions materialize as 27 executions and 28 immutable states. Each Discover execution freezes one explicit finite expansion; Appraise executions fan out into isolated sibling states; question Synthesis executions combine exact group results; and Design combines the question syntheses. The concrete history is a forward-only DAG even though the reusable dependency graph also permits another outer pass.

The varying cardinalities are intentional. Pass 1 uses the four pairs drawn in Figure 6. Pass 2 discovers three pairs, including one question with only one useful source. Pass 3 discovers five pairs, including one three-source question. The Playbook source contains none of these counts, pair identifiers, pass labels, runtime state identifiers, or execution identifiers.

25.1 Four representations instantiated by parallel appraisal

Representation	Owner	Concrete form in this example
Playbook source	Author	Five reusable definitions and their Markdown instructions. Collection, grounding, readiness, and merge declarations contain no concrete expansion members.
Step execution records	Engine	Twenty-seven records, <code>exec-01</code> through <code>exec-27</code> . Twelve use the one Appraise definition, six use the one question-Synthesis definition, and three use the one Design definition.
Sealed state manifests	Engine	Twenty-eight full states, <code>state-0</code> through <code>state-27</code> , with ordinary, sibling, one-parent Synthesis, multi-parent Synthesis, and multi-parent Design lineage.
Generated Session contexts	Engine	Twenty-seven preambles containing the exact ordinary input state or merge-request source, its checkpoint, workspace, grounded collection member or group, authorized candidate states and checkpoints, output paths, and prospective completion contracts. Accepted completion outcomes are recorded afterward.

25.2 Author-written Playbook source

The following is the complete literal source used by this trace. Its `sketch-0` collection surface is deliberately provisional. The source defines fixed reusable work; Discover supplies finite runtime members through accepted completion.

```

1 ---
2 schema: alinery.playbook/sketch-0
3
4 playbook:
5   id: parallel-research-appraisal
6   title: Parallel Research Appraisal
7   revision: 1
8   description: >-
9     Frame research questions, discover a finite set of plausible
10    question-source pairs, appraise each pair independently, synthesize each
11    question after complete coverage, and merge the question syntheses into a

```

```

12  design.
13
14  collections:
15    - id: appraisal-requests
16      member_key: [question_id, source_id]
17      emitted_by: discover-sources
18    - id: appraisal-results
19      member_key_from: appraisal-requests
20      emitted_by: appraise-pair
21    - id: question-synthesis-requests
22      member_key: [question_id]
23      emitted_by: discover-sources
24    - id: question-syntheses
25      member_key_from: question-synthesis-requests
26      emitted_by: synthesize-question
27    - id: design-requests
28      member_key: [design_id]
29      emitted_by: discover-sources
30
31  steps:
32    - id: frame-questions
33      title: Frame / Refine Questions
34      revision: 1
35      harness: codex
36      model: gpt-5.6-sol
37      settings:
38        reasoning_effort: high
39      when: 'Exists(["research-agenda.md"])'
40      inputs: [research-agenda.md]
41      outputs:
42        required: [research-questions.md]
43      instructions: "##frame-questions"
44
45    - id: discover-sources
46      title: Discover Sources and Map Plausible Pairs
47      revision: 1
48      harness: codex
49      model: gpt-5.6-sol
50      settings:
51        reasoning_effort: high
52      when: 'Exists(["research-questions.md"])'
53      inputs: [research-questions.md]
54      outputs:
55        required: [source-map.md]
56        collections:
57          required:
58            - appraisal-requests
59            - question-synthesis-requests
60            - design-requests
61      instructions: "##discover-sources"
62
63    - id: appraise-pair
64      title: Appraise One Question-Source Pair
65      revision: 1
66      harness: codex
67      model: gpt-5.6-sol
68      settings:
69        reasoning_effort: high
70      instances:
71        one_per_new_member:
72          collection: appraisal-requests
73          bind_as: request
74      when: 'Exists([request.artifact])'
75      inputs:
76        primary: request.artifact
77      outputs:
78        required:
79          collection: appraisal-results
80          key_from: request.key
81          artifact_from: request.result_artifact
82      instructions: "##appraise-pair"
83
84    - id: synthesize-question
85      title: Synthesize One Question
86      revision: 1
87      harness: codex
88      model: gpt-5.6-sol
89      settings:
90        reasoning_effort: high
91      instances:
92        one_per_new_member:
93          collection: question-synthesis-requests
94          bind_as: request
95      when: 'Exists(resolved_binding)'
96      readiness:
97        complete_coverage:
98          request_members_from: request.discover_expansion.appraisal_requests
99          successful_results: appraisal-results
100         scope: producing_discover_execution
101      resolved_binding:
102        primary: request.artifact
103        exact_results_fulfilling: request.requires_appraisals
104      inputs:
105        primary: request.artifact
106        merge_results_fulfilling: request.requires_appraisals
107      merge:

```

```

108     selected_parent_states: agent_declared_at_step_complete
109     selected_results_cover: request.requires_appraisals
110     outputs:
111         required:
112             collection: question-syntheses
113             key_from: request.key
114             artifact_from: request.result_artifact
115     instructions: "#synthesize-question"

117 - id: develop-design
118   title: Develop / Revise Design
119   revision: 1
120   harness: codex
121   model: gpt-5.6-sol
122   settings:
123     reasoning_effort: high
124   instances:
125     one_per_new_member:
126       collection: design-requests
127       bind_as: request
128   when: 'Exists(resolved_binding)'
129   readiness:
130     complete_coverage:
131       request_members_from: request.requires_question_syntheses
132       successful_results: question-syntheses
133       scope: producing_discover_execution
134   resolved_binding:
135     primary: request.artifact
136     exact_results_fulfilling: request.requires_question_syntheses
137   inputs:
138     primary: request.artifact
139     merge_results_fulfilling: request.requires_question_syntheses
140   merge:
141     selected_parent_states: agent_declared_at_step_complete
142     selected_results_cover: request.requires_question_syntheses
143   outputs:
144     required: [design.md]
145     possible:
146       - research-agenda.md
147       - research-complete.md
148   instructions: "#develop-design"
149 ---

151 # Parallel Research Appraisal

153 Begin with `research-agenda.md` in the task state. This Playbook frames the
154 agenda, discovers a finite set of plausible question-source pairs, appraises
155 those pairs independently, merges the complete results for each question, and
156 then merges the question syntheses into a design.

158 The `collections`, `instances`, `readiness`, `resolved_binding`, and `merge`
159 spellings in this specimen are provisional schema. They make late grounding,
160 complete coverage, and merge provenance explicit without selecting the final
161 portable syntax.

163 The authored `when` expressions state only whether the required exact inputs
164 exist. Before an automatic launch, Alinery derives the resolved input binding
165 and durably claims that exact Step-definition-revision and binding pair. An
166 existing claim or successful execution suppresses a duplicate automatic
167 launch. This engine rule is not authored as `NewFor(...)`; an explicit manual
168 rerun creates a different execution and successor state.

170 ## Process

172 The concrete expansion below is pass 1. A later Discover execution may freeze
173 a different finite number of questions, pairs, question groups, and handoffs.

175 ```text
176 research-agenda.md
177 |
178 v
179 Frame / Refine Questions -> research-questions.md
180 |
181 v
182 Discover Sources and Map Plausible Pairs
183 |
184 +-- appraisal-request-Q1-A.md -> Appraise(Q1,A) --+
185 +-- appraisal-request-Q1-B.md -> Appraise(Q1,B) --+--> Synthesize(Q1) --+
186 |                                     |
187 +-- appraisal-request-Q2-B.md -> Appraise(Q2,B) --+ |
188 +-- appraisal-request-Q2-C.md -> Appraise(Q2,C) --+--> Synthesize(Q2) --+
189 |                                     |
190                                     v
191                               Develop / Revise Design
192 |
193 +-- research-agenda.md --> another pass -----+
194 +-- research-complete.md; no new agenda --> quiescent
195 ```

197 The four Appraise executions may run concurrently. They begin from one exact
198 Discover successor and publish four sibling states. Under the conservative V1
199 barrier, neither question Synthesis may start until all four pass-1 appraisal
200 requests have successful results. Once that barrier is satisfied,
201 `Synthesize(Q1)` is a multi-parent merge over the successful results for
202 `(Q1,A)` and `(Q1,B)` only. `Synthesize(Q2)` similarly merges only `(Q2,B)` and
203 `(Q2,C)`. Develop / Revise Design is a merge over the two exact

```

```

204 question-synthesis results.

206 The labels above are the concrete finite expansion used for pass 1. The Step
207 definitions do not contain four authored Appraise copies. Each Discover
208 execution declares that pass's members at accepted completion, and the engine
209 grounds one execution of the reusable `appraise-pair` definition for each
210 exact member.

212 ## Exact collection semantics used by this specimen

214 A collection member is an engine-recorded tuple, not a filename match. Its
215 runtime identity includes the producing Discover execution, collection
216 identifier, and member key. Reusing `Q1` in a later research pass therefore
217 does not reuse an earlier member.

219 Discover writes ordinary Markdown files, then enumerates their exact paths and
220 relationships in its structured `step_complete` call. Alinery validates those
221 files and freezes the enumeration in the immutable successor manifest. The
222 engine never discovers members with a glob or regular expression and never
223 parses Markdown to infer a key, group, coverage requirement, or output path.

225 For pass 1's concrete expansion, the accepted collection declaration is
226 equivalent to:

228 ```yaml
229 appraisal-requests:
230   - key: {question_id: Q1, source_id: A}
231     artifact: appraisal-request-Q1-A.md
232     result_artifact: appraisal-result-Q1-A.md
233   - key: {question_id: Q1, source_id: B}
234     artifact: appraisal-request-Q1-B.md
235     result_artifact: appraisal-result-Q1-B.md
236   - key: {question_id: Q2, source_id: B}
237     artifact: appraisal-request-Q2-B.md
238     result_artifact: appraisal-result-Q2-B.md
239   - key: {question_id: Q2, source_id: C}
240     artifact: appraisal-request-Q2-C.md
241     result_artifact: appraisal-result-Q2-C.md

243 question-synthesis-requests:
244   - key: {question_id: Q1}
245     artifact: question-synthesis-request-Q1.md
246     requires_appraisals:
247       - {question_id: Q1, source_id: A}
248       - {question_id: Q1, source_id: B}
249     result_artifact: question-synthesis-Q1.md
250   - key: {question_id: Q2}
251     artifact: question-synthesis-request-Q2.md
252     requires_appraisals:
253       - {question_id: Q2, source_id: B}
254       - {question_id: Q2, source_id: C}
255     result_artifact: question-synthesis-Q2.md

257 design-requests:
258   - key: {design_id: current}
259     artifact: design-request.md
260     requires_question_syntheses:
261       - {question_id: Q1}
262       - {question_id: Q2}
263   ...

265 The `requires_appraisals` and `requires_question_syntheses` fields are
266 structured manifest relationships. Their duplicate human-readable contents in
267 the handoff files help agents understand the work but are not authoritative.

269 ## Engine-owned publication

271 Every grounded execution follows the same publication boundary:

273 1. The agent works in the execution's isolated workspace and writes the exact
274 handoff paths supplied in its generated Session context.
275 2. Intermediate edits remain unpublished.
276 3. The agent calls `step_complete`, including structured collection members or
277 selected merge parents when that execution requires them.
278 4. Alinery validates the output contract, frozen coverage, and any selected
279 parent-state identifiers.
280 5. Alinery checkpoints the resulting filesystem and writes the immutable state
281 manifest with exact parents, input binding, collection relationships,
282 outputs, Step definition, and Session provenance.
283 6. Only that sealed state becomes visible and causes trigger evaluation to
284 resume.

286 The agent never creates a state or writes the authoritative manifest.

288 <a id="frame-questions"></a>
289 ## Frame / Refine Questions

291 Read `research-agenda.md` as the primary handoff. Inspect the current design,
292 prior evidence, and the rest of the input state when useful. Turn the agenda
293 into focused research questions. Do not search for or appraise sources in this
294 Step.

296 Write `research-questions.md` with:

298 1. The design decision currently being informed.
299 2. Stable question identifiers such as `Q1` and `Q2`.

```

```

300 3. For each question:
301     - the exact question;
302     - why answering it matters now;
303     - what an adequate answer must establish;
304     - important scope boundaries;
305     - assumptions that require evidence.
306 4. Questions retired, split, combined, or carried forward, with a reason.
307 5. A concise research scope for this pass.

309 Call `step_complete` only when a different agent can discover sources without
310 reconstructing the design problem. Publish exactly `research-questions.md`.

312 <a id="discover-sources"></a>
313 ## Discover Sources and Map Plausible Pairs

315 Read `research-questions.md` as the primary handoff. Find credible candidate
316 sources and identify only the question-source pairs that merit appraisal. A
317 source may be paired with several questions, and a question may have several
318 sources. Do not create a Cartesian product merely to increase parallelism.

320 Write `source-map.md` with:

322 1. The current question identifiers and search scope.
323 2. Every retained source, with a stable source identifier, bibliographic
324    details, source type, and stable locator.
325 3. Every plausible question-source pair and why appraisal is warranted.
326 4. Questions for which no plausible source was found.
327 5. Search and retrieval limitations.

329 For each plausible pair, write one self-contained appraisal request. Each file
330 must identify the exact question and source, include enough retrieval detail,
331 state what an adequate appraisal must determine, and name its exact expected
332 result path. For pass 1's concrete expansion, write:

334 ```text
335 appraisal-request-Q1-A.md
336 appraisal-request-Q1-B.md
337 appraisal-request-Q2-B.md
338 appraisal-request-Q2-C.md
339 ```

341 A later pass writes one request for each pair that its own Discover execution
342 retains; it need not produce four members or reuse these keys.

344 For each question, also write one synthesis request. It must restate the
345 question, name the exact appraisal request keys whose successful results are
346 required, state the synthesis standard, and name its exact result path:

348 ```text
349 question-synthesis-request-Q1.md
350 question-synthesis-request-Q2.md
351 ```

353 Write `design-request.md` with the design objective, the exact question keys
354 whose syntheses are required, and the standard for integrating them.

356 Complete with one structured call that publishes every exact handoff and
357 enumerates the three collections. For pass 1's concrete expansion:

359 ```text
360 step_complete(
361     publish = [
362         "source-map.md",
363         "appraisal-request-Q1-A.md",
364         "appraisal-request-Q1-B.md",
365         "appraisal-request-Q2-B.md",
366         "appraisal-request-Q2-C.md",
367         "question-synthesis-request-Q1.md",
368         "question-synthesis-request-Q2.md",
369         "design-request.md"
370     ],
371     collections = declared_members_and_relationships
372 )
373 ```

375 Alinery, not this Session, assigns the Discover execution identifier, seals the
376 collection member occurrences, and creates the successor state. Accepted
377 completion freezes the finite appraisal and synthesis coverage sets for this
378 Discover execution.

380 <a id="appraise-pair"></a>
381 ## Appraise One Question-Source Pair

383 Alinery grounds one execution of this Step for each exact new member of
384 `appraisal-requests`. Read the assigned request artifact as the primary
385 handoff. You may inspect `research-questions.md`, `source-map.md`, and the rest
386 of the exact input state for context, but appraise only the assigned pair.

388 Retrieve and examine the assigned source. Write the exact result path supplied
389 by the request member and generated Session context. Include:

391 1. The exact question identifier and source identifier.
392 2. Retrieval status and the material actually examined.
393 3. Findings relevant to the assigned question, with precise support.
394 4. Relevance, credibility, methodology, and limitations.
395 5. Whether the source answers the question fully, partially, not at all, or

```

```

396 with contradictory evidence.
397 6. Uncertainty, important absences, and follow-up questions.
398 7. A concise statement suitable for later synthesis.

400 For the `(Q1,A)` binding, for example, complete with:

402 ```text
403 step_complete(publish = ["appraisal-result-Q1-A.md"])
404 ```

406 The engine derives the result member's exact relationship to the bound request
407 from the execution record. The agent does not enumerate or claim sibling
408 requests. A failed or incomplete execution publishes no result member.

410 <a id="synthesize-question"></a>
411 ## Synthesize One Question

413 Alinery grounds one execution of this Step for each exact new member of
414 `question-synthesis-requests`, but it starts none of them until every appraisal
415 request emitted by that same Discover execution has one successful result. In
416 pass 1, both `Synthesize(Q1)` and `Synthesize(Q2)` therefore wait for successful
417 results for `(Q1,A)`, `(Q1,B)`, `(Q2,B)`, and `(Q2,C)`. A running, failed, or
418 missing Appraise execution keeps both Synthesis executions ineligible. V1 has
419 no partial, timeout, quorum, or early-synthesis policy.

421 After the global barrier is satisfied, read the assigned synthesis request as
422 the primary handoff. The generated Session context supplies only that
423 question's exact successful appraisal-result occurrences and their exact
424 states. For `Q1`, those are the results fulfilling `(Q1,A)` and `(Q1,B)` from
425 this same Discover expansion, never the `Q2` results or any same-named result
426 from an earlier or sibling expansion. The resolved input binding contains the
427 primary synthesis request occurrence plus both exact Q1 result occurrences.

429 Compare the results and write the exact question-synthesis path supplied in the
430 request. Include:

432 1. The exact question and appraisal coverage set.
433 2. Status: `answered`, `partially answered`, or `unanswered`.
434 3. The best-supported current answer.
435 4. Agreements, contradictions, and differences in evidentiary strength.
436 5. Source limitations and remaining uncertainty.
437 6. Newly exposed research questions.
438 7. Findings sufficiently supported to inform design.

440 At completion, select every exact appraisal-result state required for complete
441 coverage and publish the one synthesis handoff:

443 ```text
444 step_complete(
445     publish = ["question-synthesis-Q1.md"],
446     selected_parent_state_ids = authorized_Q1_result_states
447 )
448 ```

450 Alinery rejects completion if the selected parents and result handoffs do not
451 cover the entire frozen Q1 request group. It then creates one multi-parent
452 successor state and records the fulfilled question-synthesis member.

454 <a id="develop-design"></a>
455 ## Develop / Revise Design

457 Alinery grounds this merge from the exact `design-requests` member emitted by
458 Discover. It becomes eligible only after every question-synthesis request named
459 by that member has one successful result. Read `design-request.md` as the
460 primary handoff and the exact question-synthesis occurrences named by
461 `request.requires_question_syntheses` as merge inputs. The generated Session
462 context supplies those exact occurrences and their authorized parent states.
463 You may inspect every authorized parent state and ask the human for
464 clarification before completing.

466 Write `design.md` with:

468 1. The problem and current design objectives.
469 2. The proposed or revised design.
470 3. Each material decision and its evidence or explicit assumption.
471 4. Agreements or tensions between question syntheses and how they were handled.
472 5. Alternatives considered.
473 6. Risks, limitations, and consequences of being wrong.
474 7. Remaining design and research questions.
475 8. Recommended next actions.

477 Then make exactly one research-continuation decision:

479 - If material research questions remain, write a new `research-agenda.md`
480 occurrence containing only questions that could materially change or validate
481 the design.
482 - If no material research questions remain for the current scope, write
483 `research-complete.md` explaining why another pass is not warranted.

485 On a continuing pass, select every exact question-synthesis parent state used
486 to satisfy the request and complete with:

488 ```text
489 step_complete(
490     publish = ["design.md", "research-agenda.md"],
491     selected_parent_state_ids = authorized_question_synthesis_states

```

```

492 )
493 ```
494
495 On the final pass, complete with:
496
497 ```text
498 step_complete(
499     publish = ["design.md", "research-complete.md"],
500     selected_parent_state_ids = authorized_question_synthesis_states
501 )
502 ```
503
504 Alinery validates complete question coverage and the selected parents before
505 creating the multi-parent design state. A new 'research-agenda.md' occurrence
506 makes Frame applicable again. 'research-complete.md' is documentary; the
507 Playbook becomes quiescent because Design publishes no new agenda occurrence.

```

Listing 2: Complete parallel research-appraisal Playbook specimen.

25.3 Engine-owned execution and state records

25.3.1 Identifier and artifact legend

```

states      = state-0 ... state-27
executions  = exec-01 ... exec-27
sessions    = session-01 ... session-27
checkpoints = checkpoint-000 ... checkpoint-027

G  = research-agenda.md      Q  = research-questions.md
M  = source-map.md          Z  = design-request.md
Pxy = appraisal-request-Qx-Y.md
Axy = appraisal-result-Qx-Y.md
Tx  = question-synthesis-request-Qx.md
Yx  = question-synthesis-Qx.md
D   = design.md              C   = research-complete.md

```

An occurrence suffix identifies the publishing execution. For example, `P1A@02` is the Q1/A appraisal-request occurrence published by `exec-02`. The suffix is manifest provenance, not part of the filename seen by an agent. The engine also writes one immutable expansion record for each Discover success:

```

K02 = expansion@exec-02
  appraisal groups: Q1=[P1A@02,P1B@02]
                   Q2=[P2B@02,P2C@02]
  synthesis requests: [T1@02,T2@02]
  design request: Z@02 requires [Q1,Q2]

K11 = expansion@exec-11
  appraisal groups: Q3=[P3D@11,P3E@11]
                   Q4=[P4F@11]
  synthesis requests: [T3@11,T4@11]
  design request: Z@11 requires [Q3,Q4]

K19 = expansion@exec-19
  appraisal groups: Q5=[P5G@19,P5H@19,P5I@19]
                   Q6=[P6H@19,P6J@19]
  synthesis requests: [T5@19,T6@19]
  design request: Z@19 requires [Q5,Q6]

```

These memberships are authoritative structured provenance. No glob, regular expression, filename prefix, or Markdown parser reconstructs them. The collection identity includes its producing Discover execution, so reusing a question or source key in another pass cannot accidentally reuse an older member.

25.3.2 Discover success freezes one finite expansion

The first Discover execution produces one successor state, not four Appraise states. Its accepted publication contains ordinary files plus an exact collection declaration:

```
StepExecution exec-02
  step_definition = discover-sources@1
  parent_states  = [state-1]
  input_binding  = [Q@01]
  sessions       = [session-02]
  lifecycle      = completed
  result_state   = state-2
  published      = [M@02, P1A@02, P1B@02, P2B@02, P2C@02,
                    T1@02, T2@02, Z@02]
  frozen_expansion = K02

StateManifest state-2
  parents        = [state-1]
  produced_by    = exec-02
  git_checkpoint = checkpoint-002
  artifacts      = full current map shown below
  collection_record = K02
  accepted_at    = 2026-08-29T13:02:00Z
```

Only after state-2 is visible does the engine ground four independent executions. Each applies the Appraise definition, receives state-2 as its one parent, and receives one exact request occurrence as its primary binding.

25.3.3 One question-Synthesis merge record expanded

The global V1 readiness barrier for K02 reaches four successful results before either question Synthesis starts. The Q1 execution then receives only the exact Q1 group as merge input. Candidate exposure is launch context; selected parents are an accepted-completion outcome:

```
StepExecution exec-07
  step_definition = synthesize-question@1
  grounding       = {expansion: K02, group: Q1}
  readiness_witness = {K02 appraisal coverage: 4/4}
  authorized_candidates = [
    {state: state-3, result: A1A@03, covers: P1A@02},
    {state: state-4, result: A1B@04, covers: P1B@02}
  ]
  input_binding   = [T1@02, A1A@03, A1B@04]
  sessions        = [session-07]
  lifecycle       = completed
  accepted_selection = [state-3, state-4]
  result_state    = state-7
  published       = [Y1@07]

StateManifest state-7
  parents        = [state-3, state-4]
  produced_by    = exec-07
  git_checkpoint = checkpoint-007
  input_binding  = [T1@02, A1A@03, A1B@04]
  published_outputs = [Y1@07]
  session_ids    = [session-07]
  accepted_at    = 2026-08-29T13:07:00Z
```

The engine validates that the declared states are authorized and collectively cover the entire frozen Q1 group. It does not make the unrelated Q2 Appraise states parents merely because their success was required by the conservative global launch barrier.

25.3.4 The final Design merge record expanded

The final Design Session is launched with two authorized question-synthesis candidates but no already-selected parents. The accepted completion supplies that semantic choice:

```
StepExecution exec-27
  step_definition      = develop-design@1
  grounding            = {expansion: K19, design request: Z@19}
  authorized_candidates = [
    {state: state-25, result: Y5@25, covers: K19/Q5},
    {state: state-26, result: Y6@26, covers: K19/Q6}
  ]
  input_binding        = [Z@19, Y5@25, Y6@26]
  sessions             = [session-27]
  lifecycle            = completed
  accepted_selection   = [state-25, state-26]
  result_state         = state-27
  published            = [D@27, C@27]

StateManifest state-27
  parents              = [state-25, state-26]
  produced_by          = exec-27
  git_checkpoint       = checkpoint-027
  input_binding        = [Z@19, Y5@25, Y6@26]
  published_outputs    = [D@27, C@27]
  session_ids          = [session-27]
  accepted_at          = 2026-08-29T13:27:00Z
```

The full state still inherits G@17, but exec-27 publishes no new research-agenda.md occurrence. Design's completion therefore cannot ground a fourth Frame execution.

25.3.5 All twenty-seven Step execution records

Every execution below uses Step-definition revision 1, one Session with the matching number, and one accepted publication. Bracketed sources identify multi-parent or merge-capable transitions.

Pass	Exec	Step; parents to result	Exact binding	Newly published occurrences
1	01	Frame; 0 to 1	G@seed-01	Q@01
1	02	Discover; 1 to 2	Q@01	M@02, K02: P1A, P1B, P2B, P2C, T1, T2, Z
1	03	Appraise Q1/A; 2 to 3	P1A@02	A1A@03
1	04	Appraise Q1/B; 2 to 4	P1B@02	A1B@04
1	05	Appraise Q2/B; 2 to 5	P2B@02	A2B@05
1	06	Appraise Q2/C; 2 to 6	P2C@02	A2C@06
1	07	Synthesize Q1; [3,4] to 7	T1@02, A1A@03, A1B@04	Y1@07
1	08	Synthesize Q2; [5,6] to 8	T2@02, A2B@05, A2C@06	Y2@08
1	09	Design; [7,8] to 9	Z@02, Y1@07, Y2@08	D@09, G@09
2	10	Frame; 9 to 10	G@09	Q@10
2	11	Discover; 10 to 11	Q@10	M@11, K11: P3D, P3E, P4F, T3, T4, Z
2	12	Appraise Q3/D; 11 to 12	P3D@11	A3D@12
2	13	Appraise Q3/E; 11 to 13	P3E@11	A3E@13
2	14	Appraise Q4/F; 11 to 14	P4F@11	A4F@14
2	15	Synthesize Q3; [12,13] to 15	T3@11, A3D@12, A3E@13	Y3@15
2	16	Synthesize Q4; 14 to 16	T4@11, A4F@14	Y4@16
2	17	Design; [15,16] to 17	Z@11, Y3@15, Y4@16	D@17, G@17
3	18	Frame; 17 to 18	G@17	Q@18
3	19	Discover; 18 to 19	Q@18	M@19, K19: P5G, P5H, P5I, P6H, P6J, T5, T6, Z
3	20	Appraise Q5/G; 19 to 20	P5G@19	A5G@20
3	21	Appraise Q5/H; 19 to 21	P5H@19	A5H@21
3	22	Appraise Q5/I; 19 to 22	P5I@19	A5I@22
3	23	Appraise Q6/H; 19 to 23	P6H@19	A6H@23
3	24	Appraise Q6/J; 19 to 24	P6J@19	A6J@24
3	25	Synthesize Q5; [20,21,22] to 25	T5@19, A5G@20, A5H@21, A5I@22	Y5@25
3	26	Synthesize Q6; [23,24] to 26	T6@19, A6H@23, A6J@24	Y6@26
3	27	Design; [25,26] to 27	Z@19, Y5@25, Y6@26	D@27, C@27

25.3.6 Every successor is a full state

The following equations specify every current artifact map exactly while avoiding 28 repetitions of inherited files. In this notation, $X + \{\text{occurrences}\}$ means copy the complete map of X , then set or replace each named logical path with the listed occurrence. $\text{merge}(X, Y)$ unions the selected parent maps at their common inherited occurrences; the agent resolves any genuine conflict before publication.

```

B02 = {M@02, P1A@02, P1B@02, P2B@02, P2C@02, T1@02, T2@02, Z@02}
B11 = {M@11, P3D@11, P3E@11, P4F@11, T3@11, T4@11, Z@11}
B19 = {M@19, P5G@19, P5H@19, P5I@19, P6H@19, P6J@19,
      T5@19, T6@19, Z@19}

state-0 = {G@seed-01}
state-1 = state-0 + {Q@01}
state-2 = state-1 + B02
state-3 = state-2 + {A1A@03}
state-4 = state-2 + {A1B@04}
state-5 = state-2 + {A2B@05}
state-6 = state-2 + {A2C@06}
state-7 = merge(state-3, state-4) + {Y1@07}
state-8 = merge(state-5, state-6) + {Y2@08}
state-9 = merge(state-7, state-8) + {D@09, G@09}

state-10 = state-9 + {Q@10}
state-11 = state-10 + B11
state-12 = state-11 + {A3D@12}
state-13 = state-11 + {A3E@13}
state-14 = state-11 + {A4F@14}
state-15 = merge(state-12, state-13) + {Y3@15}
state-16 = state-14 + {Y4@16}
state-17 = merge(state-15, state-16) + {D@17, G@17}

state-18 = state-17 + {Q@18}
state-19 = state-18 + B19
state-20 = state-19 + {A5G@20}
state-21 = state-19 + {A5H@21}
state-22 = state-19 + {A5I@22}
state-23 = state-19 + {A6H@23}
state-24 = state-19 + {A6J@24}
state-25 = merge(state-20, state-21, state-22) + {Y5@25}
state-26 = merge(state-23, state-24) + {Y6@26}
state-27 = merge(state-25, state-26) + {D@27, C@27}

```

These are full maps, not output deltas. Because `state-27` does not set `research-agenda.md`, its current map still contains `G@17`. Likewise, setting a new `source-map.md`, `design-request.md`, or `design.md` path changes the current occurrence without deleting the earlier occurrence from immutable history.

25.4 Engine-generated Session context

For this trace the illustrative runtime root is `/runtime/parallel-research`. For every `exec-NN`, the exact generated values are:

```

session    = session-NN
workspace  = /runtime/parallel-research/exec-NN/workspace
artifacts  = /runtime/parallel-research/exec-NN/workspace/artifacts

```

The path rule is exact within this example but does not freeze a production layout. Each compact packet below supplies every varying pre-launch fact. The engine appends the matching authored instruction section from Listing 2. A prospective `completion_contract` never contains the agent's future parent selection; accepted outcomes are listed separately after each pass.

25.4.1 Pass 1 contexts

```
[exec-01 / session-01]
step=Frame / Refine Questions (frame-questions@1)
input=state-0 / checkpoint-000
primary=research-agenda.md; occurrence=G@seed-01
predecessor=human seed seed-01
required=[research-questions.md]
completion_contract=publish [research-questions.md]

[exec-02 / session-02]
step=Discover Sources and Map Plausible Pairs (discover-sources@1)
input=state-1 / checkpoint-001
primary=research-questions.md; occurrence=Q@01
predecessor=Frame exec-01
required=[source-map.md, appraisal-requests,
          question-synthesis-requests, design-request]
completion_contract=publish exact files and finite collection declaration

[exec-03 / session-03]
step=Appraise One Pair (appraise-pair@1)
input=state-2 / checkpoint-002
grounding={expansion:K02,member:Q1/A,group:Q1}
primary=appraisal-request-Q1-A.md; occurrence=P1A@02
required=[appraisal-result-Q1-A.md]
completion_contract=publish [appraisal-result-Q1-A.md]

[exec-04 / session-04]
step=Appraise One Pair (appraise-pair@1)
input=state-2 / checkpoint-002
grounding={expansion:K02,member:Q1/B,group:Q1}
primary=appraisal-request-Q1-B.md; occurrence=P1B@02
required=[appraisal-result-Q1-B.md]
completion_contract=publish [appraisal-result-Q1-B.md]

[exec-05 / session-05]
step=Appraise One Pair (appraise-pair@1)
input=state-2 / checkpoint-002
grounding={expansion:K02,member:Q2/B,group:Q2}
primary=appraisal-request-Q2-B.md; occurrence=P2B@02
required=[appraisal-result-Q2-B.md]
completion_contract=publish [appraisal-result-Q2-B.md]

[exec-06 / session-06]
step=Appraise One Pair (appraise-pair@1)
input=state-2 / checkpoint-002
grounding={expansion:K02,member:Q2/C,group:Q2}
primary=appraisal-request-Q2-C.md; occurrence=P2C@02
required=[appraisal-result-Q2-C.md]
completion_contract=publish [appraisal-result-Q2-C.md]

[exec-07 / session-07]
step=Synthesize One Question (synthesize-question@1)
coverage_gate={expansion:K02,successful:4,required:4}
grounding={group:Q1,primary:T1@02}
request_source=state-2 / checkpoint-002
resolved_binding=[T1@02,A1A@03,A1B@04]
authorized_candidates=[state-3/checkpoint-003/A1A@03,
                      state-4/checkpoint-004/A1B@04]
required=[question-synthesis-Q1.md]
completion_contract=select authorized parents covering Q1; publish required

[exec-08 / session-08]
step=Synthesize One Question (synthesize-question@1)
coverage_gate={expansion:K02,successful:4,required:4}
grounding={group:Q2,primary:T2@02}
request_source=state-2 / checkpoint-002
resolved_binding=[T2@02,A2B@05,A2C@06]
authorized_candidates=[state-5/checkpoint-005/A2B@05,
                      state-6/checkpoint-006/A2C@06]
required=[question-synthesis-Q2.md]
completion_contract=select authorized parents covering Q2; publish required

[exec-09 / session-09]
step=Develop / Revise Design (develop-design@1)
coverage_gate={expansion:K02,question_groups:2/2}
grounding={primary:Z@02}
request_source=state-2 / checkpoint-002
resolved_binding=[Z@02,Y1@07,Y2@08]
authorized_candidates=[state-7/checkpoint-007/Y1@07,
                      state-8/checkpoint-008/Y2@08]
required=[design.md]
possible=[research-agenda.md,research-complete.md]
completion_contract=select complete synthesis parents; publish design plus
                  exactly one decision handoff
```

Accepted outcomes after pass 1 (not Session context).

```
exec-01 accepted publish=[Q@01]
exec-02 accepted publish=[M@02,P1A@02,P1B@02,P2B@02,P2C@02,
                        T1@02,T2@02,Z@02]; freeze=K02
```

```

exec-03 accepted publish=[A1A@03]
exec-04 accepted publish=[A1B@04]
exec-05 accepted publish=[A2B@05]
exec-06 accepted publish=[A2C@06]
exec-07 accepted parents=[state-3,state-4]; publish=[Y1@07]
exec-08 accepted parents=[state-5,state-6]; publish=[Y2@08]
exec-09 accepted parents=[state-7,state-8]; publish=[D@09,G@09]

```

25.4.2 Pass 2 contexts

```

[exec-10 / session-10]
step=Frame / Refine Questions (frame-questions@1)
input=state-9 / checkpoint-009
primary=research-agenda.md; occurrence=G@09
predecessor=Design exec-09
required=[research-questions.md]
completion_contract=publish [research-questions.md]

[exec-11 / session-11]
step=Discover Sources and Map Plausible Pairs (discover-sources@1)
input=state-10 / checkpoint-010
primary=research-questions.md; occurrence=Q@10
predecessor=Frame exec-10
required=[source-map.md, appraisal-requests,
          question-synthesis-requests, design-request]
completion_contract=publish exact files and finite collection declaration

[exec-12 / session-12]
step=Appraise One Pair (appraise-pair@1)
input=state-11 / checkpoint-011
grounding={expansion:K11,member:Q3/D,group:Q3}
primary=appraisal-request-Q3-D.md; occurrence=P3D@11
required=[appraisal-result-Q3-D.md]
completion_contract=publish [appraisal-result-Q3-D.md]

[exec-13 / session-13]
step=Appraise One Pair (appraise-pair@1)
input=state-11 / checkpoint-011
grounding={expansion:K11,member:Q3/E,group:Q3}
primary=appraisal-request-Q3-E.md; occurrence=P3E@11
required=[appraisal-result-Q3-E.md]
completion_contract=publish [appraisal-result-Q3-E.md]

[exec-14 / session-14]
step=Appraise One Pair (appraise-pair@1)
input=state-11 / checkpoint-011
grounding={expansion:K11,member:Q4/F,group:Q4}
primary=appraisal-request-Q4-F.md; occurrence=P4F@11
required=[appraisal-result-Q4-F.md]
completion_contract=publish [appraisal-result-Q4-F.md]

[exec-15 / session-15]
step=Synthesize One Question (synthesize-question@1)
coverage_gate={expansion:K11,successful:3,required:3}
grounding={group:Q3,primary:T3@11}
request_source=state-11 / checkpoint-011
resolved_binding=[T3@11,A3D@12,A3E@13]
authorized_candidates=[state-12/checkpoint-012/A3D@12,
                      state-13/checkpoint-013/A3E@13]
required=[question-synthesis-Q3.md]
completion_contract=select authorized parents covering Q3; publish required

[exec-16 / session-16]
step=Synthesize One Question (synthesize-question@1)
coverage_gate={expansion:K11,successful:3,required:3}
grounding={group:Q4,primary:T4@11}
request_source=state-11 / checkpoint-011
resolved_binding=[T4@11,A4F@14]
authorized_candidates=[state-14/checkpoint-014/A4F@14]
required=[question-synthesis-Q4.md]
completion_contract=select the authorized Q4 parent; publish required

[exec-17 / session-17]
step=Develop / Revise Design (develop-design@1)
coverage_gate={expansion:K11,question_groups:2/2}
grounding={primary:Z@11}
request_source=state-11 / checkpoint-011
resolved_binding=[Z@11,Y3@15,Y4@16]
authorized_candidates=[state-15/checkpoint-015/Y3@15,
                      state-16/checkpoint-016/Y4@16]
required=[design.md]
possible=[research-agenda.md,research-complete.md]
completion_contract=select complete synthesis parents; publish design plus
                    exactly one decision handoff

```

Accepted outcomes after pass 2 (not Session context).

```

exec-10 accepted publish=[Q@10]
exec-11 accepted publish=[M@11,P3D@11,P3E@11,P4F@11,
                        T3@11,T4@11,Z@11]; freeze=K11
exec-12 accepted publish=[A3D@12]
exec-13 accepted publish=[A3E@13]
exec-14 accepted publish=[A4F@14]
exec-15 accepted parents=[state-12,state-13]; publish=[Y3@15]
exec-16 accepted parents=[state-14]; publish=[Y4@16]
exec-17 accepted parents=[state-15,state-16]; publish=[D@17,G@17]

```

25.4.3 Pass 3 contexts

```

[exec-18 / session-18]
step=Frame / Refine Questions (frame-questions@1)
input=state-17 / checkpoint-017
primary=research-agenda.md; occurrence=G@17
predecessor=Design exec-17
required=[research-questions.md]
completion_contract=publish [research-questions.md]

[exec-19 / session-19]
step=Discover Sources and Map Plausible Pairs (discover-sources@1)
input=state-18 / checkpoint-018
primary=research-questions.md; occurrence=Q@18
predecessor=Frame exec-18
required=[source-map.md, appraisal-requests,
          question-synthesis-requests, design-request]
completion_contract=publish exact files and finite collection declaration

[exec-20 / session-20]
step=Appraise One Pair (appraise-pair@1)
input=state-19 / checkpoint-019
grounding={expansion:K19,member:Q5/G,group:Q5}
primary=appraisal-request-Q5-G.md; occurrence=P5G@19
required=[appraisal-result-Q5-G.md]
completion_contract=publish [appraisal-result-Q5-G.md]

[exec-21 / session-21]
step=Appraise One Pair (appraise-pair@1)
input=state-19 / checkpoint-019
grounding={expansion:K19,member:Q5/H,group:Q5}
primary=appraisal-request-Q5-H.md; occurrence=P5H@19
required=[appraisal-result-Q5-H.md]
completion_contract=publish [appraisal-result-Q5-H.md]

[exec-22 / session-22]
step=Appraise One Pair (appraise-pair@1)
input=state-19 / checkpoint-019
grounding={expansion:K19,member:Q5/I,group:Q5}
primary=appraisal-request-Q5-I.md; occurrence=P5I@19
required=[appraisal-result-Q5-I.md]
completion_contract=publish [appraisal-result-Q5-I.md]

[exec-23 / session-23]
step=Appraise One Pair (appraise-pair@1)
input=state-19 / checkpoint-019
grounding={expansion:K19,member:Q6/H,group:Q6}
primary=appraisal-request-Q6-H.md; occurrence=P6H@19
required=[appraisal-result-Q6-H.md]
completion_contract=publish [appraisal-result-Q6-H.md]

[exec-24 / session-24]
step=Appraise One Pair (appraise-pair@1)
input=state-19 / checkpoint-019
grounding={expansion:K19,member:Q6/J,group:Q6}
primary=appraisal-request-Q6-J.md; occurrence=P6J@19
required=[appraisal-result-Q6-J.md]
completion_contract=publish [appraisal-result-Q6-J.md]

[exec-25 / session-25]
step=Synthesize One Question (synthesize-question@1)
coverage_gate={expansion:K19,successful:5,required:5}
grounding={group:Q5,primary:T5@19}
request_source=state-19 / checkpoint-019
resolved_binding=[T5@19,A5G@20,A5H@21,A5I@22]
authorized_candidates=[state-20/checkpoint-020/A5G@20,
                      state-21/checkpoint-021/A5H@21,
                      state-22/checkpoint-022/A5I@22]
required=[question-synthesis-Q5.md]
completion_contract=select authorized parents covering Q5; publish required

[exec-26 / session-26]
step=Synthesize One Question (synthesize-question@1)
coverage_gate={expansion:K19,successful:5,required:5}
grounding={group:Q6,primary:T6@19}

```

```

request_source=state-19 / checkpoint-019
resolved_binding=[T6@19,A6H@23,A6J@24]
authorized_candidates=[state-23/checkpoint-023/A6H@23,
                      state-24/checkpoint-024/A6J@24]
required=[question-synthesis-06.md]
completion_contract=select authorized parents covering Q6; publish required

[exec-27 / session-27]
step=Develop / Revise Design (develop-design@1)
coverage_gate={expansion:K19,question_groups:2/2}
grounding={primary:Z@19}
request_source=state-19 / checkpoint-019
resolved_binding=[Z@19,Y5@25,Y6@26]
authorized_candidates=[state-25/checkpoint-025/Y5@25,
                      state-26/checkpoint-026/Y6@26]
required=[design.md]
possible=[research-agenda.md,research-complete.md]
completion_contract=select complete synthesis parents; publish design plus
                      exactly one decision handoff

```

Accepted outcomes after pass 3 (not Session context).

```

exec-18 accepted publish=[Q@18]
exec-19 accepted publish=[M@19,P5G@19,P5H@19,P5I@19,P6H@19,P6J@19,
                        T5@19,T6@19,Z@19]; freeze=K19
exec-20 accepted publish=[A5G@20]
exec-21 accepted publish=[A5H@21]
exec-22 accepted publish=[A5I@22]
exec-23 accepted publish=[A6H@23]
exec-24 accepted publish=[A6J@24]
exec-25 accepted parents=[state-20,state-21,state-22]; publish=[Y5@25]
exec-26 accepted parents=[state-23,state-24]; publish=[Y6@26]
exec-27 accepted parents=[state-25,state-26]; publish=[D@27,C@27]

```

The 27 contexts use only five definition revisions. In particular, no Appraise Session is authored in advance for a concrete pair. Discover determines the finite decomposition, the engine grounds exact instances, and the scheduler controls how many may run concurrently.

25.5 State to execution to state

25.5.1 Concrete forward-only DAG

The derived dependency view fans out and later loops, but the concrete history contains only forward transitions. All Appraises in one pass keep the exact Discover successor as parent even if siblings finish earlier:

```

Pass 1
state-0 -> exec-01 Frame -> state-1 -> exec-02 Discover -> state-2
  state-2 -> exec-03 Appraise Q1/A -> state-3 ---+
  state-2 -> exec-04 Appraise Q1/B -> state-4 ---+> exec-07 Synth Q1 -> state-7 ---+
  state-2 -> exec-05 Appraise Q2/B -> state-5 ---+                               +-> exec-09 Design -> state-9
  state-2 -> exec-06 Appraise Q2/C -> state-6 ---+> exec-08 Synth Q2 -> state-8 ---+

Pass 2
state-9 -> exec-10 Frame -> state-10 -> exec-11 Discover -> state-11
  state-11 -> exec-12 Appraise Q3/D -> state-12 ---+
  state-11 -> exec-13 Appraise Q3/E -> state-13 ---+> exec-15 Synth Q3 -> state-15 ---+
  state-11 -> exec-14 Appraise Q4/F -> state-14 ----> exec-16 Synth Q4 -> state-16 ---+> exec-17 Design -> state-17

Pass 3
state-17 -> exec-18 Frame -> state-18 -> exec-19 Discover -> state-19
  state-19 -> exec-20 Appraise Q5/G -> state-20 ---+
  state-19 -> exec-21 Appraise Q5/H -> state-21 ---+> exec-25 Synth Q5 -> state-25 ---+
  state-19 -> exec-22 Appraise Q5/I -> state-22 ---+                               +-> exec-27 Design -> state-27
  state-19 -> exec-23 Appraise Q6/H -> state-23 ---+                               |
  state-19 -> exec-24 Appraise Q6/J -> state-24 ---+> exec-26 Synth Q6 -> state-26 ---+

```

The pass labels are explanatory only. There is no runtime pass object, split node, join gateway, active branch, or state-history cycle. exec-16 also demonstrates that the same Synthesis definition can publish

from one selected parent when one question has only one appraisal result. “Merge” describes the multi-parent runtime case, not a separate authored Step kind.

25.5.2 Literal handoff contents for all three passes

The following compact files are the complete handoff payloads for this worked trace. They demonstrate control, collection, and provenance semantics rather than claiming to be an adequate literature review.

Seed and pass 1: four pair members.

```
[research-agenda.md@seed-01]
Determine how a state-based Playbook can expand unknown research work,
run it concurrently, and merge it without authored routing nodes.

[research-questions.md@exec-01]
Q1. What immutable identity should ground one dynamically discovered job?
Q2. What provenance is sufficient to combine parallel results safely?

[source-map.md@exec-02]
A = planning operator grounding; B = event provenance and idempotency;
C = multi-parent version history. Retained pairs: Q1/A, Q1/B, Q2/B, Q2/C.

[appraisal-request-Q1-A.md@exec-02]
Evaluate operator grounding as an identity model for Q1. Result: appraisal-result-Q1-A.md.
[appraisal-request-Q1-B.md@exec-02]
Evaluate event provenance as an identity model for Q1. Result: appraisal-result-Q1-B.md.
[appraisal-request-Q2-B.md@exec-02]
Evaluate event provenance for safe combination under Q2. Result: appraisal-result-Q2-B.md.
[appraisal-request-Q2-C.md@exec-02]
Evaluate multi-parent histories for Q2. Result: appraisal-result-Q2-C.md.

[question-synthesis-request-Q1.md@exec-02]
Synthesize Q1 from exact members Q1/A and Q1/B. Result: question-synthesis-Q1.md.
[question-synthesis-request-Q2.md@exec-02]
Synthesize Q2 from exact members Q2/B and Q2/C. Result: question-synthesis-Q2.md.
[design-request.md@exec-02]
Integrate exact syntheses Q1 and Q2 into the state and execution model.

[appraisal-result-Q1-A.md@exec-03]
Grounded operator identity supports one instance per exact bound member.
[appraisal-result-Q1-B.md@exec-04]
Immutable producer, collection, and member keys support replay-safe identity.
[appraisal-result-Q2-B.md@exec-05]
Successful result provenance can witness exact request coverage.
[appraisal-result-Q2-C.md@exec-06]
Explicit selected parents distinguish considered candidates from lineage.

[question-synthesis-Q1.md@exec-07]
Identify work by producing Discover execution, collection, and exact member key.
[question-synthesis-Q2.md@exec-08]
Combine only successful exact result occurrences and record selected parents.

[design.md@exec-09]
Draft 1: freeze explicit collection members, late-ground one Appraise per member,
and require agent-declared parents for question and Design combination.
[research-agenda.md@exec-09]
R1. Test whether cardinality may vary without changing definitions.
R2. Test a question group with only one successful appraisal input.
```

Pass 2: three pair members.

```
[research-questions.md@exec-10]
Q3. Can a later Discover execution freeze a different member count safely?
Q4. Does the same Synthesis definition work with one selected parent?
```

```
[source-map.md@exec-11]
D = late grounding; E = durable work claims; F = one-source evidence policy.
Retained pairs: Q3/D, Q3/E, Q4/F.

[appraisal-request-Q3-D.md@exec-11]
Evaluate late grounding for Q3. Result: appraisal-result-Q3-D.md.
[appraisal-request-Q3-E.md@exec-11]
Evaluate durable claim identity for Q3. Result: appraisal-result-Q3-E.md.
[appraisal-request-Q4-F.md@exec-11]
Evaluate one-source synthesis semantics for Q4. Result: appraisal-result-Q4-F.md.
[question-synthesis-request-Q3.md@exec-11]
Synthesize Q3 from Q3/D and Q3/E. Result: question-synthesis-Q3.md.
[question-synthesis-request-Q4.md@exec-11]
Synthesize Q4 from Q4/F. Result: question-synthesis-Q4.md.
[design-request.md@exec-11]
Integrate exact syntheses Q3 and Q4 into the current design.

[appraisal-result-Q3-D.md@exec-12]
Definitions remain fixed while explicit collection members ground instances late.
[appraisal-result-Q3-E.md@exec-13]
One durable claim per exact member prevents descendant reevaluation duplicates.
[appraisal-result-Q4-F.md@exec-14]
One covered member is a complete group; multi-parent combination is unnecessary.
[question-synthesis-Q3.md@exec-15]
Cardinality belongs to each Discover expansion, not to the Playbook definition.
[question-synthesis-Q4.md@exec-16]
The Synthesis definition is valid with one parent when its frozen group has one member.

[design.md@exec-17]
Draft 2: support variable finite expansions and one- or multi-parent Synthesis.
[research-agenda.md@exec-17]
R3. Test a three-parent Synthesis and cross-question reuse of one source.
R4. Make the global appraisal barrier and group-local parent selection explicit.
```

Pass 3: five pair members and final completion.

```
[research-questions.md@exec-18]
Q5. Can one question merge three independently appraised sources?
Q6. Can one source participate in two question bindings without conflation?

[source-map.md@exec-19]
G = multi-source synthesis; H = shared-source question scoping;
I = contradictory evidence; J = merge-workspace materialization.
Retained pairs: Q5/G, Q5/H, Q5/I, Q6/H, Q6/J.

[appraisal-request-Q5-G.md@exec-19]
Evaluate G for Q5. Result: appraisal-result-Q5-G.md.
[appraisal-request-Q5-H.md@exec-19]
Evaluate H for Q5. Result: appraisal-result-Q5-H.md.
[appraisal-request-Q5-I.md@exec-19]
Evaluate I for Q5. Result: appraisal-result-Q5-I.md.
[appraisal-request-Q6-H.md@exec-19]
Evaluate H specifically for Q6. Result: appraisal-result-Q6-H.md.
[appraisal-request-Q6-J.md@exec-19]
Evaluate J for Q6. Result: appraisal-result-Q6-J.md.
[question-synthesis-request-Q5.md@exec-19]
Synthesize Q5 from Q5/G, Q5/H, and Q5/I. Result: question-synthesis-Q5.md.
[question-synthesis-request-Q6.md@exec-19]
Synthesize Q6 from Q6/H and Q6/J. Result: question-synthesis-Q6.md.
[design-request.md@exec-19]
Integrate exact syntheses Q5 and Q6 and decide whether research should continue.

[appraisal-result-Q5-G.md@exec-20]
Three exact result states can be authorized for one question merge.
[appraisal-result-Q5-H.md@exec-21]
Shared source identity does not erase the bound question identity.
[appraisal-result-Q5-I.md@exec-22]
Contradiction belongs in the synthesis rather than being hidden by readiness.
[appraisal-result-Q6-H.md@exec-23]
```

Q6/H is distinct from Q5/H because its request member and question differ.
 [appraisal-result-Q6-J.md@exec-24]
 Merge inputs require exact state-qualified paths or equivalent materialization.
 [question-synthesis-Q5.md@exec-25]
 Q5 is answered from three exact parents, with disagreement retained explicitly.
 [question-synthesis-Q6.md@exec-26]
 Q6 is answered from two exact parents; shared source H remains question-scoped.

 [design.md@exec-27]
 Draft 3: explicit expansions, late grounding, conservative global readiness,
 group-local exact parent selection, and multi-parent Design form one state DAG.
 [research-complete.md@exec-27]
 The exemplar now covers variable fan-out, isolated siblings, one-/two-/three-parent
 Synthesis, final merge, recurrence, and quiescence without hidden control flow.

25.5.3 Eligibility and coverage after every publication

Assume numerical completion order only to make the trace readable. The Appraise executions in each wave may finish in any order. Every sealed sibling wakes evaluation, but the exact member claims created from its Discover expansion prevent duplicate automatic grounding on descendant states.

State	Newly eligible work	Exact reason
state-0	Frame 01	G@seed-01 exists and has no automatic Frame claim.
state-1	Discover 02	Q@01 exists and has no automatic Discover claim.
state-2	Appraise 03-06	K02 freezes four exact members; one execution is grounded for each.
state-3	none new	K02 coverage is 1/4; the other members are already grounded or claimed.
state-4	none new	Coverage is 2/4. Q1 is locally covered, but the conservative global barrier remains false.
state-5	none new	Coverage is 3/4; no Synthesis starts early.
state-6	Synthesize 07, 08	Coverage reaches 4/4; both frozen question groups may now run.
state-7	none new	Synthesize 08 is already grounded or running; Design lacks Y2@08.
state-8	Design 09	Z@02, Y1@07, Y2@08 form one new complete Design binding.
state-9	Frame 10	Design published G@09; its exact Frame binding has no claim.
state-10	Discover 11	Q@10 exists and has no automatic Discover claim.
state-11	Appraise 12-14	K11 freezes three exact members.
state-12	none new	K11 coverage is 1/3; remaining members are already grounded or claimed.
state-13	none new	Coverage is 2/3; the global barrier remains false.
state-14	Synthesize 15, 16	Coverage reaches 3/3; Q3 and single-member Q4 are ready.
state-15	none new	Synthesize 16 is already grounded or running; Design lacks Y4@16.
state-16	Design 17	Z@11, Y3@15, Y4@16 form one new complete Design binding.
state-17	Frame 18	Design published G@17; its exact Frame binding has no claim.
state-18	Discover 19	Q@18 exists and has no automatic Discover claim.
state-19	Appraise 20-24	K19 freezes five exact members.
state-20	none new	K19 coverage is 1/5; remaining members are grounded or claimed.
state-21	none new	Coverage is 2/5.
state-22	none new	Coverage is 3/5; Q5 is locally covered but the global barrier remains false.
state-23	none new	Coverage is 4/5; no Synthesis starts early.
state-24	Synthesize 25, 26	Coverage reaches 5/5; both question groups may now run.
state-25	none new	Synthesize 26 is already grounded or running; Design lacks Y6@26.
state-26	Design 27	Z@19, Y5@25, Y6@26 form one new complete Design binding.
state-27	none; quiescent	Design published C@27 but no new agenda. Frame 18 already claimed the exact G@17 binding.

Requirements exposed by fan-out and merge. Discover must publish one explicit finite collection record; the engine must ground and durably claim one Appraise execution per exact member; coverage must be derived from successful result provenance; and a merge Session must distinguish authorized candidates from the exact parent subset declared at accepted completion. None of these facts may be reconstructed from filenames, mutable counters, wall-clock order, or which states an agent happened to inspect.

25.5.4 How the outer loop advances and stops

exec-09 publishes G@09, making Frame applicable from state-9. exec-17 similarly publishes G@17 from state-17. The third Design execution publishes D@27 and C@27 but no new agenda occurrence. state-27 still inherits G@17; Frame's automatic execution claim for that exact binding was created by exec-18, so it cannot ground a duplicate fourth pass.

All 27 automatic activations enumerated above have been grounded and completed in this trace. Their durable exact-binding claims and successful-execution provenance keep the preserved ancestor and sibling states from silently relaunching the same work. The exact claim key and provenance scope remain open. The Playbook is quiescent rather than terminated: a later human or agentic transition may publish another agenda occurrence from any selected state and ordinary evaluation may begin another branch.

25.6 Details intentionally left open

This trace fixes behavior, not final serialization. It leaves open the portable syntax for collection declarations, symbolic member binding, group readiness, and result relationships; the durable automatic-claim key; provenance scope across sibling branches; failure and retry policy; candidate freezing and merge workspace materialization; internal Git refs and checkpoint storage; and the final structured `step_complete` wire format.

Those open details do not weaken the demonstrated commitments: one Discover success freezes a finite expansion; one exact request grounds one Appraise execution; all appraisal members from that expansion must succeed before V1 question Synthesis starts; each question Synthesis selects only its declared group parents; Design selects the exact question-synthesis parents it uses; every accepted execution publishes one immutable successor; and a new agenda occurrence is the only mechanism that advances the outer loop.

25.7 Collection-driven continuation and a smaller merge contract

The parallel trace permits a simpler continuation convention than mutually exclusive approval, revision, continuation, or completion marker artifacts. Develop always publishes `design.md` and must also declare one `followup-research` collection, which may be empty. Every member is one exact primary Markdown handoff occurrence recorded in the immutable state manifest. A nonempty collection supplies downstream Frame work; an explicitly empty collection supplies none, so the Playbook becomes quiescent. The empty declaration distinguishes an intentional lack of follow-up work from a missing output. An approval artifact remains useful only when approval is itself a domain record, such as a human or regulatory sign-off; it is not an execution primitive.

This convention supersedes the provisional `research-agenda.md` versus `research-complete.md` choice in the worked records above. Human-readable reasons for continuing or stopping belong in `design.md` or in the emitted follow-up handoffs. The engine does not need routing markers such as `approved.md` or `needs-revision.md`.

```
outputs:
  required: [design.md]
  collections:
    required:
      - followup-research

# One downstream Frame execution per exact member.
followup-research: [handoff-1, handoff-2, ...]

# No downstream Frame execution; the Playbook is quiescent.
followup-research: []
```

The collection member is the exact handoff artifact occurrence, not a separate “work item” object. The bound artifact is the Step execution’s primary handoff, but its Sessions may inspect the complete input state. The engine iterates the collection recorded in the manifest; it does not discover members with a filename glob, regular expression, or Markdown parsing. The ordinary fan-out form is therefore:

```
inputs:
  request:
    for_each: appraisal-requests
```

For each exact member of `appraisal-requests`, this declaration enumerates one candidate Appraise application with that member bound as `request`. Once the complete binding and readiness conditions resolve and the engine acquires the automatic claim, it creates the Appraise Step execution. Scheduling controls when that execution runs, and only accepted `step_complete` publishes its one successor state.

25.7.1 Full question-Synthesis definition

The same collection model removes the provisional authored `resolved_binding` and per-Step merge blocks. A reduced, self-contained question-Synthesis definition is:

```
- id: synthesize-question
  title: Synthesize One Question
  revision: 1

  harness: codex
  model: gpt-5.6-sol
  settings:
    reasoning_effort: high

  inputs:
    question:
      for_each: question-synthesis-requests
    appraisals:
      complete_set_for: question.requires_appraisals

  when:
    all_fulfilled: question.discover_expansion.appraisal_requests

  outputs:
    synthesis:
      required: true
      artifact_from: question.result_artifact
      member_of: question-syntheses
      fulfills: question

  instructions: |
    Synthesize the bound research question using every appraisal result
    supplied in `appraisals`.

    Compare the evidence, identify agreements and disagreements, assess
    whether the question is answered, partially answered, or unanswered,
    and record any remaining uncertainty.

    Write the synthesis to the artifact path specified for `synthesis`.
```

The first input performs late finite expansion: one candidate Synthesis application is enumerated for each exact member of `question-synthesis-requests`. The engine creates the Step execution only after the second input binds the complete successful result set fulfilling that question's required appraisal members, the global readiness condition holds, and the automatic claim succeeds. Every Appraise result manifest records which exact request member it fulfills, so the engine resolves this set from immutable provenance rather than filenames.

The `when` clause preserves the conservative global V1 barrier. Every Appraise request emitted by the same Discover expansion must have a successful result before any question Synthesis starts. This revealed an important distinction that the reduced schema must preserve:

```
readiness gate:
  every appraisal request from this Discover expansion

bound merge inputs:
  only the appraisal results required by this question
```

Thus Q2 results may help make `Synthesize(Q1)` eligible without becoming Q1 inputs or parents. From the question-local `complete_set_for` relationship, the engine derives the exact input binding, authorized candidate states, and required coverage. At `step_complete`, the agent supplies the exact selected parent-state identifiers; the engine validates that they are authorized and cover the required request members before sealing the successor. This model-wide completion rule need not be repeated in every merge-capable Step definition.

When one Synthesis consumes the entire collection, readiness and binding are the same relationship and the definition becomes smaller still:

```
inputs:
  appraisals:
    complete_set_for: appraisal-requests
```

No separate when clause is needed in that case. The two reusable collection patterns are therefore:

Collection-driven execution.

For each member of collection C , execute Step X with that member.

After a complete result set exists for collection C , execute Step Y with those results.

26 Semantic reference algorithm for the Playbook execution engine

This section turns the settled model into executable-style pseudocode while leaving storage, retry, and scheduler policies abstract. It specifies the semantic order of operations: evaluate immutable state, enumerate candidate applications, resolve exact bindings, test readiness, acquire a durable claim, run an atomic Step execution through one or more Sessions, and publish exactly one immutable successor only after accepted completion.

There is no next-action selector. A sealed state wakes evaluation; it does not become an active state or move a hidden cursor. Evaluation ranges over the immutable state and provenance index, discovers every currently applicable exact binding, and may queue all successfully claimed applications. Concurrency limits affect dispatch timing, not logical applicability.

26.1 Inputs and runtime records

Let P denote one validated, immutable Playbook definition version, and let $\text{Steps}(P)$ be the finite set of fixed Step-definition versions it contains. One such definition is $d \in \text{Steps}(P)$. A Playbook revision is therefore a definition version: it changes when authored semantics change, not when the Playbook executes, loops, retries, or publishes a state.

Let s denote one sealed task state. A sealed state s is durably represented by one engine-generated immutable manifest recording its identity, parents, workspace checkpoint, visible artifact occurrences, publishing execution, and provenance. Lowercase s denotes one state; uppercase S remains the set of sealed states in the abstract history $H = (S, R, \Pi)$. The runtime algorithms query an append-only index $\mathcal{H}$ representing that history:

$$d \in \text{Steps}(P), \quad \text{History}(\mathcal{H}) = H = (S, R, \Pi), \quad s \in S. \quad (27)$$

For a read-only history snapshot I , evaluation derives a candidate grounding g for d . Exact input resolution either waits or returns (b, A, K) , where b is the exact artifact binding, A is the set of authorized merge-candidate state identifiers, and K is the required coverage set:

$$I = \text{SnapshotOfSealedHistory}(\mathcal{H}), \quad \text{ResolveExactInputs}(d, g, I) \in \{\text{waiting}\} \cup \{(b, A, K)\}. \quad (28)$$

Once readiness holds, the engine creates one durable Step execution e . The store $\mathcal{C}$ holds all Step-execution records and any associated automatic claims; an explicitly requested manual execution has a record but no automatic claim. While a fully bound, durably created execution awaits dispatch, $\mathcal{Q}$ contains a reference to it. Execution e may span several sequential Sessions, but accepted completion can publish at most one successor s' , which is then added to $\mathcal{H}$.

Symbol	Meaning and example
<i>P</i>	One validated, immutable Playbook definition version containing fixed Step-definition versions. <i>Example:</i> <code>parallel-research-appraisal@1</code> fixes the Discover, Appraise, Synthesis, and Develop definitions before Discover emits any concrete question-source pairs.
<i>d</i>	One fixed Step-definition version belonging to <i>P</i> . <i>Example:</i> <code>appraise-pair@1</code> consumes one exact appraisal request and must publish the matching appraisal result.
<i>s</i>	One sealed task state, durably represented by its immutable state manifest. <i>Example:</i> <code>state-3</code> records parent <code>state-2</code> , creating execution <code>exec-03</code> , checkpoint <code>checkpoint-003</code> , and result occurrence <code>A1A@03</code> .
<i>H</i>	The task's append-only index of immutable sealed state manifests, artifact occurrences, collection memberships, fulfillment relations, and lineage. <i>Example:</i> it records that <code>state-2</code> froze <code>appraisal-request-Q1-A.md</code> in collection occurrence <code>K02</code> , and that <code>state-3</code> later published its fulfilling result.
<i>I</i>	One read-only snapshot of the sealed history used throughout a single evaluation pass. <i>Example:</i> after <code>state-6</code> is sealed, the engine evaluates every Step definition against a snapshot containing <code>state-0</code> through <code>state-6</code> .
<i>g</i>	A candidate grounding: enough exact state or collection context to try resolving one application of <i>d</i> . <i>Example:</i> the Q1 synthesis request <code>T1@02</code> identifies a possible <code>synthesize-question@1</code> application even before every required Q1 appraisal result exists.
<i>b</i>	One complete exact artifact input binding. Authorized merge candidates and coverage obligations are returned separately as <i>A</i> and <i>K</i> . <i>Example:</i> Q1 Synthesis binds exactly <code>T1@02</code> , <code>A1A@03</code> , and <code>A1B@04</code> .
<i>A</i>	The exact set of authorized merge-candidate state identifiers for one execution. <i>Example:</i> Q1 Synthesis may select <code>state-3</code> and <code>state-4</code> , which supply <code>A1A@03</code> and <code>A1B@04</code> .
<i>K</i>	The exact collection members whose successful-result coverage is required by one execution. <i>Example:</i> Q1 Synthesis requires coverage of <code>P1A@02</code> and <code>P1B@02</code> from frozen collection <code>K02</code> .
<i>e</i>	One engine-owned Step execution, which may contain several sequential Sessions but may publish at most one successor state. <i>Example:</i> <code>exec-07</code> may run Q1 Synthesis through several Sessions but can seal at most one successor, <code>state-7</code> .
<i>C</i>	The durable store of Step-execution records and associated automatic-application claims. <i>Example:</i> it stores the claim to apply <code>appraise-pair@1</code> to occurrence <code>P1A@02</code> together with the resulting <code>exec-03</code> record.
<i>Q</i>	The scheduler's queue of references to fully bound, durably created Step executions. <i>Example:</i> it may hold <code>exec-03</code> through <code>exec-06</code> until capacity is available to dispatch the four independent Appraise executions.

The table names symbols that cross procedure boundaries or belong to the formal runtime model. Procedure-local intermediate values use descriptive names at their first assignment instead of receiving separate notation rows.

A candidate grounding is derived data, not a partially created Step execution. This resolves a subtle ordering issue exposed by `for_each`: a collection member may identify a possible Synthesis application before its `complete_set_for` results or global readiness barrier exist. The engine creates the durable execution only after the complete exact binding resolves and the automatic claim succeeds.

Algorithm 1: Reactive evaluation, durable claiming, and dispatch

```

1 procedure PlaybookEngine( $P, \mathcal{H}, \mathcal{C}, \mathcal{Q}$ ):
2   Evaluate( $P, \mathcal{H}, \mathcal{C}, \mathcal{Q}$ )
3   while  $P$  remains attached to the task do
4     DispatchQueued( $\mathcal{Q}$ )
5     event  $\leftarrow$  AwaitEngineEvent()
6     if event publishes a sealed state then
7       add event.state to  $\mathcal{H}$ 
8       Evaluate( $P, \mathcal{H}, \mathcal{C}, \mathcal{Q}$ )
9     else if event explicitly requests a manual execution then
10       $e \leftarrow$  CreateManualExecution(event)
11      enqueue  $e$  in  $\mathcal{Q}$                                      // no automatic claim

12 procedure Evaluate( $P, \mathcal{H}, \mathcal{C}, \mathcal{Q}$ ):
13    $I \leftarrow$  SnapshotOfSealedHistory( $\mathcal{H}$ )
14   foreach Step-definition revision  $d \in \text{Steps}(P)$  do
15     foreach  $g \in \text{Groundings}(d, I)$  do
16       resolution  $\leftarrow$  ResolveExactInputs( $d, g, I$ )
17       if resolution = waiting then
18         continue
19        $(b, A, K) \leftarrow$  resolution                          // binding, candidates, coverage
20        $(\text{isReady}, \text{readinessWitness}) \leftarrow$  EvaluateWhen( $d, g, b, I$ )
21       if  $\neg \text{isReady}$  then
22         continue
23        $e \leftarrow$  ClaimAndCreate( $P, d, b, A, K, \text{readinessWitness}, \mathcal{C}$ )
24       if  $e \neq \perp$  then
25         enqueue  $e$  in  $\mathcal{Q}$ 

26 procedure DispatchQueued( $\mathcal{Q}$ ):
27   while scheduler capacity remains and  $\mathcal{Q} \neq \emptyset$  do
28      $e \leftarrow$  ChooseQueuedBySchedulerPolicy( $\mathcal{Q}$ )
29     if CompareAndSet( $e$ , queued, starting) then
30       dispatch RunExecution( $e$ ) asynchronously
  
```

`ClaimAndCreate()` is one atomic durable operation. It derives the automatic application identity from the Playbook revision, Step-definition revision, and exact binding; refuses an already claimed application according to the selected claim policy; and otherwise persists the claim and new queued execution together. Its exact key, provenance scope, crash recovery, and failure/retry semantics remain open. Explicit manual execution receives a fresh execution identifier and bypasses automatic duplicate suppression, but uses the same isolation and sealing path.

26.2 Exact collection grounding and complete-set resolution

Algorithm 2: Enumerate candidate groundings and resolve exact inputs

```

1 procedure Groundings(d, I):
2   groundings  $\leftarrow$  ExactStateLocalGroundings(d, I)
3   foreach input alias input declaring for_each: collectionName do
4     memberBindings  $\leftarrow$  { input  $\mapsto$  member | collection  $\in$ 
        Collections(collectionName, I), member  $\in$  collection.members }
5     groundings  $\leftarrow$  ProvenanceJoin(groundings, memberBindings)
6   foreach unbound concrete collection collectionName named only by complete_set_for do
7     groundings  $\leftarrow$  ProvenanceJoin(groundings, { collectionScope  $\mapsto$  collection |
        collection  $\in$  Collections(collectionName, I) })
8   return groundings

9 procedure ResolveExactInputs(d, g, I):
10  b  $\leftarrow$  BindExactOrdinaryAndForEachInputs(d, g, I)
11  A  $\leftarrow$   $\emptyset$  // authorized result-state candidates
12  K  $\leftarrow$   $\emptyset$  // required request-member coverage
13  foreach input alias input declaring complete_set_for: collectionRef do
14    requiredMembers  $\leftarrow$  ResolveFrozenMembers(collectionRef, g, b, I)
15    resultCover  $\leftarrow$  ResolveAuthorizedCover(requiredMembers, I)
16    if resultCover = incomplete then
17      return waiting
18    bind input to the exact successful result occurrences in resultCover
19    A  $\leftarrow$  A  $\cup$  SupplyingStateIds(resultCover)
20    K  $\leftarrow$  K  $\cup$  requiredMembers
21  return (b, A, K)

```

ExactStateLocalGroundings resolves ordinary named inputs only within one sealed state. A pure collection-driven definition begins with the neutral binding. Every `for_each` member comes from one explicit frozen collection record in a sealed manifest; no glob, regular expression, filename prefix, or Markdown parser may create membership. Repeated keys in another collection occurrence remain distinct because the collection identity includes its producing execution.

ResolveExactInputs() deliberately does not evaluate the authored `when` condition. This preserves the distinction established by Variation 3: EvaluateWhen() may require successful coverage of an entire Discover expansion, while `complete_set_for` binds only the exact results required by the assigned question. Readiness witnesses never become input artifacts, authorized merge candidates, or parent states merely because they made the condition true.

26.3 Atomic Step execution across sequential Sessions

Algorithm 3: Run one atomic Step execution

```

1 procedure RunExecution(e):
2   workspace ← MaterializeIsolatedWorkspace(e.binding, e.authorizedCandidateStateIds)
3   session ← StartSession(e, workspace); append session to e.sessions
4   while e is open do
5     event ← AwaitExecutionEvent(e)
6     if event = step_continue then
7       yield the current write-capable Session
8       session ← StartContinuationSession(e, workspace); append session to e.sessions
9     else if event = step_complete(completion) then
10      sealResult ← CompleteAndSeal(e, workspace, completion)
11      if sealResult = accepted then
12        return
13      deliver validation errors and resume the continuable execution
14    else if event = step_fail(reason) then
15      durably record failure outside the successful state DAG
16      close e without publishing a state
17      return
18    else if the current Session exits without a decision then
19      mark e as awaiting explicit continuation
20      await a user or agent continuation request
21    session ← StartContinuationSession(e, workspace); append session to e.sessions

```

Only one Session in *e.sessions* may be active and write-capable at a time. Every continuation receives the same exact binding, unpublished workspace, generated runtime context, and original authored instructions plus a continuation instruction. It does not reevaluate applicability, acquire another claim, create a retry, or publish an intermediate state. Idle, process exit, or end of turn never implies completion.

26.4 Validated completion and engine-owned state publication

Algorithm 4: Validate completion and atomically seal one successor state

```

1 procedure CompleteAndSeal(e, workspace, completion):
2   acquire e's completion lock and serialize against continuation
3   suspend writes from every Session belonging to e
4   if e.resultState  $\neq \emptyset$  then
5     return accepted
6   if  $\neg$ ValidOutputsAndCollections(e, workspace, completion) then
7     restore write access; return rejected
8   if e has mechanically fixed ordinary parentage then
9     parents  $\leftarrow \{e.ordinaryParentStateId\}$ 
10  else
11    parents  $\leftarrow completion.selectedParentStateIds$ 
12    if parents  $\not\subseteq e.authorizedCandidateStateIds$  or
13       $\neg$ Covers(parents, e.binding, e.requiredCoverage) then
14      restore write access; return rejected
15  checkpoint  $\leftarrow$  CheckpointCompleteWorkspace(workspace)
16  occurrences  $\leftarrow$  NewOccurrences(completion.publish, e, checkpoint) // bytes may match
17  collections  $\leftarrow$  ValidateAndFreezeCollections(completion.collections, occurrences)
18  s'  $\leftarrow$  BuildFullStateManifest(parents, e, checkpoint, occurrences, collections,
19    e.binding, e.sessions)
20  if  $\neg$ AtomicPublish(s', e) then
21    restore write access; return rejected
22  mark e successful and immutable; stop all participating Sessions
23  emit StateSealed(s')
24  return accepted

```

ValidOutputsAndCollections requires every declared output and every required collection declaration. An explicitly empty required collection is valid and freezes zero members; a missing declaration is invalid. New artifact occurrences are assigned only to explicitly published paths, even when their bytes did not change. Unrepublished inherited artifacts retain their existing occurrences.

For an ordinary execution, parentage is mechanical. For a merge-capable execution, the agent supplies the exact selected parent-state identifiers at `step_complete`; the engine accepts them only if they are authorized and collectively cover the locally bound merge inputs. One selected parent is valid when it provides complete coverage. States used only as global-readiness witnesses are never added. The engine, not the agent, checkpoints the workspace, constructs the full immutable manifest, and makes the new state visible. Validation, checkpoint, or publication failure exposes no state and no successful fulfillment provenance, leaving a rejected execution continuable.

26.5 Collection-driven continuation, quiescence, and invariants

When Develop completes, it must publish `design.md` and explicitly declare `followup-research`. A nonempty collection is frozen into the successor manifest and later `for_each` evaluation enumerates one Frame candidate per exact member. An empty collection enumerates none. If no configured work is currently eligible or running, the Playbook is quiescent, not successful or terminated. Whether an incomplete but non-running continuable execution changes the UI's quiescence label remains open.

1. Every sealed state is immutable, full, independently valid, and explicit in the state DAG.
2. One accepted Step execution publishes exactly one successor state; explicit failure publishes none.
3. Intermediate workspace changes are invisible to Playbook evaluation.
4. Concurrent executions use isolated workspaces; sibling successes never combine implicitly.
5. Every Session belongs to one Step execution, and participating Sessions are sequential.
6. Ordinary success has one mechanically known parent. Merge success records every and only the agent-selected authorized parents.
7. Candidate exposure and readiness evidence are not lineage.
8. Collection membership and fulfillment relationships come from immutable manifest provenance.
9. Automatic claim acquisition and execution creation are one atomic durable operation.
10. A state publication wakes evaluation but creates no active frontier, cursor, authored edge, split, join, or loop object.

26.6 Mechanisms intentionally abstract

The algorithms above name but do not silently decide:

- the serialized trigger, collection, binding, and completion-wire syntax;
- the exact automatic-claim key, sibling provenance scope, failure retention, retry policy, and crash recovery;
- which exact result set is authorized when several successful executions fulfill one request member;
- merge-candidate construction, candidate-version policy, multi-checkpoint workspace materialization, and conflict presentation;
- whether `complete_set_for` over an empty collection is invalid, vacuously ready, or requires an anchor parent;
- which sealed state supplies the ordinary launch workspace when the same exact inherited occurrence is visible in several descendants;
- the Git object/ref layout and physical atomic-publication mechanism;
- scheduler capacity, fairness, and resource policy;
- manual filesystem-state capture UX and external side effects outside the sealed workspace.

A Executable Playbook specimens

The following specimens turn the motivating process graphs into complete, inspectable Playbook files. Each listing is included directly from its Markdown source so the report and the working specimen cannot drift apart.

A.1 Linear research to design

The linear specimen begins with `research-brief.md` and produces, in sequence, framed questions, a source map, an evidence synthesis, and a design. Its YAML front matter supplies the executable Step configuration, while its Markdown sections supply the corresponding agent instructions.

```

1 ---
2 schema: alinery.playbook/sketch-0

4 playbook:
5   id: linear-research-to-design
6   title: Linear Research to Design
7   revision: 1
8   description: >-
9     Turn one research brief into framed questions, a source map, an evidence
10    synthesis, and an evidence-grounded design in a single pass.

12 steps:
13   - id: frame-questions
14     title: Frame / Refine Questions
15     revision: 1
16     harness: codex
17     model: gpt-5.6-sol
18     settings:
19       reasoning_effort: high
20     when: 'Exists("research-brief.md")'
21     outputs: [research-questions.md]
22     instructions: "#frame-questions"

24   - id: discover-sources
25     title: Discover Sources
26     revision: 1
27     harness: codex
28     model: gpt-5.6-sol
29     settings:
30       reasoning_effort: high
31     when: 'Exists("research-questions.md")'
32     outputs: [source-map.md]
33     instructions: "#discover-sources"

35   - id: appraise-and-synthesize
36     title: Appraise + Synthesize Evidence
37     revision: 1
38     harness: codex
39     model: gpt-5.6-sol
40     settings:
41       reasoning_effort: high
42     when: 'Exists("source-map.md")'
43     outputs: [evidence-synthesis.md]
44     instructions: "#appraise-and-synthesize"

46   - id: develop-design
47     title: Develop / Revise Design
48     revision: 1
49     harness: codex
50     model: gpt-5.6-sol
51     settings:
52       reasoning_effort: high
53     when: 'Exists("evidence-synthesis.md")'
54     outputs: [design.md]
55     instructions: "#develop-design"

```

```

56 ---
57
58 # Linear Research to Design
59
60 Turn one research brief into focused research questions, a map of relevant
61 sources, an evidence synthesis, and an evidence-grounded design. Begin with
62 `research-brief.md` in the task state.
63
64 ## Process
65
66 ```text
67 research-brief.md
68   -> Frame / Refine Questions -> research-questions.md
69   -> Discover Sources         -> source-map.md
70   -> Appraise + Synthesize    -> evidence-synthesis.md
71   -> Develop / Revise Design  -> design.md
72 ```
73
74 <a id="frame-questions"></a>
75 ## Frame / Refine Questions
76
77 Read `research-brief.md` as the primary handoff. Turn the initial idea into a
78 set of precise, answerable research questions. Do not perform the
79 literature search or attempt to answer the questions in this Step.
80
81 Write `research-questions.md` with:
82
83 1. A concise restatement of the problem and intended design decision.
84 2. Stable question identifiers such as `Q1`, `Q2`, and `Q3`.
85 3. For each question:
86   - the exact question;
87   - why answering it matters;
88   - what an adequate answer would need to establish;
89   - important scope boundaries and exclusions;
90   - assumptions that should be tested rather than silently accepted.
91 Keep the set focused. Split a question when its parts require different kinds
92 of evidence. Combine questions that would be answered by the same evidence.
93
94 The Step is complete only when `research-questions.md` is coherent enough for a
95 different agent to discover sources without reconstructing the original intent.
96
97 <a id="discover-sources"></a>
98 ## Discover Sources
99
100 Read `research-questions.md` as the primary handoff. Find plausible sources for
101 each question and map every source to the question or questions it might inform.
102 This Step discovers candidates; it does not claim that a source is credible,
103 relevant enough, or sufficient to answer a question.
104
105 Prefer primary literature, standards, official technical documentation, and
106 other sources appropriate to the domain. Preserve enough bibliographic detail
107 and a stable locator for another agent to retrieve every source.
108
109 Write `source-map.md` with:
110
111 1. The search scope, terminology, and important exclusions.
112 2. A section for each stable question identifier.
113 3. For every candidate source:
114   - title and authors or issuing organization;
115   - publication venue and date when available;
116   - DOI, canonical URL, or other stable locator;
117   - source type;
118   - the question it may inform;
119   - a brief reason it appears plausibly relevant;
120   - any access or retrieval limitation.
121 4. Questions for which no plausible source was found.
122
123 Avoid unsupported relevance claims and do not synthesize conclusions yet.
124
125 <a id="appraise-and-synthesize"></a>
126 ## Appraise + Synthesize Evidence

```

```

128 Read `source-map.md` as the primary handoff. Retrieve and examine the mapped
129 sources, judge their relevance and credibility, and synthesize the available
130 evidence question by question. You may inspect `research-questions.md` and the
131 rest of the input state for context.

133 Write `evidence-synthesis.md` with:

135 1. A short account of the appraisal method and important limitations.
136 2. A section for every stable question identifier.
137 3. For each question:
138     - status: `answered`, `partially answered`, or `unanswered`;
139     - the best-supported answer presently available;
140     - the evidence supporting each material claim;
141     - source relevance, credibility, and limitations;
142     - agreements, contradictions, and meaningful absences in the evidence;
143     - uncertainty and unresolved assumptions;
144     - remaining or newly exposed research questions.
145 4. A final list of findings that are sufficiently supported to inform design.

147 Distinguish source findings from your own inference. Do not conceal conflicting
148 evidence merely to produce a cleaner conclusion.

150 <a id="develop-design"></a>
151 ## Develop / Revise Design

153 Read `evidence-synthesis.md` as the primary handoff. Apply the supported findings
154 to the object being designed. You may inspect the complete input state, but do
155 not rewrite the upstream research handoffs.

157 Write `design.md` with:

159 1. The problem and design objectives.
160 2. The proposed design, described concretely enough for review or planning.
161 3. Each material design decision and the evidence or explicit assumption behind
162    it.
163 4. Alternatives considered and why they were not selected.
164 5. Risks, limitations, and consequences of being wrong.
165 6. Open design questions and open research questions.
166 7. Recommended next actions.

168 Do not present a partially answered research question as settled. Record all
169 remaining research and design questions clearly in `design.md` for future work.

```

Listing 3: Complete linear research Playbook specimen.